



Ben Gurion University of the Negev
Faculty of Engineering Science

Department of Mechanical Engineering

Final report – Designing and Implementing a High-level path planning and control system for an autonomous racer

Student Name: Yuval Grimberg 315823328

25-87

12/06/25

Supervisor: Dr. Igal Bilik

Abstract

This project focuses on the development of a High-Level Path Planning and control system to an autonomous electric race car for participation in the driverless category of the Formula Student competition. The primary goal is to implement robust control and path planning architecture that ensures precise navigation and optimal performance within the constraints of the competition. By Leveraging simulation tools such as AirSim, the team aims to validate the system's performance under diverse conditions and transition to real-world testing. This initiative not only showcases engineering innovation but also serves as a platform for students to bridge the gap between theoretical knowledge and practical application, ultimately pushing the boundaries of autonomous driving technologies.

Since we can't guarantee that it would be possible to implement the code into the actual car, the scope of this project will be mainly simulator-based with preparation for real-world application. However, as of right now, the project was almost fully integrated into the autonomous test car and the car handled as planned.

the main goal of the team is to create the first Israeli Autonomous electric Formula Student vehicle that would be able to compete.

Table of Figures

Figure 1 : Formula Student competitions around the world 6
Figure 2 : 2022 Croatia FSAE Competition 8
Figure3 : The architecture of the autonomous system 9
Figure 4 : Trackdrive Track layout 10
Figure 5 : Acceleration Track layout 10
Figure 6 : Skipad Track layout 11
Figure 7 : Example of cone detection from the Innoviz one lidar From a BGR test run 13
Figure 8 : Yolo V1 Representation 13
Figure 9 : Exemplary cone detection 14
Figure 10 : Illustration of Kalman filter. [5] 15
Figure 11 : examples of path planning algorithms 16
Figure 12 : reference frame of the car 17
Figure 13 : Illustration of the Dynamic Bicycle model 19
Figure 14 : Stanley and Pure Pursuit controller schematic diagrams. 21
Figure 15 : Pure Pursuit (right) and Stanley (left) controllers geometric relationship 21
Figure 16 : High level Overview of the GNC pipeline. 23
Figure 17 : Detailed presentation of High-Level Path planning and control system. 23
Figure 18 : Architecture of Vehicle Control strategy 24
Figure 19 : GUI from the simulator at action. 26
Figure 20 : System overview of the simulator from [18]. 27
Figure 21 : Client API connection example from [18]. 27
Figure 22 : Examples of Fetching Ground Truth Data from [18]. 28
Figure 23 : Depth camera output to simulate the LIDAR 28
Figure 24 : Examples of the visualization of the path planner in action. 30
Figure 25 : Basic usage of the Path Planner from [19]. 30
Figure 26 : Steering controllers Path tracking comparison, Stanley (Left) and PP (Right) 34
Figure 27 : Frames from an actual run with without location info 35
Figure 28 : PID Tunning - CW Are P, PD, PI and PID 36
Figure 29 : Plots for Map 1. 37
Figure 30 : Plots for Map 2. 38
Figure 31 : Plots for map 3. 39
Figure 32 : Screenshot from the runtime of map 3. 40
Figure 33 : Frame from the Skipad track. 40
Figure 34 : Skipad plots. Path tracking (right) and Speed profile (left) 41
Figure 35 : Acceleration Plots on a test run of 10 [m/s] 41
Figure 36 : Acceleration plots at 14 [m/s] 42
Figure 37 : Running the simulator and the test vehicle to test real world integration 44

List OF Tables

Table 1 : Points Awarded for Each segment of the competition. 7
Table 2: Comparison of control methods 20
Table 3 : Comparison of the lateral controllers with their steering equations 32
Table 4 : PID GAINS 35
Table 5 : Lap times in the runs and fallen cones 42
Table 6 : Run time of read and update nodes. 43
Table 7 : Future development plan 46
Table 8 : Risk analysis table 48

List OF Symbols

Symbol	Description	Units
X	Global position in the X-axis	$[m]$
Y	Global position in the Y-axis	$[m]$
θ	Yaw Angle between bicycle frame to X-Axis	$[rad]$
β	Vehicle slip angle (angle between velocity vector and vehicle longitudinal axis)	$[rad]$
v	Speed of the car (magnitude of velocity vector)	$[\frac{\mathrm{m}}{\mathrm{s}}]$
v_x	Longitudinal velocity	$[\frac{\mathrm{m}}{\mathrm{s}}]$
v_y	Lateral velocity	$[\frac{\mathrm{m}}{\mathrm{s}}]$
f_x	Longitudinal force acting on the vehicle	[N]
f_y	Lateral force acting on the vehicle	[N]
M_z	Yaw moment (torque about the z-axis)	$[\mathrm{N} \cdot \mathrm{m}]$

m	Mass of the car	$[kg]$
I_z	Moment of inertia	$[kg \cdot m^2]$
$\dot{x}$	Global velocity in the X direction	$\left[\frac{\mathrm{m}}{\mathrm{s}}\right]$
$\dot{y}$	Global velocity in the Y direction	$\left[\frac{\mathrm{m}}{\mathrm{s}}\right]$
$\dot{\theta}$	Yaw rate (rate of change of heading)	$\left[\frac{\mathrm{rad}}{\mathrm{s}}\right]$
δ	Steering angle (wheel input angle)	$[rad]$
$T_{accelaration}$	High level Throttle and brakes command	-

Table of Contents

Abstract 1
Table of Figures 2
List OF Tables 3
List OF Symbols 3
1. Introduction 6
1.1. BGRacing Team 8
1.1.1. BGR Autonomous Team 8
1.2. Driverless dynamic events 9
1.3. Scope and Objectives 11
2. Literature Review and Theoretical Background 12
2.1. Perception Mapping (PM) 12
2.1.1. LiDAR (Light Detection and Ranging): 12
2.1.2. Cameras: 13
2.1.3. IMU (Inertial Measurement Unit): 14
2.1.4. GNSS (Global Navigation Satellite System): 14
2.1.5. Sensor Fusion and State Estimation 14
2.2. Classical Path Planning Algorithms 15
2.2.1. RRT (Rapidly Exploring Random Tree): 15
2.2.2. Triangulation and Voronoi Diagrams: 15
2.3. Vehicle Dynamics 16
2.3.1. Kinematic Bicycle Model 17
2.3.2. Dynamic Bicycle Model 18
2.4. Control 19
2.4.1. Coupled vs. Decoupled Control: 19

2.4.2. Longitudinal Control: 19
2.4.3. Lateral Control: 19
3. System requirements: 22
4. System Architecture: 22
4.1. Overview of Architecture 23
5. Implementation Details: 25
5.1. Simulator Overview: 25
5.1.1. Simulator implementation: 26
5.2. Path Planner implementation: 29
5.3. Controller implementation: 30
5.3.1. Longitudinal – Curvature-Based PID Speed Control 31
5.3.2. Lateral – Either PP or Stanley controllers. 31
6. Analysis Validation and Results: 32
6.1. Planned tuning and Optimization: 32
6.1.1. PID Speed Controller Tuning: 32
6.1.2. Steering Controllers (Pure Pursuit and Stanley): 33
6.2. Validation and performance Metrics: 33
6.3. Results: 34
6.3.1. Steering Controllers comparison: 34
..... 34
..... 34
6.3.2. Global Reference fame Vs Ego-centric navigation: 35
6.3.3. PID Tunning: 35
6.3.4. Trackdrive maps: 36
6.3.5. Skipad: 40
6.3.6. Acceleration: 41
6.3.7. Lap times and Node times: 42
7. Summary 43
7.1. Overview 43
7.2. Key Technical Achievements 44
7.3. Reasoning behind choices 44
7.4. Lessons learned 44
7.5. Future Improvements 45
8. Plans and Risk Analysis: 46
8.1. Future Development Plan 46
8.2. Risk analysis table 47
8.3. Concluding Remarks 48
9. Bibliography 49
10. Appendix 51

10.1. Appendix A -Path Planner full pipeline 51
10.2. Appendix B - Pure Pursuit pseudo code 54
10.3. Appendix C – Stanley controller pseudo code 54
10.4. Apendix D – Main.py 55
10.5. Apendix E - Manager.py 57
10.6. Apendix F – controller.py 62
10.7. Apendix G – planner.py 66
10.8. Apendix H – car_state.py 68
10.9. Apendix I – cones.py 69
10.10. Apendix J – map_data.py 71
10.11. Apendix K – referee_state.py 72
10.12. Apendix L – sim_util 73
10.13. Apendix M – controllers.py 76
10.14. Apendix N – Visualization.py 95
10.15. Apendix O – Midterm report feedback 108

1. Introduction

Formula Student is a prestigious global engineering competition organized by the Society of Automotive Engineers (SAE), challenging university students to design, construct, and race Formula-style race cars. This competition provides a platform for students to apply their theoretical knowledge to real-world engineering challenges
while promoting innovation, teamwork, and practical skills.



Figure 1 : Formula Student competitions around the world

Participants are tasked with designing every aspect of the race car, including the chassis, suspension, powertrain, aerodynamics, and software. These designs must adhere to strict safety regulations and budget constraints, balancing innovation with feasibility. The competition uniquely combines static events, such as design presentations and cost analysis, with dynamic events testing vehicle speed, handling, and endurance.

Among the categories are internal combustion, electric, and driverless vehicles, each representing a step forward in automotive technology. The driverless category poses a particularly advanced challenge: teams must develop vehicles capable of autonomously navigating complex cone-defined tracks. This requires integrating advanced perception systems, path planning, and decision-making algorithms to ensure safety and precision. Emphasizing multidisciplinary engineering, the driverless category showcases the potential of autonomous technologies while highlighting the importance of seamless hardware-software integration[1].

The overall Scoring system for EV (Electric Vehicle), CV (Combustion Vehicle) And Driverless cars are:

Table 1 : Points Awarded for Each segment of the competition.

	Discipline	EV&CV (Pts)	Driverless (Pts)
Statics	• Business Plan	• 75	• -
	• Cost and Manufacturing	• 100	• -
	• Engineering design	• 150	• 150
Dynamics	• Acceleration	• 50	• 75
	• Skipad	• 50	• 75
	• Autocross	• 100	• 100
	• Endurance	• 250	• -
	• Efficiency	• 75	• -
	• Trackdrive	• -	• 200
Overall		1000 Pts	600 Pts

1.1. BGRacing Team

BGRacing is the Formula Student team representing Ben Gurion University. Founded in 2006, the team has participated in various competitions, with a notable achievement of being the first Israeli team to complete the endurance track using a fully electric racing car. BGRacing operates through several departments, including mechanical, electrical, systems, logistics, and autonomy, each contributing to the team's overall performance.

In response to the Formula Student Driverless competition, BGRacing established an



Figure 2: 2022 Croatia FSAE Competition

autonomous Department to develop the team's capabilities in this field. This department is divided into smaller teams, each focusing on a specific aspect of the autonomous driving system. Our team's responsibility within this structure is to develop cone detection algorithms using LiDAR point cloud data.

1.1.1. BGR Autonomous Team

As Mentioned Before, The BGR Autonomous team consists of 3 Major Teams.

The teams are:

- **PM** – Perception and mapping, this team is responsible to work with all the sensors Including Lidar, Camera, Imu and GNSS Sensors to fuse all the information to make the car sense and perceive their surrounding and state.
- **Simulator** – This team is in charge to work on the technical aspects of the simulator like importing the car dynamics into the simulator mainframe and create a debugging environment to optimize the performance as much as possible without driving the car and risking crashing it.
- **Planning and control** – The Path planning and control team oversees implementing a path planner and creating a high-level control scheme that would later be transferred into the Communication team, dynamics team and the electrical High Voltage team to create an almost optimal instruction on how to make the car drive and behave.

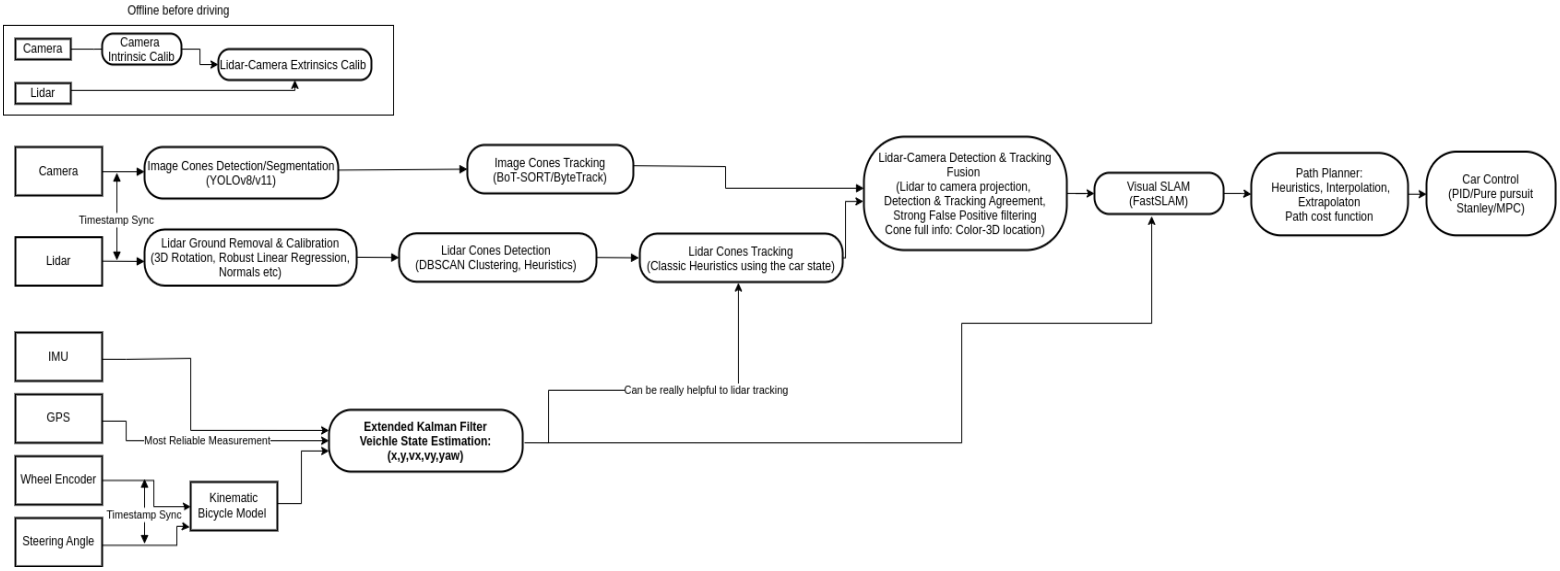


Figure 3: The architecture of the autonomous system

In This project we'll focus on the Path planning and control aspect of the car which are shown in the outer right section of the last figure.

1.2. Driverless dynamic events

The dynamic events in Formula Student are the on-track events where the performance of the teams' vehicles is tested. Every dynamic event consists of a race against the clock as wheel-to-wheel races are absolutely forbidden[2]. Here are the details of each dynamic event involving driverless vehicles:

- **Autocross:**

This event tests the car's dynamic ability in a one-lap sprint on an unknown track. The car must finish the event as fast as possible while avoiding any collisions with the cones placed on the track. This happens to be the most relevant challenge for driverless cars since it requires reliable cone detection, a fast path planning algorithm, an accurate enough SLAM module and a robust control pipeline

- **Trackdrive:**

  This event evaluates the reliability and endurance of autonomous vehicles. The car must navigate a set course on the Autocross track until ten laps are finished. Here the precision of the SLAM module and the reliability of the control pipeline are extensively tested.

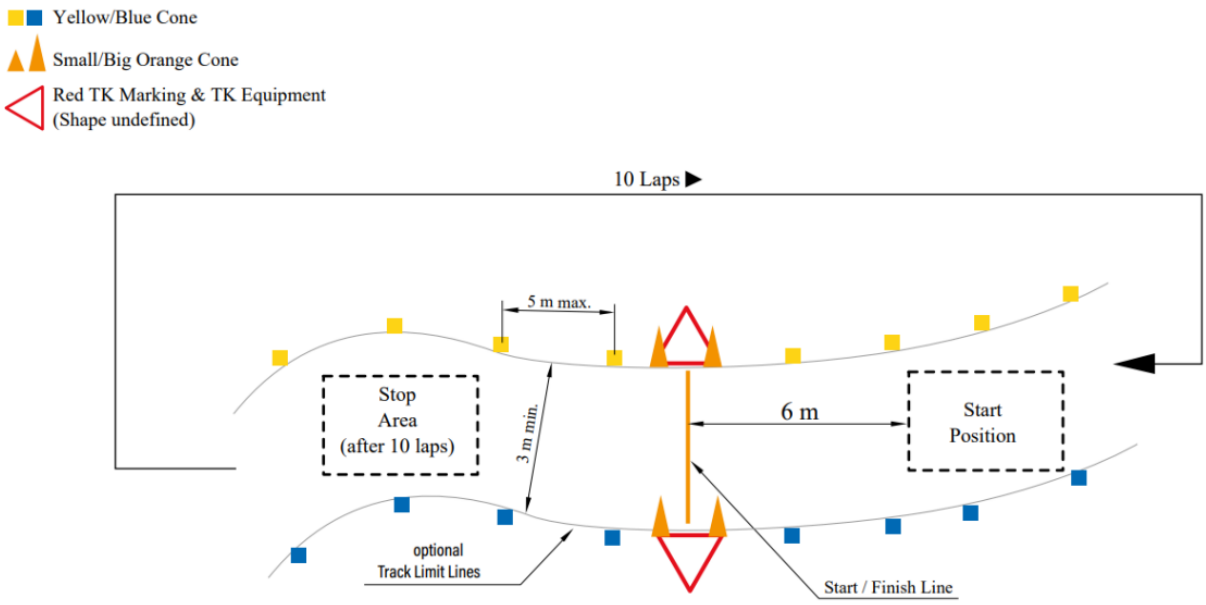


Figure 4 : Trackdrive Track layout

- **Acceleration:**

  The Acceleration event evaluates the car's acceleration in a straight line, from a standing start, over 75 meters. The performance of the car in this event is evaluated based on the time it takes to reach the finish line.

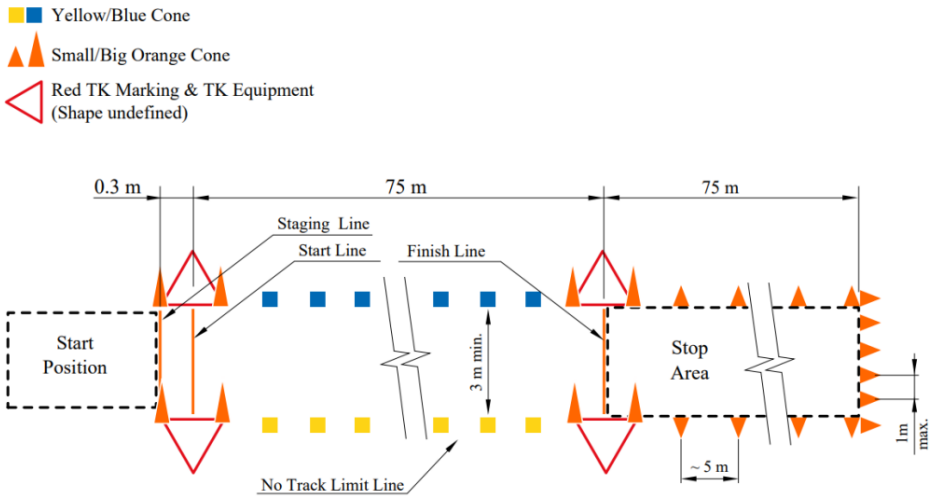


Figure 5 : Acceleration Track layout

- **Skipad:**

  The Skidpad event measures the car's lateral grip on a flat surface while making a constant radius turn. The skidpad track consists of two pairs of concentric circles in a figure eight pattern with fixed and previously known dimensions. The performance of the car in this event is evaluated based on the lap time mean of both second turns on each circle.

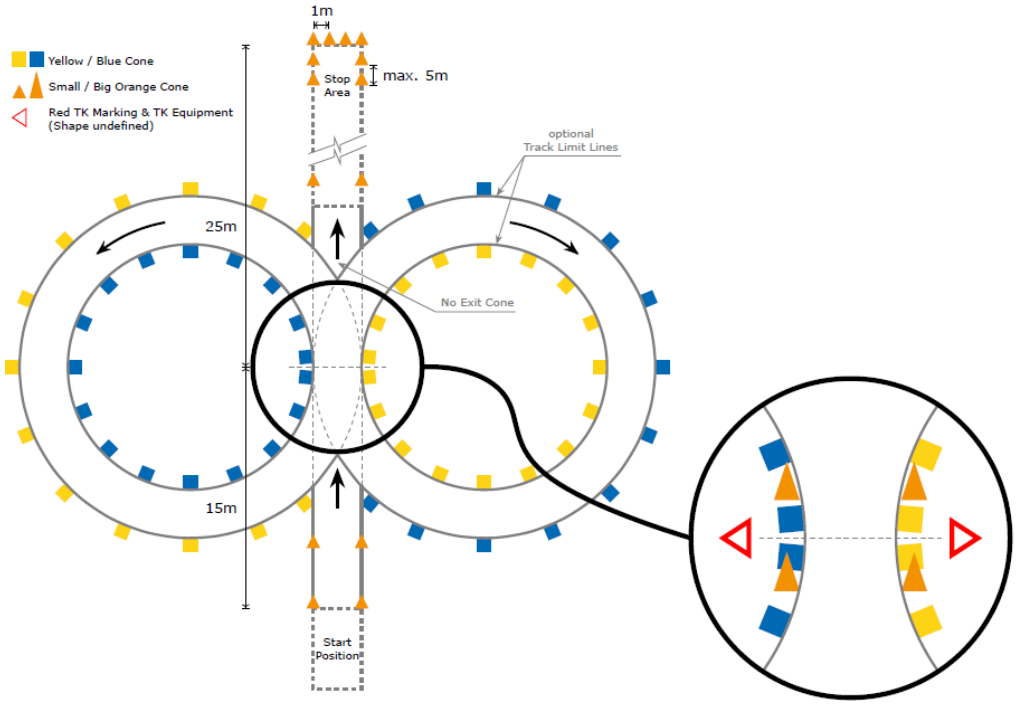


Figure 6 : Skipad Track layout

1.3. Scope and Objectives

Scope

This project is primarily focused on developing a High-Level Path Planning and Control system for an autonomous electric race car, with the main implementation occurring in a simulated environment. While the goal is to prepare the system for real-world application, the project will focus on simulation tools to ensure functionality and reliability before attempting hardware integration. The simulation-based approach allows the team to test and refine the system without risking vehicle damage or delays due to hardware limitations.

Additionally, the project scope includes:

- Creating a reliable simulation environment with realistic vehicle dynamics.
- Ensuring modularity in the architecture to facilitate future integration into the physical vehicle.
- Addressing key challenges such as cone detection, path planning, and high-level control.

Objectives

The project aims to achieve the following objectives:

1. **Develop a reliable and efficient Path Planning and control system**:
    - Implement algorithms for steering and speed control to achieve smooth and accurate navigation.
2. **Validate the system in a simulated environment**:
    - Use tools like AirSim to simulate competition scenarios and assess the performance of the Path Planning and Control pipeline under diverse conditions.
3. **Lay the groundwork for real-world implementation**:
    - Ensure the software architecture is compatible with the hardware setup to enable future testing on a physical vehicle.
4. **Compete successfully in the Formula Student Driverless category**: (Optional)
    - Focus on delivering a functional system that can complete all competition events within the defined timeline.

2. Literature Review and Theoretical Background

2.1. Perception Mapping (PM)

Perception Mapping form the cornerstone of autonomous vehicle operations, offering essential environmental and vehicle-state data for accurate localization, path planning, and control implementation. Within the Formula Student Driverless (FSD) competition context, these systems integrate multiple advanced sensors to achieve a comprehensive and reliable situational awareness:

2.1.1. LiDAR (Light Detection and Ranging):

LiDAR sensors utilize laser pulses to calculate distances with exceptional precision, enabling the creation of dense 3D point clouds representing the vehicle's surroundings. This is particularly beneficial for detecting track cones and static obstacles, even under challenging lighting conditions. We use The Innoviz one lidar that a solid state lidar that has a Detection range of 0.5-250 meters, an Angular resolution: 0.1°x0.1°, the Maximum field of view is 115°x25°And it runs either in 10 or 15 FPS.

This lidar forms the primary basis for mapping and localization.

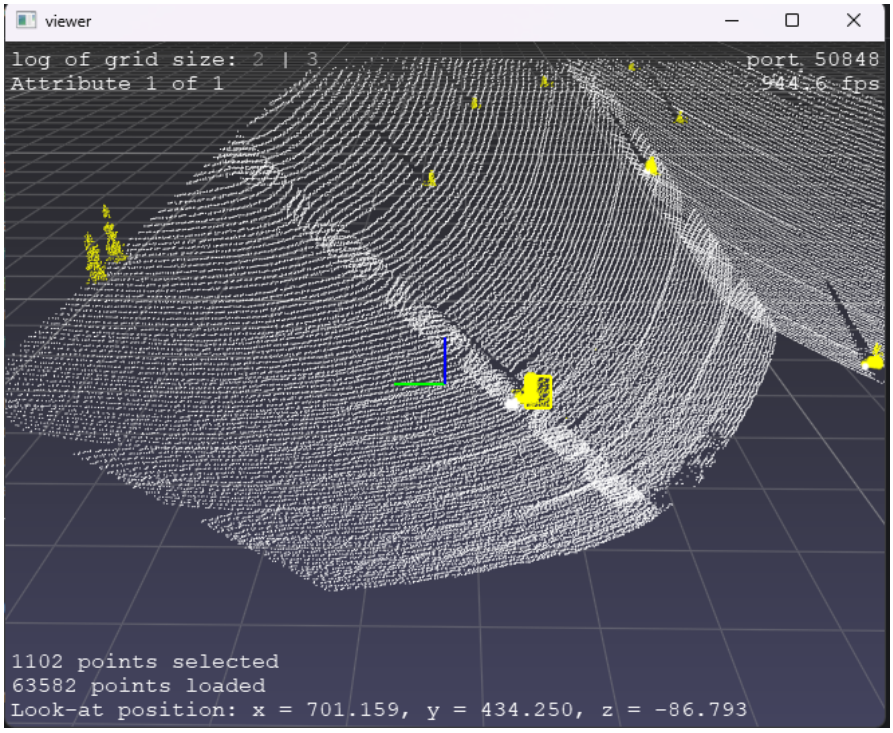


Figure 7 : Example of cone detection from the Innoviz one lidar From a BGR test run

2.1.2. Cameras:

Cameras capture rich visual data, enabling object detection and classification through state-of-the-art machine learning algorithms such as YOLO ("You Only Look Once"). These systems identify cones and track boundaries, providing real-time feedback for path planning algorithms.

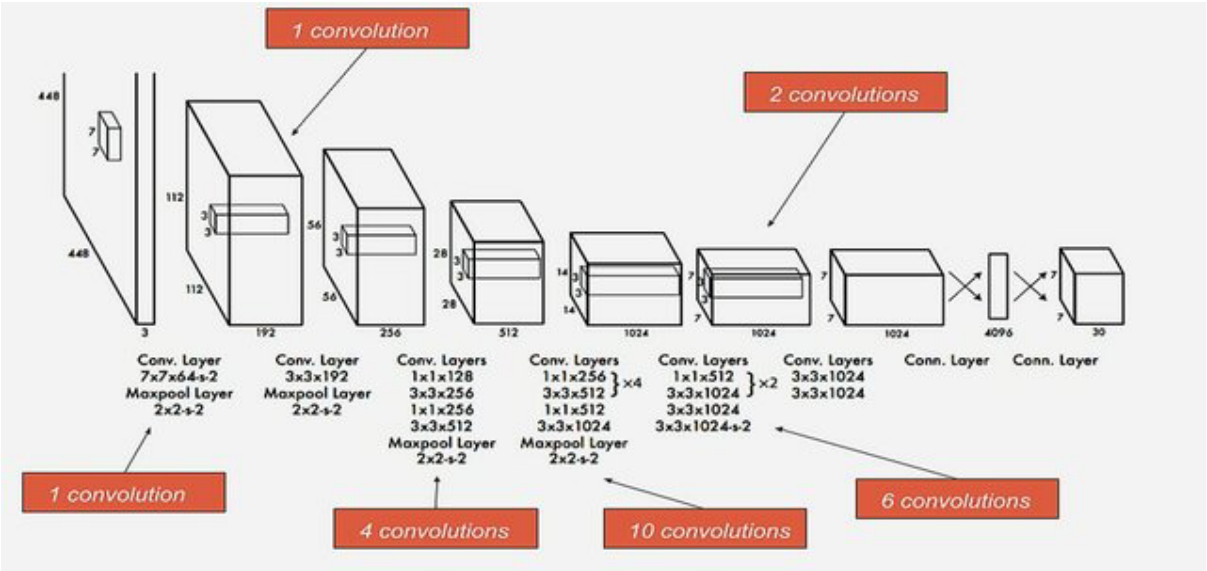


Figure 8 : Yolo V1 Representation

from: [3]," Real-time Object Detection: YOLOv1 Re-Implementation in PyTorch"

Here is how it looks.



Figure 9 : Exemplary cone detection

from:[4] , "Autonomous Vehicles on the Edge: A Survey on Autonomous Vehicle Racing"

2.1.3. IMU (Inertial Measurement Unit):

The IMU supplies high-frequency measurements of linear acceleration and angular velocity, critical for maintaining accurate vehicle state estimates during rapid maneuvers or when GNSS signals are unavailable.

2.1.4. GNSS (Global Navigation Satellite System):

GNSS delivers precise global positioning data, essential for outdoor navigation. Techniques like Real-Time Kinematic (RTK) positioning enhance this system's accuracy within a few centimeters, making it indispensable for cone-based navigation in competitive environments.

2.1.5. Sensor Fusion and State Estimation

By combining these sensors through methods like the Extended Kalman Filter (EKF), a robust and cohesive understanding of both the vehicle state and its environment is achieved. This enables downstream modules, such as path planning and control, to operate effectively in the challenging and dynamic settings typical of FSD events. The info we recive is already estimated using a kalman filter and state estimation of the various sensors. The sim is using a ground trouth referee state where the kalman filter isnt needed at first.

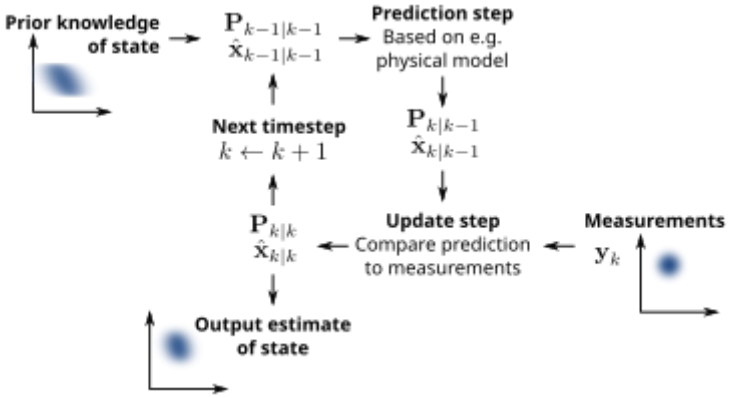


Figure 10 : Illustration of Kalman filter. [5]

2.2. Classical Path Planning Algorithms

Path planning involves determining an optimal trajectory for an autonomous vehicle to navigate from its current position to a designated target while avoiding obstacles. Classical algorithms, including Rapidly Exploring Random Trees (RRT) and triangulation with Voronoi diagrams, form the foundational strategies in this domain:

2.2.1. RRT (Rapidly Exploring Random Tree):

- RRT is a probabilistic, sampling-based algorithm that incrementally constructs a search tree, exploring the configuration space by prioritizing difficult-to-access regions. This allows for efficient pathfinding in high-dimensional or constrained environments.

2.2.2. Triangulation and Voronoi Diagrams:

- **Triangulation** divides the workspace into interconnected triangles, ensuring path feasibility and connectivity. Algorithms like Delaunay triangulation maximize the minimum angles of the triangles, enhancing numerical stability.
- **Voronoi Diagrams** partition the space into regions equidistant from obstacles, producing paths that maximize safety by maintaining clearance.

Classical path planning algorithms provide an essential starting point for trajectory generation, often forming the first stage of a hierarchical planning framework. In FSD applications, these methods supply initial coarse paths that are subsequently refined through optimization or predictive control frameworks to meet real-time constraints and ensure competitive performance.

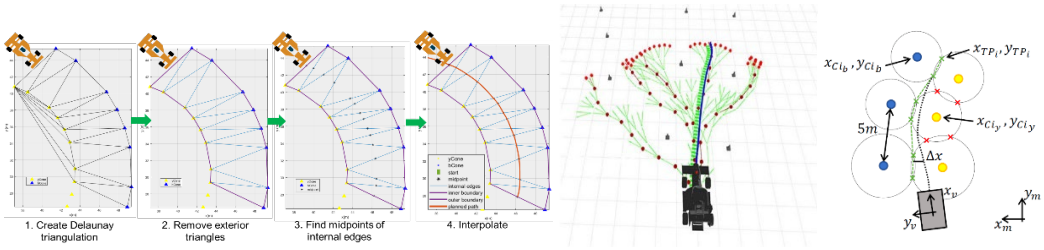


Figure 11 : examples of path planning algorithms from left to right - Triangulation, RRT, Varonie. [6]

2.3. Vehicle Dynamics

The precision of the mathematical model used to describe the system plays a critical role in control system behavior. Choosing an appropriate model is usually a relevant and critical task which will depend on the current scenario and the control strategy. The system aimed to control in this work is a Formula Student racing vehicle, therefore a vehicle model that describes the behavior of the car on track is mandatory to implement a robust control system.

Vehicle modeling is a major aspect that is widely studied in multiple engineering disciplines. In control theory, not only the physics principles of the model are important but also its discretization and integration methods as well as the linearization strategy (if applied).

In this Project we would mainly focus on nonlinear model approaches, specifically the kinematic and dynamic bicycle model that transforms the four wheeled frame of the car into a simplified two wheeled frame that looks like a bicycle.

in addition, the presented models will always assume that the system evolves in the two-dimensional space so 3 DOF will be considered (2 translation axis X, Y and 1 rotation axis Z). This assumption is consistent within Formula Student environments, where the track is a flat surface without elevation.

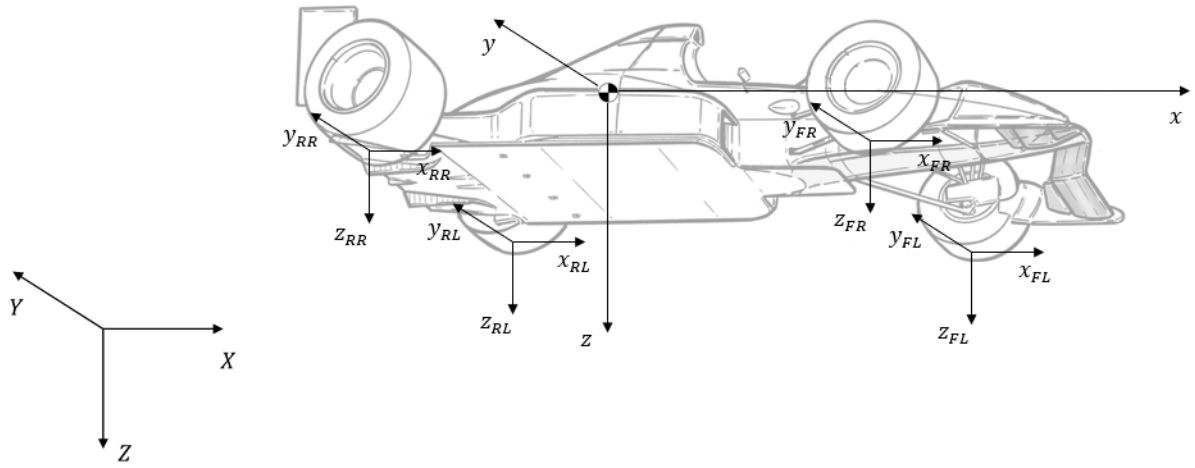


Figure 12 : reference frame of the car

from [7], "Path Planning and Guidance Laws of a Formula Student Driverless Car"

2.3.1. Kinematic Bicycle Model

The kinematic model provides a simple mathematical description of vehicle movement with a set of trigonometric relationships. This steady-state approach neglects any lateral tire forces and works under the assumption of null skidding. This model is well known in the literature ([8] to perform well at low-speed profiles (less than 5m/s) and yields a very simple formulation that can be suitable for low-speed applications.

The Matrix of Equations Looks like this.

(1)

$$\frac{d}{dt}\begin{bmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\end{bmatrix} = \begin{bmatrix}v\cdot\cos(\beta+\theta)\\ v\cdot\sin(\beta+\theta)\\ \omega\end{bmatrix}$$

(2)

$$\beta = \arctan\left(\frac{L_r}{l_r+l_f}\cdot tan(\delta)\right)$$

$Where\ \dot{x}, \dot{y}, and\ \dot{\theta}$ represent the longitudinal, lateral and angle velocities in global frame

The COG (center of gravity) slip angle is shown here by β and it's the angle between the velocity vector and the longitudinal axis from the vehicle frame.

2.3.2. Dynamic Bicycle Model

The dynamic representation is characterized by considering vehicle acceleration, so forces are used for the vehicle movement calculation. Thus, this model considers the front and rear lateral tire forces as well as the yaw moment caused by them. The dynamic model is more appropriate for working at higher speeds because it includes the tire model which is of special interest in the study of vehicle dynamics.

The dynamic bicycle model is expressed as follows:

(3)
$$\frac{d}{dt}\begin{bmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\\ \dot{v_x}\\ \dot{v_y}\\ \dot{\omega}\end{bmatrix} = \begin{bmatrix} v_x \cdot \cos(\theta) - v_y \cdot \sin(\theta) \\ v_x \cdot \sin(\theta) - v_y \cdot \cos(\theta) \\ \omega \\ \frac{1}{m} \cdot \left(F_x - F_{y_f} \cdot \sin(\delta) + m \cdot v_y \cdot \omega\right) \\ \frac{1}{m} \cdot \left(F_{y_r} - F_{y_f} \cdot \sin(\delta) - m \cdot v_x \cdot \omega\right) \\ \frac{1}{I_z} \cdot \left(F_{y_f} \cdot \cos(\delta) \cdot l_F - F_{y_r} \cdot l_r + M_z\right) \end{bmatrix}$$

Here $\dot{x}, \dot{y}\ and\ \omega$ represent the longitudinal, lateral and yaw acceleration. In this model, the longitudinal or motor force F_m is assumed to be applied on the CoG, so the longitudinal force expression is:

(4)
$$F_x = F_m - C_r - S \cdot C_d \cdot V_x^2$$

Cr stands for rolling resistance coefficient and $S \cdot C_d = \rho_{air} \cdot A \cdot C_d \cdot 0.5$ is the aerodynamic drag coefficient.

If we assume small slip angles, we will get the following equation

(5)
$$F_{y_{f\backslash r}} = C_{f\backslash r} \cdot \alpha_{f\backslash r}$$

However, racing this assumption is far away from reality, that's why we would use this term:

(6) $$F_{y_{f\backslash r}} = F_{Z_{f\backslash r}} \cdot D_{f\backslash r} \cdot \sin\left(C_{f\backslash r} \cdot arctan\left(B_{f\backslash r} \cdot \alpha_{f\backslash r}\right)\right)$$

Where $F_{Z_{f\backslash r}}$represents the vertical load at each tire and the Pacejka constants $D_{f\backslash r}$, $C_{f\backslash r}$and $B_{f\backslash r}$represent the peak value of the curve, the shape factor and the stiffness factor, respectively

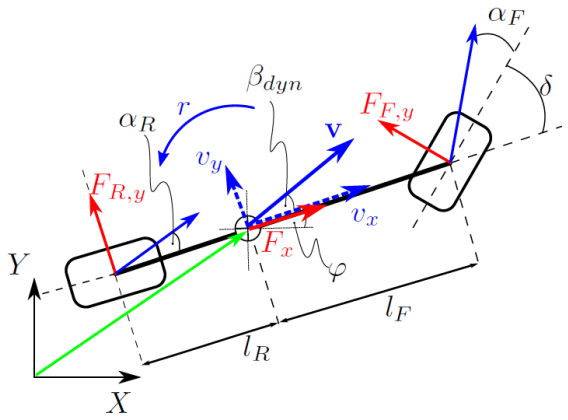


Figure 13 : Illustration of the Dynamic Bicycle model

The position vector is green, velocities in blue and forces in red. From[9], "Learning-based Model Predictive Control for Autonomous Racing,"

2.4. Control

we can control the car's performance and state in various ways, the

2.4.1. Coupled vs. Decoupled Control:

Coupled control systems handle lateral and longitudinal control simultaneously, improving performance at high speeds but increasing complexity [10]. Decoupled systems, on the other hand, simplify implementation by handling each axis independently. A Great example of a coupled controller is an Mpc controller that would handle the acceleration and steering angle of the car

2.4.2. Longitudinal Control:

Uses Proportional-Integral-Derivative (PID) controllers to regulate speed based on the desired trajectory.[11]

2.4.3. Lateral Control:

Usage of all sorts of controllers, either its PID or other variations of geometric controller like Pure pursuit and Stanley controllers.[12]

Table 2: Comparison of control methods

Controller	Description	Advantages	Limitations
PID Controller	Minimizes lateral and heading error using proportional, integral, and derivative components.	Simple, and effective for low-to-moderate curvature scenarios.	Performance degrades at high speeds or dynamic conditions.
Pure Pursuit	Calculates steering angle based on a target point along the desired path, ensuring alignment.	Intuitive, computationally efficient, ideal for real-time applications.	not effective at high speeds or sharp turns due to lateral forces.
Stanley Controller	Minimize cross-track and heading error by incorporating velocity and error metrics into control laws.	Reduces oscillations, improves adherence to the path.	Slower response times in dynamic or rapidly changing conditions.
Model Predictive Controller (MPC)	Optimizes control inputs over prediction horizon to follow a path while respecting system constraints	Handles complex dynamics and constraints, adaptable to different scenarios	Computationally expansive, complex, requires high computing power for real time use.

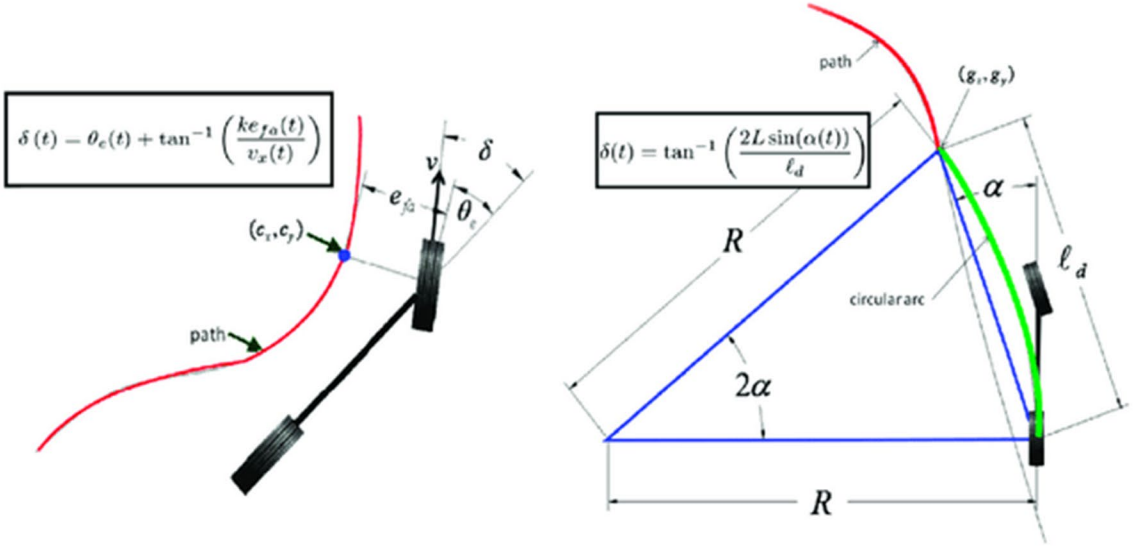


Figure 14 : Stanley and Pure Pursuit controller schematic diagrams.

The Stanley controller is shown on the left and the Pure Pursuit shown on the right.

from[13], "Delivery of Healthcare Resources Using Autonomous Ground Vehicle Convoy Systems"

pseudo code and the development of the geometric relationship in the appendix section.

Full geometric relationship and formulation of the equations can be found in the Coursera Specialization " Self-Driving cars specialization" [14], [15]

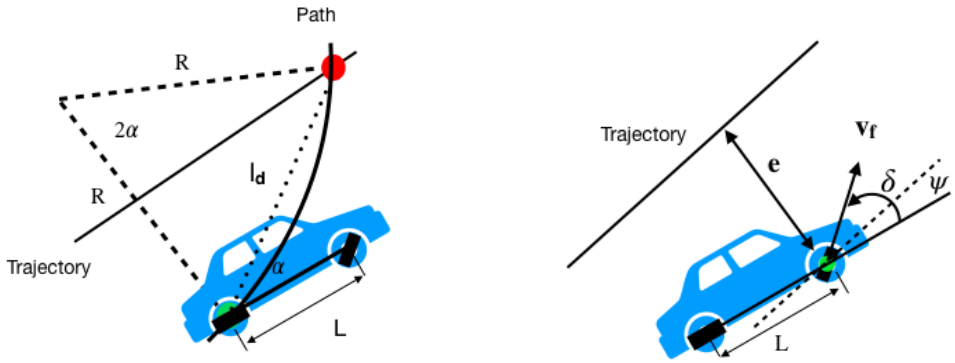


Figure 15 : Pure Pursuit (right) and Stanley (left) controllers geometric relationship

From [16] "Three Methods of Vehicle Lateral Control: Pure Pursuit, Stanley and MPC"

3. System requirements:

The system is designed to meet the following requirements:

1. **Speed:**
    - Target operating speed: 10 m/s.
2. **Scalability:**
    - Operates in simulation using referee state data from the simulator or using perception algorithms.
    - Scalable to real-world environments with perception team inputs.
3. **Inputs and Outputs:**
    - Inputs:
        - Vehicle state: position, velocity, heading.
        - Visual cues: cone detections or equivalent simulation data.
    - Outputs:
        - Control commands: throttle/brake and steering angle.
4. **Dynamic Speed Adjustment:**
    - Adjust speed dynamically based on track curvature and environmental factors.
5. **Tracking Accuracy:**
    - Maintains an average Mean Squared Error (MSE) of 1.5[m], ensuring the vehicle stays on track.
6. **Performance:**
    - Entire control and path planning pipeline must run at a minimum refresh rate of 10-15 [Hz] (66-100 [ms] per iteration) to ensure real-time usability and synchronization with LIDAR data.

4. System Architecture:

Shown here is the Block diagram of the Path planning and control system, were going to have a high-level diagram that would show the Full pipeline of our high level and low-level control

system, and a detailed diagram that would show in depth our path planning and control algorithm.

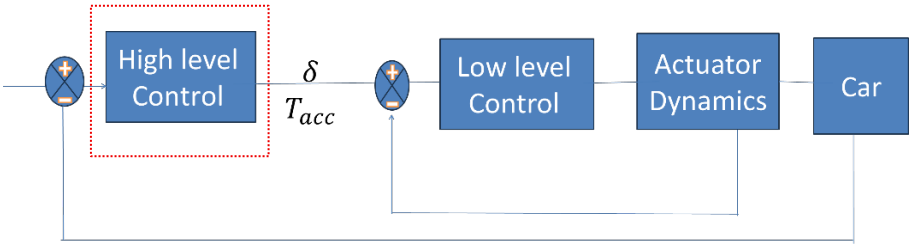


Figure 16 : High level Overview of the GNC pipeline.

As we can see, after the High level control block, the commands are being sent to the low level control and actuator dynamics which consists the control laws for the engine, and the low level steering command that is based on Ackerman geometry.[17] while were testing the algorithm in our sim environment all of the mentioned parts are part of the "Black Box" which is the car dynamic model and the sim's physics engine.

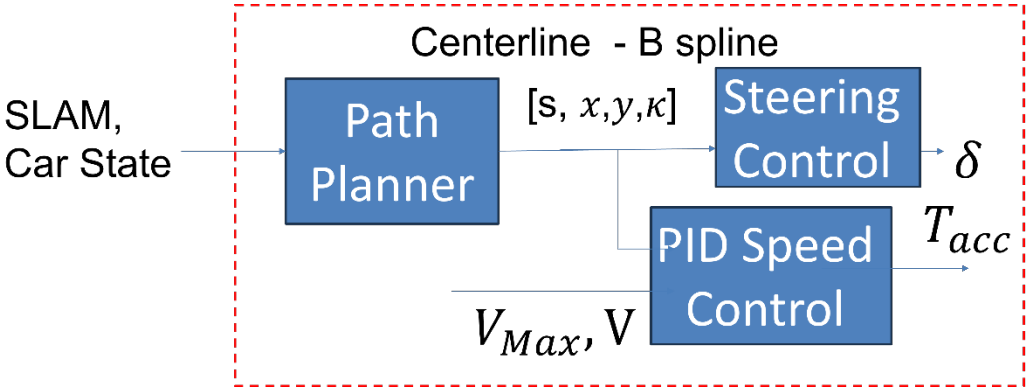


Figure 17 : Magnified and detailed presentation of High-Level Path planning and control system.

The inputs in both block diagrams are the Slam map (what the car sees and its location), and the car state. This information comes either from the sim environment in terms of measurement or call from the API (referee state) or in real life from the sensor readings which consists of imu, cameras and LIDAR.

4.1. Overview of Architecture

The system consists of the following components:

1. Car State:
   - This Blocks tracks the current state of the car, including position, velocity orientation and other relevant parameters that Comes either from the sensors inside the actual car or the information from the simulator API.
   - This information is later passed on to the path planner and it serves as a critical input also for the control block, ensuring real time decision making and great handling.

2. Perception – VSLAM:
    - This blocks process and sends input from sensors (cameras, LIDAR and IMU) to detect cones and estimate the cars surrounding.
    - It combines Data from multiple Sensors to create a map of the surroundings, enabling robust situational awareness and localization between the world coordinate axis to the car axis.
3. Path Planner:
    - This block includes the path planning calculation, it takes the car state (pos, orientation) and Visual preparation queues (Cones by type) and it generates a path that is used as a reference path for the path tracking control algorithms.
4. Control:
    - The control module translated the path created by the path planner and the current car state to find the control commands that would instruct the vehicle to drive in the given path with a given speed.
    - The control block can be either a coupled or decoupled controller, that outputs the commands.
5. Update State And perception:
    - This block Simulates the actual car and its response to the actions and changes after each control command
    - This block shows the feedback loop that integrates new senor data and feedback to the impact of the last commands in real time.
    - This block ensures that the system maintains situational awareness and adapts dynamically to changes in the environment.

This chapter provides a shallow high-level overview of the system architecture, in chapter 5 we will elaborate about the path planner, simulator environment and control scheme and the reasoning for it.

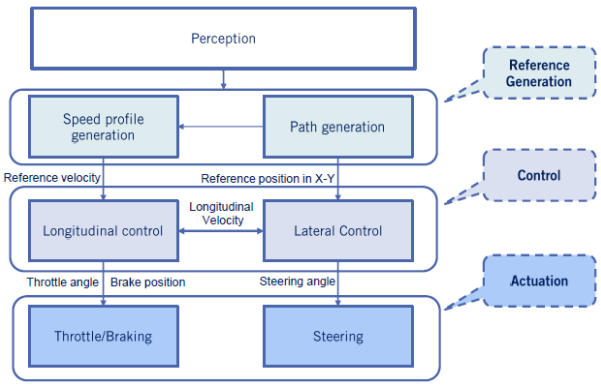


Figure 18 : Architecture of Vehicle Control strategy from the online course "Introduction to Self-Driving cars" [15]

5. Implementation Details:

This chapter outlines the technical implementation of the system, providing a detailed explanation of how the various components described in Chapter 4 were realized. The implementation phase focused on transforming theoretical designs into a functional, real-time path planning and control system capable of meeting the requirements of the Formula Student Driverless (FSD) competition.

Key objectives during the implementation include:

- **Modular Design**: Ensuring each system component (e.g., Path Planner, Control) is implemented as an independent module for scalability and maintainability.
- **Real-Time Performance**: Achieving the required refresh rate of 10-15 [Hz] through optimized algorithms and efficient coding practices.
- **Simulation Integration**: Developing the system to work seamlessly with the Simulator API, enabling iterative testing and validation in a virtual environment.
- **Real world Vehicle integration:** ensuring that the system can work and communicate with all the other team's modules including accessing the real-world sensor data, perception algorithms and low-level car control.

This section would follow an in-depth look into the specific algorithms, coding frameworks and sim environment that would help us achieve the project goals.

This chapter is going to have three chapters about each component of architecture, it would focus on the simulator, on the path planner and on the chosen control scheme.

5.1. Simulator Overview:

The chosen simulator platform that was chosen is the Formula Student Driverless (FSD) simulator. [18] This sim is a community project that has the goal to provide an end-to-end simulation for FSD Teams. It simulates all commonly used sensors and is ROS compatible.

The project is an Unreal Based version of the AirSim simulator by Microsoft.



Figure 19 : GUI from the simulator at action.

The Simulator Is hosted on a GitHub Repository, and its open source so it widely used in FSD teams to simulate, test and validate their designs. [18]

The simulator includes a Detailed examples of official tracks, custom tracks and different types of runs like Skipped and Acceleration. To simulate as close as possible the real world setting you can add artificial sensor noise to examine the robustness of the system.

The Sim runs the Nvidia Based PhysX - Vehicles Engine, this is why it isn't possible to work with model-based controllers, and it forces teams in the long run to use system identification methods to optimize performance.

To sum things up, the simulator has several advantages:

- Safe And cost-effective testing environment
- Realistic Vehicle dynamics for fine tuning and optimizing algorithms
- Allows for rapid development

The limitations of the simulator are mainly the fact that not all aspects of real-world scenarios can be tested in the simulator including slip angle, weather affect and realistic sensor noise, the way to address it is to acknowledge the fact that this whole project is a "POC" (proof of concept) and that why the speed and accuracy requirements are flexible.

5.1.1. Simulator implementation:

The simulator connects to the coding interface by either a setpoint API or either by a ROS based interface called "ROS BRIDGE" that uses the API client as a topic and a subscriber to access all the Data.

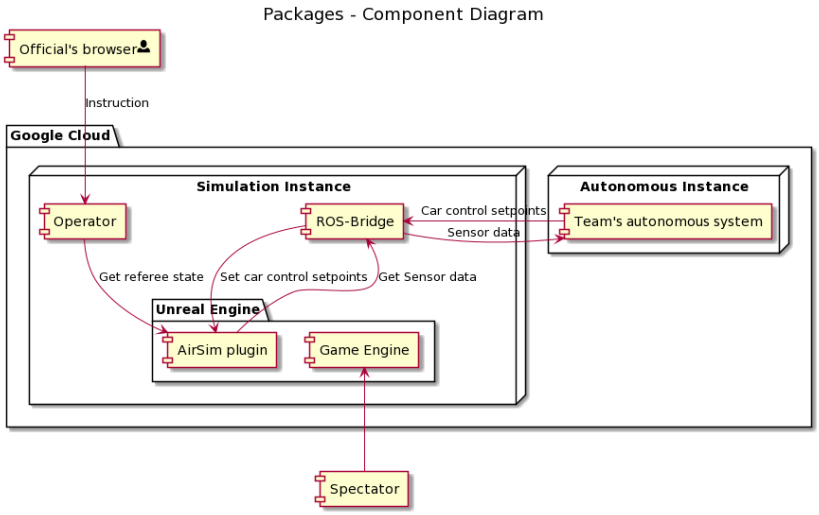


Figure 20 : System overview of the simulator from [18].

Seen here is the system overview of the simulator, you can see here the ROS-Bridge that talks with the Sims plugin and the game engine. The Operator and spectator nodes are irrelevant for this scope of the project and were used only during the online FSD competitions that were during Covid. As for writing this report, we're not using the API python client method that is shown here, also, were going to show how we can access the "Ground Truth" data via the referee state.

```python
# This code adds the fsds package to the pyhthon path.
# It assumes the fsds repo is cloned in the home directory.
# Replace fsds_lib_path with a path to wherever the python directory is located.
import sys, os
fsds_lib_path = os.path.join(os.path.expanduser("~"), "Formula-Student-Driverless-Simulator", "python")
sys.path.insert(0, fsds_lib_path)

import time
import fsds

# connect to the AirSim simulator
client = fsds.FSDSClient()

# Check network connection
client.confirmConnection()

# After enabling api controll only the api can controll the car.
# Direct keyboard and joystick into the simulator are disabled.
# If you want to still be able to drive with the keyboard while also
# controll the car using the api, call client.enableApiControl(False)
client.enableApiControl(True)

# Instruct the car to go full-speed forward
car_controls = fsds.CarControls()
car_controls.throttle = 1
client.setCarControls(car_controls)

time.sleep(5)

# Places the vehicle back at it's original position
client.reset()
```

Figure 21 : Client API connection example from [18].

```
state = client.getCarState()

# velocity in m/s in the car's reference frame
print(state.speed)

# nanosecond timestamp of the latest physics update
print(state.timestamp)

# position (meter) in global reference frame.
print(state.kinematics_estimated.position)

# orientation (Quaternionr) in global reference frame.
print(state.kinematics_estimated.orientation)

# m/s
print(state.kinematics_estimated.linear_velocity)

# rad/s
print(state.kinematics_estimated.angular_velocity)

# m/s^2
print(state.kinematics_estimated.linear_acceleration)

# rad/s^2
print(state.kinematics_estimated.angular_acceleration)
```

Figure 22 : Examples of Fetching Ground Truth Data from [18].

As mentioned before, the LIDAR on the Airsim is a mechanical lidar, the output it gives to the perception team isn't the same as the output in the real world. Because a mechanical lidar rotates it brings slices of point clusters and data which isn't suitable for the Perception and mapping algorithms, that's why we simulated the LIDAR using a depth camera. As for the writing of this project, we haven't used it fully to completely validate the perception algorithms on the simulator, but we have a fairly accurate simulation of the Innoviz Point cloud using depth cameras.

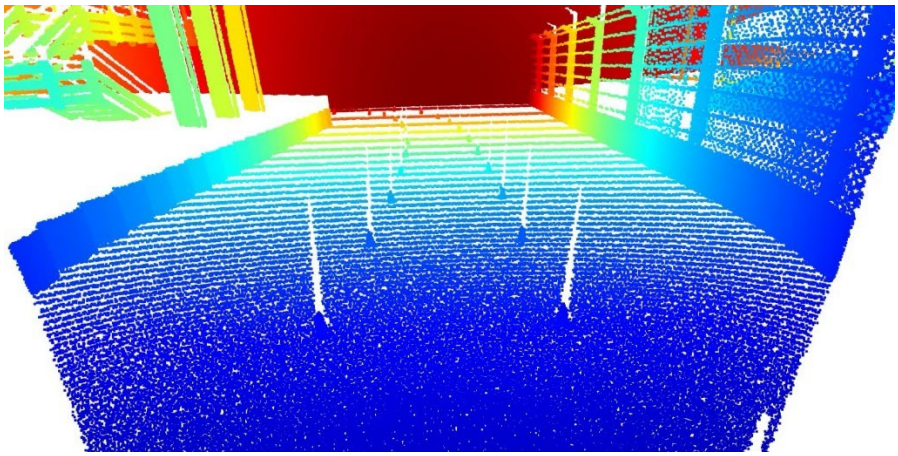


Figure 23 : Depth camera output to simulate the LIDAR

5.2. Path Planner implementation:

The Path planner that was chosen is the FT-FSD Path Planning [19] that was developed by "FaSTTUBe "(Berlin Based Team from "TU Berlin") for the 2022 Formula student season. This Path planner was made specifically for the formula student competitions, and it uses a novel geometric approach that offers robustness like finding a path even when some of the cones are missing.

The Algorithm works with the following steps:

1.Cone Sorting – process of raw cone data and sorting it into left and right using geometric criteria.

2.Cone Matching – Matching between left and right cones to define track boundaries, also it fills gaps where cones are missing.

3.Path calculation – Creates a smooth centerline using interpolation and curvature analysis. Takes the constraints of the run making a path that is feasible.

4.Global path optimization – optimizing the path to make it into an optimal global path that maximizes performance, safety and the rules of the run.

This Algorithm takes as an input the location of the car, the direction of the car and the global position of all the visual cones (with or without color labeling). It outputs a Parameterized B spline That is represented as a MX4 array where M is the number of points on the path (the length of the prediction horizon).

The columns represent the spline parameter (distance along the Path), X, Y coordinates and curvature of the path. $[s, x, y, \kappa]$

Like the simulator, this path planner can create a path for all types of runs in the competition like skipad and track drive runs. A Full pipeline is in the Appendix section.

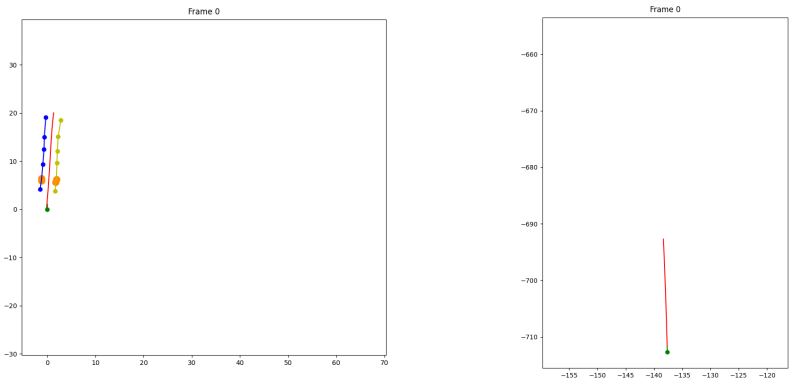


Figure 24 : Examples of the visualization of the path planner in action.

Skipad is on the right and track drive is on the left. From FSD-Path Planning GitHub [19].

The Path planner is completely stateless and is called by a very straightforward manner.

```python
from fsd_path_planning import PathPlanner, MissionTypes, ConeTypes

path_planner = PathPlanner(MissionTypes.trackdrive)
# you have to load/get the data, this is just an example
global_cones, car_position, car_direction = load_data()
# global_cones is a sequence that contains 5 numpy arrays with shape (N, 2),
# where N is the number of cones of that type

# ConeTypes is an enum that contains the following values:
# ConeTypes.UNKNOWN which maps to index 0
# ConeTypes.RIGHT/ConeTypes.YELLOW which maps to index 1
# ConeTypes.LEFT/ConeTypes.BLUE which maps to index 2
# ConeTypes.START_FINISH_AREA/ConeTypes.ORANGE_SMALL which maps to index 3
# ConeTypes.START_FINISH_LINE/ConeTypes.ORANGE_BIG which maps to index 4

# car_position is a 2D numpy array with shape (2,)
# car_direction is a 2D numpy array with shape (2,) representing the car's direction vector
# car_direction can also be a float representing the car's direction in radians

path = path_planner.calculate_path_in_global_frame(global_cones, car_position, car_direction)

# path is a Mx4 numpy array, where M is the number of points in the path
# the columns represent the spline parameter (distance along path), x, y and path curvature
```

Figure 25 : Basic usage of the Path Planner from [19].

5.3. Controller implementation:

The control Block (Speed or Steering) is responsible for converting the planned trajectory into actionable steering and acceleration commands. It ensures accurate path following while maintaining stability and responsiveness. The following control strategies are implemented, decoupled lateral and longitudinal control schemes that include a curvature based PID controller for speed control and two geometric controllers like Pure pursuit and Stanley controllers to maintain path tracking.

The reason for using decoupled control has to do with the fact that our main goal is to implement a simple and competently cheap solution to make it work before focusing on complex solutions. Also, it was shown that a dynamic pure pursuit controller that considers slip angle dynamics can perform very well in racing conditions as shown here [20].

The PID Speed controllers is a simple controller that regulates the speed using the current speed and target speed (which depends on the curvature) to create accelerations and break commands to minimize the error, which is the speed difference between the current speed of the car to the wanted speed. As of right now we're using a PID based on the Simple-PID python library.[21]

The controller's output looks like this where δ is the steering angles are in degrees and converted to radians. $T_{accelaration}$ is the throatle and break response that represent either full throttle or full breaks.

(7) $$\delta \in \pm rad[30°]$$

(8) $$T_{accelaration} \in \pm 1$$

5.3.1. Longitudinal – Curvature-Based PID Speed Control

This is how the curvature aware PID speed controller was designed

The controller tracks the speed error of $V_{ref}(\kappa)$ which is a function of the curvature.

(9) $$V_{ref} = V_{max} \cdot e^{-\alpha \cdot |\kappa|}$$

We can see here that the reference speed adjusts based on the curvature and a negative gain α that gives us a situation where the refrance speed is always equal or smaller to the maximum allowed speed. This ensures early breaking before corners and full speed on straights.

5.3.2. Lateral – Either PP or Stanley controllers.

Presented here is a small table of comparisons between the chosen lateral controllers with their equations for the bicycle model steering angle.

Table 3 :Comparison of the lateral controllers with their steering equations

Controller	Method	Control Equation
Pure Pursuit	Geometric lookahead	$\delta = \tan^{-1}\left(2 \cdot L \cdot \frac{\sin(\alpha)}{L_d}\right)$
Stanley	Feedback on lateral + heading error	$\delta = \psi_{error} + \tan^{-1}\left(k \cdot \frac{e_{lat}}{v}\right)$

6. Analysis Validation and Results:

This chapter outlines the planned methods for analyzing and validating the performance of the autonomous system. The focus is on tuning the PID speed controller and optimizing the Pure Pursuit and Stanley steering controllers to improve path tracking and speed control.

6.1. Planned tuning and Optimization:

6.1.1. PID Speed Controller Tuning:

The PID controller will be tuned to manage the vehicle's speed dynamically based on path curvature and desired trajectory.

The tuning process will follow the Ziegler–Nichol's method [22], with parameters $K_p, K_i and\ K_d$ adjusted iteratively to:

- Maintain a maximum speed of **10 m/s** on acceleration runs.
- Dynamically reduce speed in curves based on curvature, ensuring stable navigation and allowing quick response to changes.

The tunning method itself was done by hand without completely accurate measurements given the fact that we don't have full access to the vehicle dynamics and that because we don't use the ROS-Bridge we don't have a fully accurate time step, so we tuned it via a constant timestep logic which yielded somewhat of a noisy result. The tunning isn't optimal but it's suitable to make the speed performance and tracking acceptable.

6.1.2. Steering Controllers (Pure Pursuit and Stanley):

The steering controllers will be tuned to improve path-tracking performance. The following parameters will be adjusted:

Pure Pursuit:

The Pure Pursuit controller computes the steering angle by targeting a point on the path at a given lookahead distance.

The Lookahead Distance Determines how far ahead on the path the controller looks to calculate the steering angle. Larger values improve stability on straights but can reduce accuracy in tight curves.

- The tuning of The Lookahead Distance ensures smooth steering, avoids oscillations and sharp corrections. It also helps minimize Cross-track error in the process.

Stanley Controller:

The Stanley controller minimizes lateral deviation from the path while aligning the vehicle's heading with the path direction.

Tunning Parameters:

- **Proportional Gain for Cross-Track Error ($k_{stanley}$)**: Affects how aggressively the controller corrects lateral deviations. Higher values improve tracking in curves but may cause oscillations on straights.
- **Softening Constant (k_{soft}):** Avoids division by zero at low speeds and smoothens response

By tuning these parameters, we would minimize the Cross-Track Error while ensuring stable and smooth steering inputs and we would balance the trade-off between aggressive corrections and oscillations.

6.2. Validation and performance Metrics:

To validate the system's performance the following methods will be used:

1. **Performance Metrics and Plots:**
    - Mean Squared Error (MSE) and Root Mean Squared Error (RMSE):
        - Measure the overall deviation from path across a complete run. $MSE \leq 1.5[m^2]$ as a validation target and $RMSE \leq 1.22\ [m]$
    - Speed Profiles
        - Validate that the PID controller maintains desired speed and dynamically adjusts it given the curvature.

- Acceleration plots
  - Measure the acceleration plots in various ways over time, including GG Plot [23].
- Logger Data
  - Measure and log the incoming data from the test runs either its runtime of the nodes, errors, control command, lap times and more.
- Steering controller comparison
  - Show using Path tracking metrics and referee state which steering controller is better.
- Control inputs
  - Show both the acceleration and steering control inputs to see if the controllers are close to saturation.

.

6.3. Results:

In this segment the results would be shown, the simulator has 3 competitions maps and both skipad and acceleration maps. First, a comparison between the steering controllers in terms of path tracking would be shown, the comparison between global reference or location free Navigation, the effects of the PID tunning and after that we'll show the same set of plots for each distinct run including the lap times and number of cones that were hit.

6.3.1. Steering Controllers comparison:

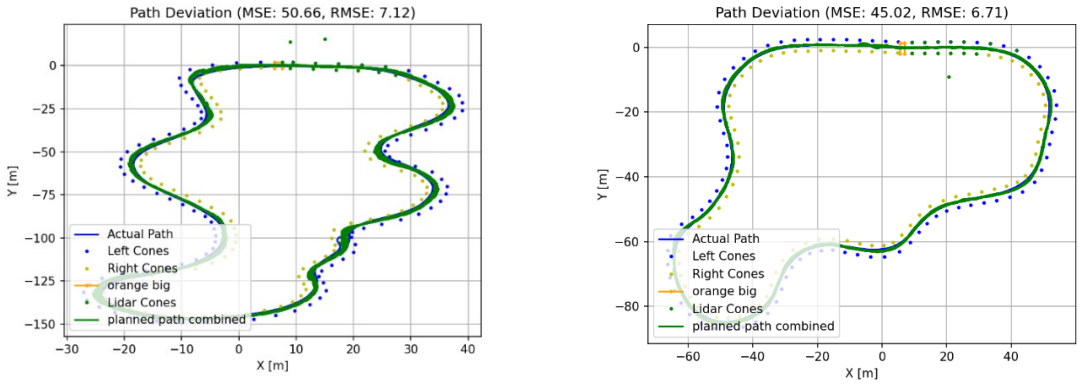


Figure 26 : Steering controllers Path tracking comparison, Stanley (Left) and PP (Right)

As we can see, the Stanley controller performed worse than the PP controller. There are a lot of reasons for it, but most of them revolve that the Stanley controller is unstable at low-speed situations and that it suffered from severe oscillations. It's possible that a better turning of the

Stanley gains would've shown a different outcome, but it was shown that pure pursuit was way simpler to implement and tune and to optimize.

The Pure pursuit Look Ahed Distance was 6 $[m]$ and the Stanley gains were $K_{Stanley} = 0.8\ and\ K_{soft} = 0.3$. The units of the RMSE and MSE are in $[cm]$ and $[cm^2]$.

6.3.2. Global Reference fame Vs Ego-centric navigation:

Our current Vehicle has some with the imu and the location data, so we had to derive a solution to this scenario, so we've also added an argument for running the car with an Ego-centric based coordinate system where the world comes toward the car and the car is stationary.

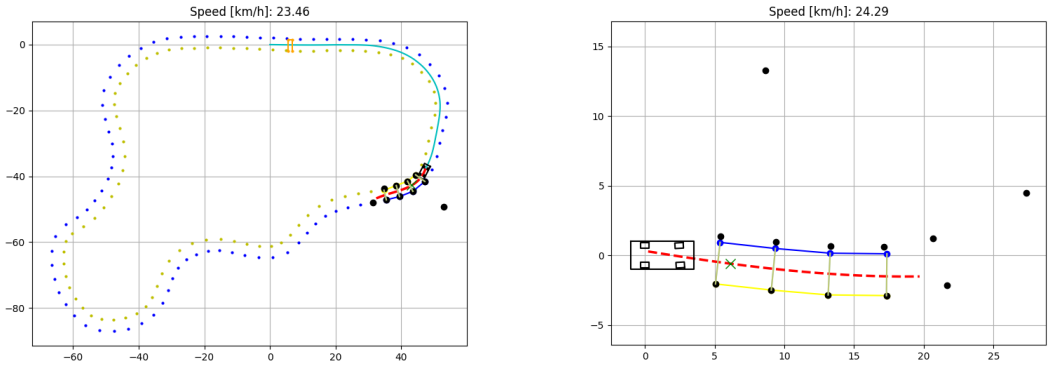


Figure 27 : Frames from an actual run with location info (Left) and without location info (Right)

As shown, the left plots also show the referee cones as well as the measured cones via the LIDAR (in black) while on the right you see that you don't have those.

6.3.3. PID Tunning:

As mentioned before, the PID was tuned by hand, the full tunning process wouldn't be shown here but the comparison of the P, PD, PI and PID controllers would be shown here. The values for each one of them are the values that were chosen after multiple test runs.

Table 4 : PID GAINS

Gain	Value
K_p	0.4
K_i	0.025
K_d	0.025

of the following plots were made on an acceleration test run with a target speed of 25 $[km/h]$ with the breaking feature of the PID set to off. $[\alpha = 0]$.

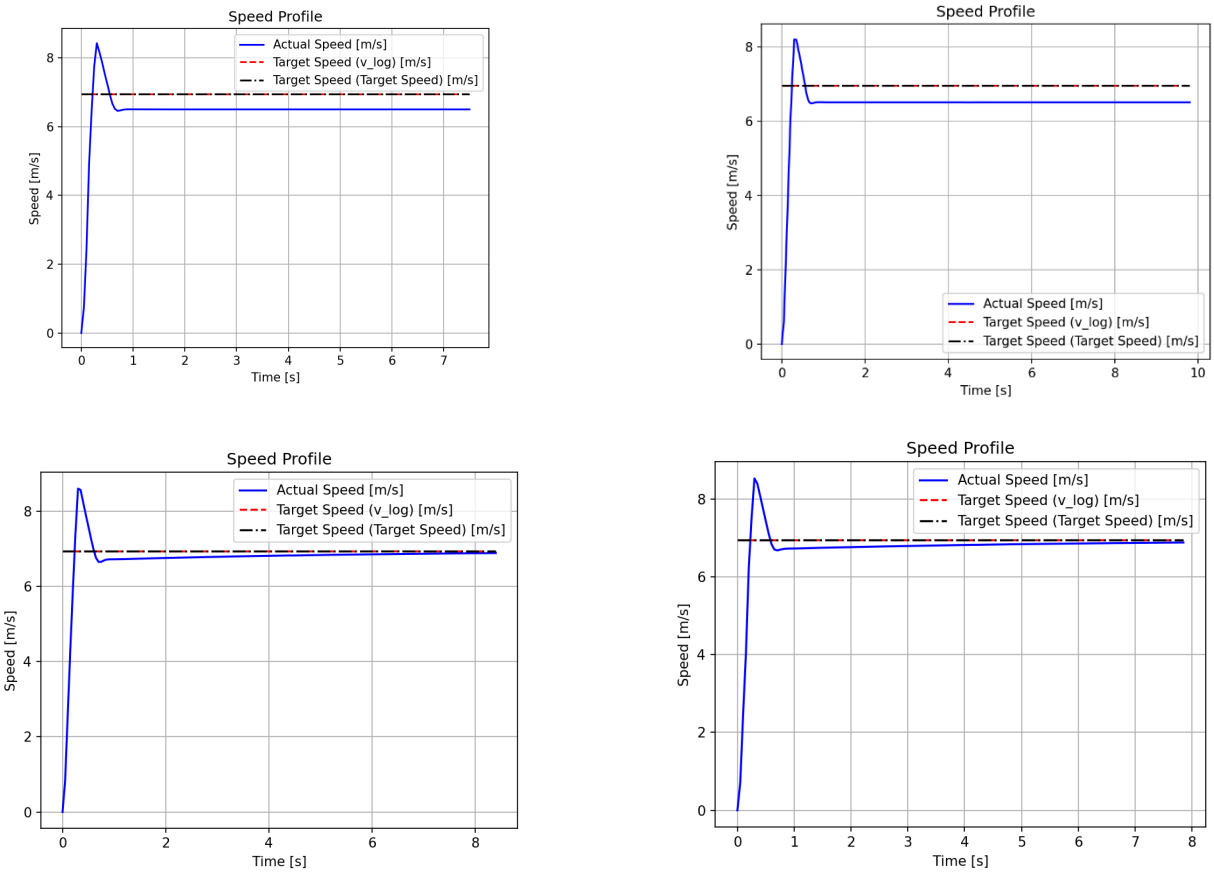


Figure 28 : PID Tunning - CW Are P, PD, PI and PID

As we can see, A Proportional controller would be more than enough to handle a straight-line acceleration, we can see how the PD controller has Damping effect that isn't noticeable here given the small K_d gain, however we can see how the integral term helps minize the Steady state error. Because we don't have a need in this scenario to track a changing speed, we don't see any difference between the PI and PID controllers, however, in later plots we would see how the speed tracking works and how the derivate term helps us with the transient state while the speed changes.

6.3.4. Trackdrive maps:

In this section we would show 3 sets of plots for each map, the difficulty of each track increases, the last track simulates a sunset so it on theory should challenge more the perception algorithms. In the end we would show the lap times, and the cones that were hit in each run. The target speed for all the runs is 25 $[km/h]$ which is about 7 $[\frac{m}{s}]$.

6.3.4.1. Map 1:

Here we would show the following plots for the first map, we would show a path tracking plot, a speed profile and various acceleration plots. In the end we would show the lap times, and the cones that were hit in each run.

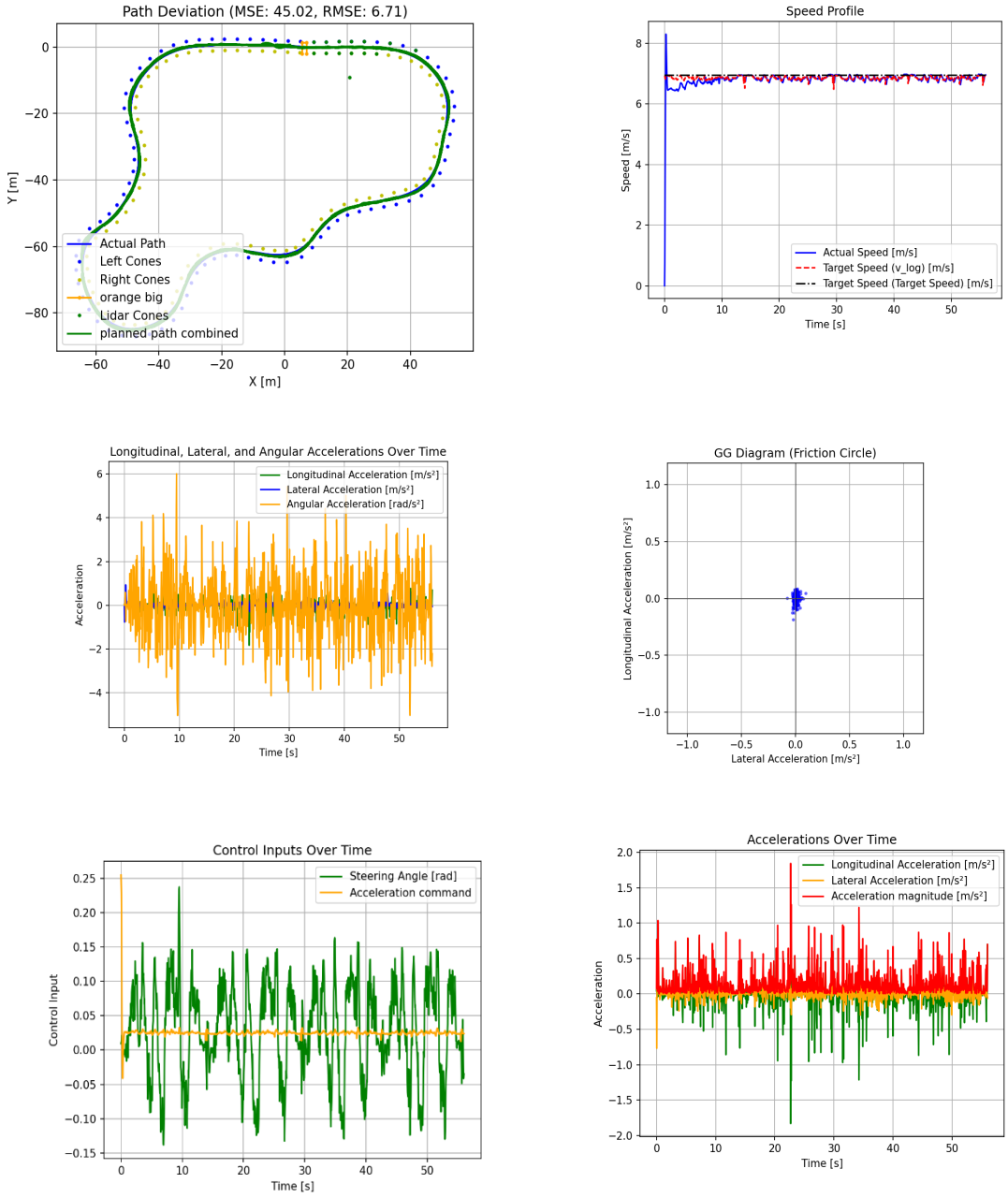


Figure 29 : Plots for Map 1. On left we have Path tracking, accelerations and control inputs over time. On the right we have Speed profile, GG Diagram and longitudinal lateral acceleration and acceleration magnitude over time.

As we can see, the speed and path tracking are working, and the controllers aren't close to being saturated.

6.3.4.2. Map 2:

Here are the same sets of plots for map number 2.

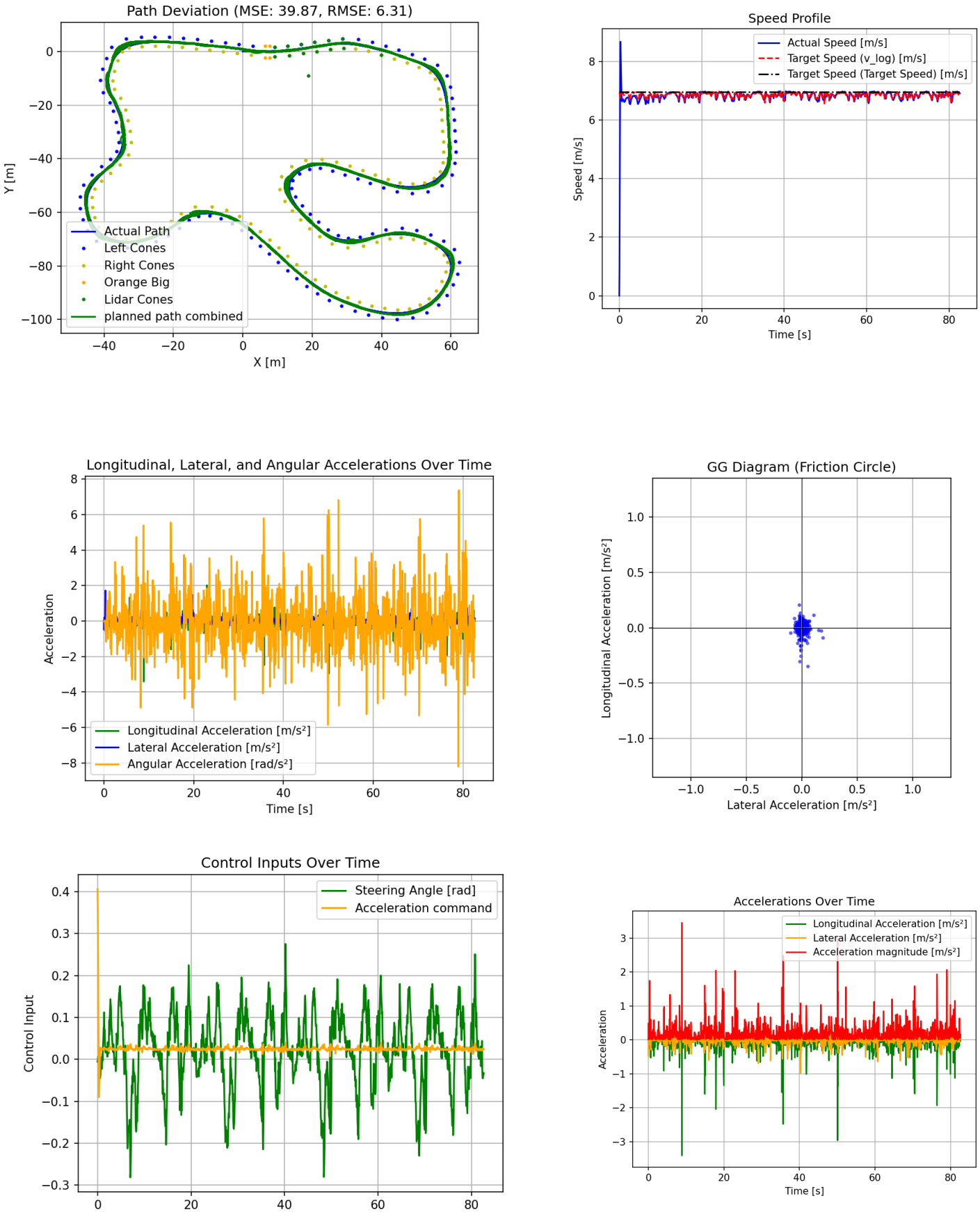


Figure 30 : Plots for Map 2. On left we have Path tracking, accelerations and control inputs over time. On the right we have Speed profile, GG Diagram and longitudinal lateral acceleration and acceleration magnitude over time.

Also, here we can see that overall, the controllers worked as intended.

6.3.4.3. Map 3

Here are the plots for map 3.

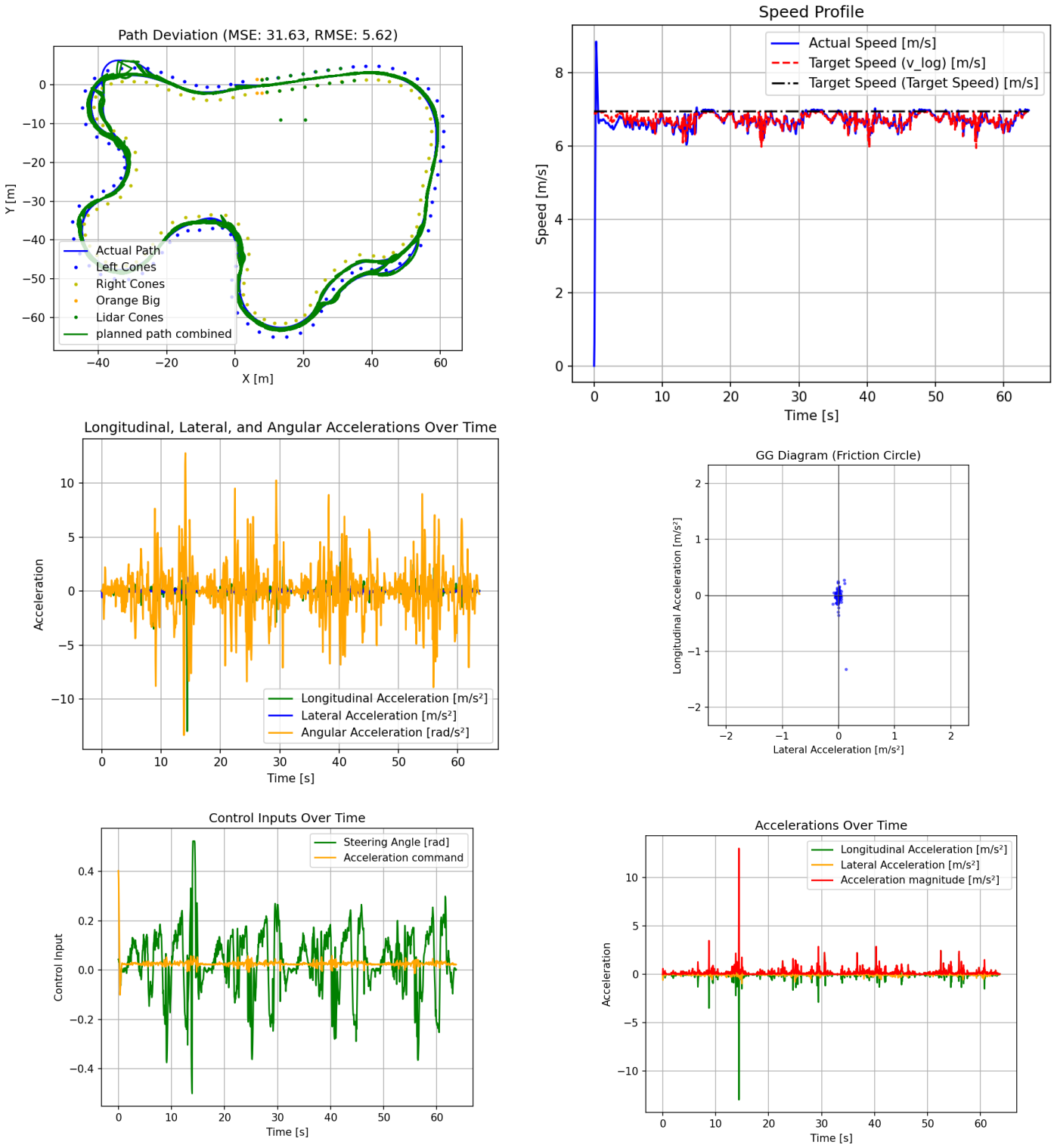


Figure 31 : Plots for map 3. On left we have Path tracking, accelerations and control inputs over time. On the right we have Speed profile, GG Diagram and longitudinal lateral acceleration and acceleration magnitude over time.

As we can see here, the third track was more challenging for our algorithm, it has likely couple of reasons, first, the turns are very sharp and often. Another thing that was shown was the fact that the path planner didn't properly match correctly the cones, so it diverged from the track. The way to solve it in future iterations is by taking the common approach of most teams which says that the first lap is a slow learning lap where all the cones are localized and then a global path for all the track is created.

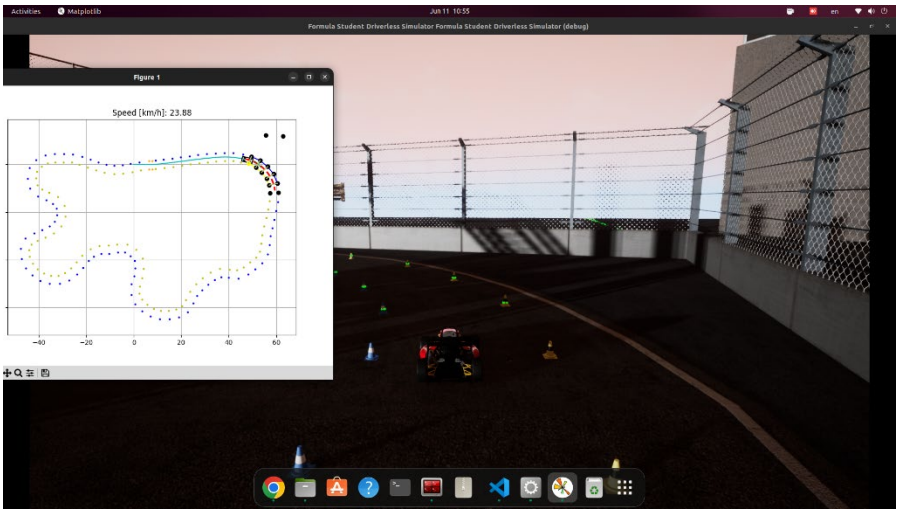


Figure 32 : Screenshot from the runtime of map 3, the setting affects the performance

6.3.5. Skipad:

Here the plots for the skipad will be shown, we don't need all the acceleration plots, so we would put here a screenshot from the actual frame of the run of the skipad and path tracking and acceleration plots.

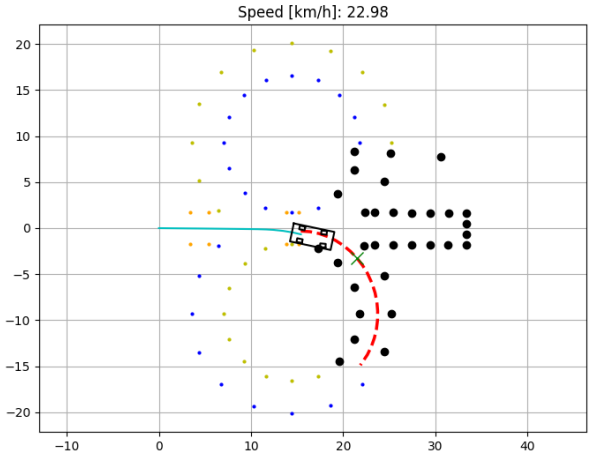


Figure 33 : Frame from the Skipad track.

The black dots are the cones that were detected by the LIDAR, as mentioned, the simulator simulates random measurement noise to test robustness, so this is why we have some artifacts that are false detections.

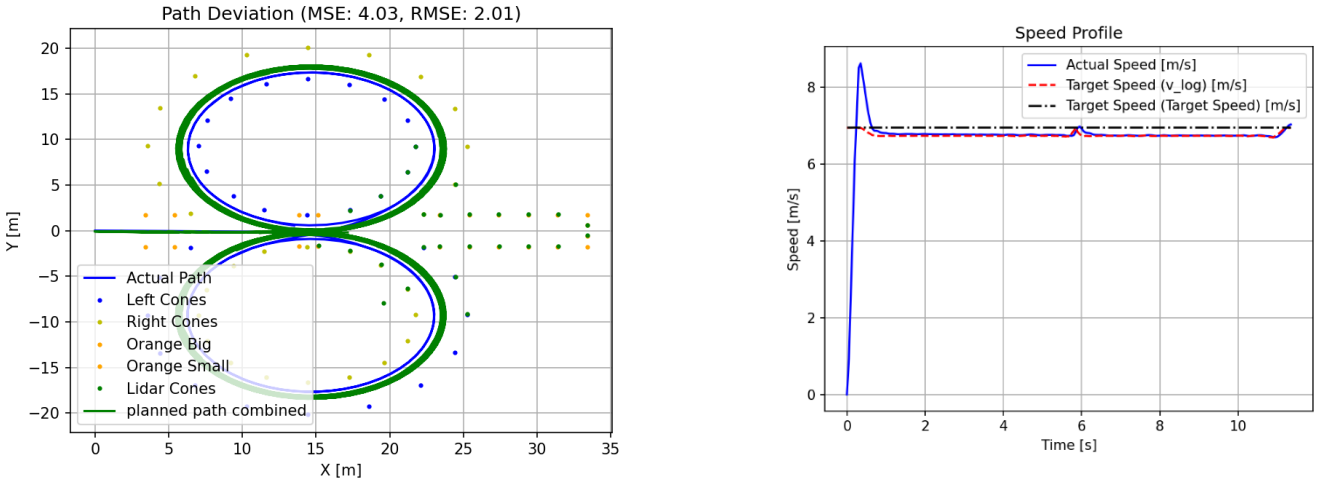


Figure 34 : Skipad plots. Path tracking (right) and Speed profile (left)

As we can see, also on the skipad run the controllers and the path planner worked as they should.

6.3.6. Acceleration:

Presented here are the plots for the Acceleration run, we would put here just the speed profile, and the control input plots because all the other plots aren't as informing. Well put here two versions of the speed profile and the control inputs, one for a run in 10 $\left[\frac{m}{s}\right]$ and another one when the speed is 50 [kph] which is ~ 14 $[\frac{\mathrm{m}}{\mathrm{s}}]$.

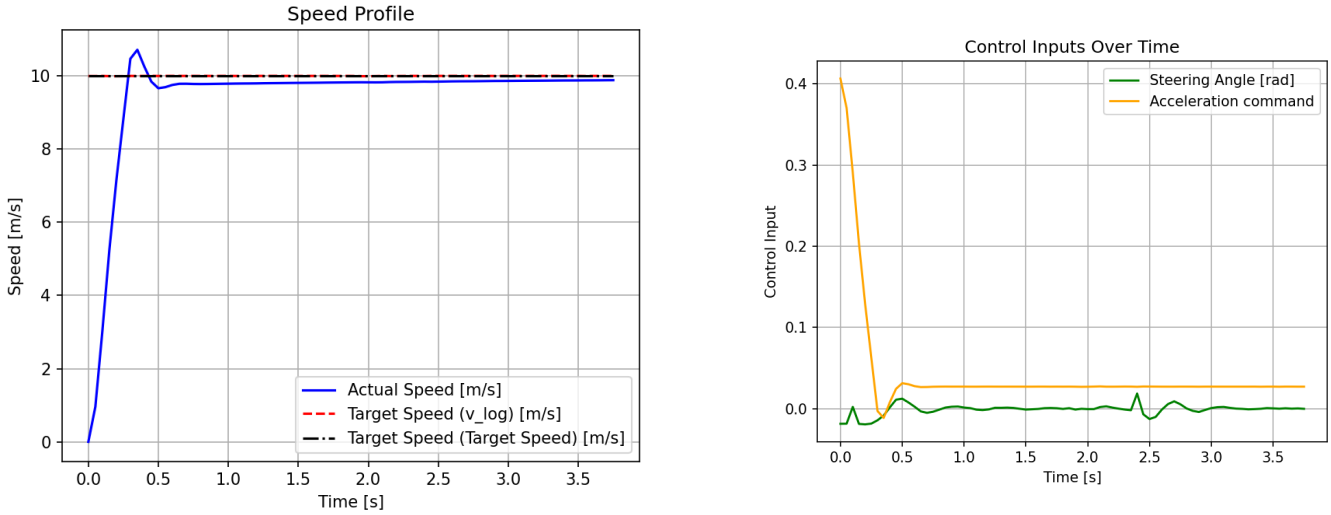


Figure 35 : Acceleration Plots on a test run of 10 [m/s]

As we can see, because its straight line the steering control inputs are close to zero. Also, the Vehicle top speed in the simulator is about 27 $[\frac{m}{s}]$ so it doesn't use the throttle as much.

Next, we would show the same plots for the run at higher speed

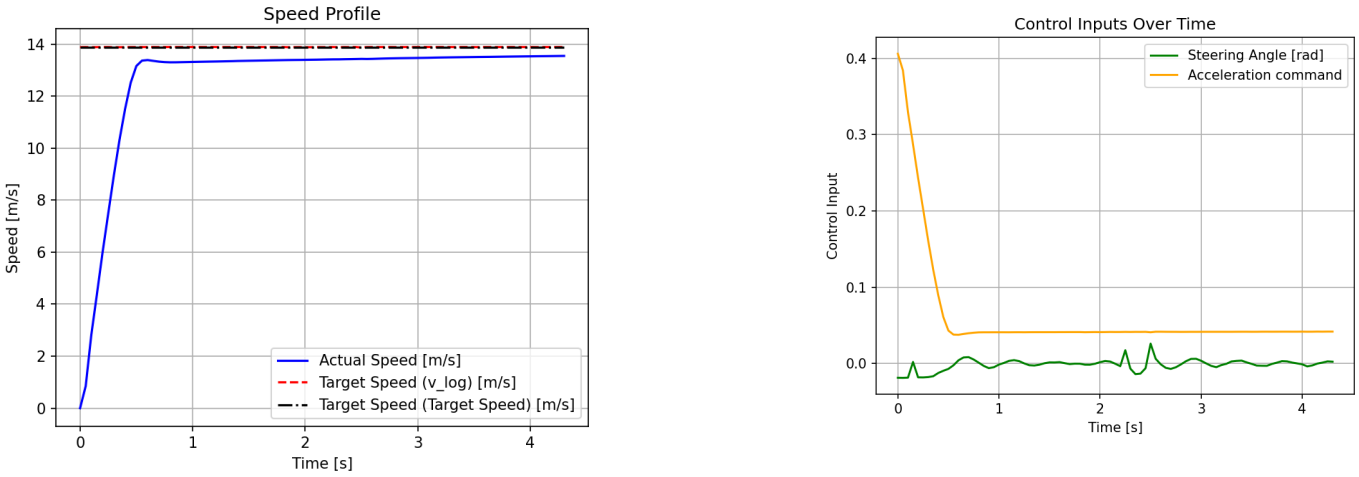


Figure 36 : Acceleration plots at 14 [m/s]

As shown here, even though the speed is much higher, the throttle control input aren't as big as compared to the last run, also we can see that on higher speed the PID gains create an under damped system this shows the need for an improvement in the speed controller either by implementing gain scheduling based on the speed or wither by transforming into an LQR controller that would change the gains dynamically in each call.

6.3.7. Lap times and Node times:

In this section lap times and run times would be shown.

Table 5 : Lap times in the runs and fallen cones

Run	Lap 1 $[s]$	Lap 2 $[s]$	Lap 3 $[s]$	Lap 4 $[s]$	Avg lap $[s]$	Cones
Map1	49.363	48.963	48.911	48.912	49.037	0
Map2	66.366	65.885	65.876	65.915	66.01	4
Map3	48.291	47.546	47.267	47.184	47.572	16
Skipad	8.134	7.989	7.997	7.880	8	2
Acceleration @ 10 $[\frac{m}{s}]$	7.633	7.637	7.623	7.645	7.634	0

Next, we would put on a table a small batch of the runtime logs, the avg run time and the frequency of the High-Level system. The logger measures the runtime of the reading of the algorithms (which includes the calculation of the path planner and control inputs) and the update time which includes the commands to the simulator and the feedback. The first index is the first call after the initializing of the algorithms which takes longer than the avg call.

Table 6 : Run time of read and update nodes.

#	Read $[s]$	Update $[s]$	Sum $[s]$
1	0.0174	0.0059	0.0233
2	0.0163	0.007	0.0233
3	0.0141	0.0071	0.0212
Avg	0.014	0.005	0.019

Seen here on the table that the runtimes for each segment are consistent, even though the sum and the avg are not the sum of all the indexes in the table above we can see that the avg runtime frequency is about 51.9 [Hz].

7. Summary

7.1. Overview

This project outline the decisions and the path that was taken to develop the first Israeli autonomous formula student vehicle, this project focused mainly on the path planning and control algorithms of the vehicle, the main scope of the project was to implement this GNC pipeline in a simulated environment, but happily, we out did the expectations and this project is being tested and implemented in the actual test vehicle as of right now. The path planning and control stack consist of mainly two parts. The first one is a path planner that is a modified version of a path planner used by a team from TU-Berlin and the other part is a decoupled control system that implements a curvature aware PID controller and Lateral geometric controllers such as Pure Pursuit and Stanley. We managed to surpass the requirements either based on runtime or accuracy, we found out that the Pure pursuit controller is more suitable for our needs mainly because of its simplicity and the fact that it's stable during low-speed maneuvers, unlike the Stanley who tends to oscillations and aggressive maneuvers.

Besides the controllers we've created a Series of performance plots and metrics to see how good the algorithms handle, we created a modular stack that is future proof to ROS2 implementation and included real world integration inside of the control stack or semi real-world control stack.



Figure 37 : Running the simulator and the test vehicle to test real world integration

7.2. Key Technical Achievements

The key achievements of this project were success of the curvature aware PID, the tracking of the Pure pursuit controller and the multi team integration that led us to implement this stack not just on the simulator but also on the actual car, including the integration and connection to the VCU (Vehicle control unit), LIDAR Module and the real-world sensors such as the IMU and the GPS.

7.3. Reasoning behind choices

The main objective of the project and of the team is to create a POC that would let us compete in the Fsd events this summer. Our reasoning stood behind 3 values:

1. Simplicity and reliability - Decoupled controllers (PID + Pure Pursuit/Stanley) were chosen over heavier MPC to ensure deterministic real-time execution on modest hardware, we also chose a "Plug and Play" Path planner and simulator so we could test on the go and be flexible along the process.
2. Simulator first approach – for half of the year the test vehicle didn't work and we didn't have a test bed to test our algorithms. We had to adapt and conduct thousands of tests in the simulator to make everything work.
3. Modularity and Future proof – The main code base changed his whole structure dozens of times, in the end, once we had a working pipeline, we changed the entire logic of the code base to make it as modular as possible so it could accompany multiple controllers, sensor integration and ROS2 structure to have an actual timestep and communication between each segment.

7.4. Lessons learned

The main lesson that was learned in this project is how hard is to implement a somewhat "simple" solution like we chose. You always have an edge case that you need to tackle and test

otherwise things would stop working at some point or another. The main thing though that was learned besides the whole implementation of known engineering concepts is how to work in a multi team coding pipeline and how to use GitHub in a proper way to manage versions that are either beta versions or production versions.

7.5. Future Improvements

This project can improve in multiple ways, including the following things:

1. In house path planning – the current version uses a modified version of "Fsd Path-Planning". An in-house path planner could benefit the team, but it requires a lot of effort to create an all-around path planner like we have right now.
2. Optimal control – implement an LQR based controller either for speed control or for longitudinal. [24],[25], [26]
3. Coupled control policies – The next logical step is to implement a coupled control system such as an MPC or even a learning-based MPC.[9] A great simple solution is to implement a nonlinear slip aware pure pursuit controller that would let the car drive aggressively and even implement future Car features like Torque vectoring.
4. RL Based control policies – Instead of using basic control laws, a growing number of groups and companies around the world are using model-based RL policies that implements the advantages of a dynamic model of the vehicle with the power of AI to find the drawbacks of each dynamic model and be more robust overall. [27],[26]
5. ROS2 Migration – as of right now, the communication between each componet is done in a very simplistic manner, levraging ROS2 would let us fist of all have proper timestamps and knowledge about each componet at any given time. Also, ROS2 has a lot of autonomus robots featurs that we can levrage. [28]

8. Plans and Risk Analysis:

In this section we'll have an update on the tasks that were shown in the Midterm report, their status and their updated dates and phases. As said before, all the goals were met, and the car would likely compete in this year's autonomous competition.

8.1. Future Development Plan

Table 7 : Future development plan

Task Name	Start Date	End Date	Phase	Status
Literature Review	01/08/2024	31/10/2024	Research	Completed
Define System Requirements	01/11/2024	15/11/2024	Planning	Completed
Feasibility Study and Risk Assessment	16/11/2024	30/11/2024	Planning	Completed
High-Level System Design	01/12/2024	20/12/2024	Design	Completed
Detailed Component Design	21/12/2024	10/01/2025	Design	Completed
Path Planner implementation (not sim env)	21/12/2024	15/01/2025	Integration	Completed
Pure Pursuit Controller development	11/01/2025	05/02/2025	Development	Completed
PID Speed Controller Development	06/02/2025	28/02/2025	Development	Completed
Simulator Setup and Calibration	21/03/2025	10/04/2025	Integration	Completed
Controller-Simulator Integration	11/04/2025	30/04/2025	Integration	Completed
Initial System Validation	01/05/2025	15/05/2025	Testing	Completed
Performance Tuning	16/05/2025	01/06/2025	Testing	Completed
Path Tracking Validation	16/05/2025	02/06/2025	Testing	Completed
Documentation and Final Report	01/05/2025	11/06/2025	Closing	Completed
Final Presentation Preparation	11/05/2025	12/06/2025	Closing	Completed

8.2. Risk analysis table

Task Name	Start Date	
Literature Review	01-08-2024	31-10-202
Define System Requirements	01/11/2024	15-11-202
Feasibility Study and Risk Assessment	16-11-2024	30-11-202
High-Level System Design	01-12-2024	20-12-202
Detailed Component Design	21-12-2024	10-01-202
Path Planner implementation	21/12/2024	15/01/20
Pure Pursuit Controller Implementation	11-01-2025	05-02-202
PID Speed Controller Development	06-02-2025	28-02-202
Simulator Setup and Calibration	21-03-2025	10-04-202
Controller-Simulator Integration	11-04-2025	30-04-202
Initial System Validation	01-05-2025	15-05-202
Performance Tuning	16-05-2025	15-06-202
Path Tracking Validation	16-06-2025	10-07-202
Documentation and Final Report	01-08-2025	15-08-202
Final Presentation Preparation	16-08-2025	31-08-202

Risk analysis Factor is a way to measure how important the risk is, the power of the risk is a multiplication of likelihood and impact.

Table 8 : Risk analysis table

8.3. Concluding Remarks

This project delivers a comprehensive proof-of-concept path planning and control framework that seamlessly processes raw cone detections and the car's state into a smooth B-spline centreline, applies curvature-aware PID speed control and Pure Pursuit or stanley steering, and issues real-world commands with the same code in both simulation and on-vehicle environments. As part of the BGR Racing team, it establishes a modular, collaboration-ready foundation for Israel's first autonomous student racer—primed for ROS 2 migration, hardware-in-the-loop validation, and competitive deployment.

Risk	Likelihood (1-5)	Impact (1-5)	Power of the Risk	Mitigation Strategy
Path Planner-Simulator Integration Challenges	3	4	12	Step-by-step debugging, using logging tools.
Controller tuning difficulties	4	3	12	Test different parameter sets, automate tuning.
Real-time simulation bottlenecks	4	5	20	Optimize computation, use profiling tools.
-	**Future considerations**	**Not Direct risks in scope**	**Of the project**	-
Scaling to real-world environments	3	4	12	Real-world implementation will require additional validation. Not fully in the scope of the project
Perception module delays	2	4	8	Use simulated referee state for initial testing. Not fully in the scope of this project
Limited test time before competition	3	5	15	Focus on critical validation early, prioritize testing scenarios. Not fully in the scope of this project

9. Bibliography

[1] O. Martínez, "Motion Planning and Control Pipeline for a Formula Student Autonomous Vehicle," 2024.

[2] "FSG Competition Handbook 2025."

[3] M. Shenoda, "Real-time Object Detection: YOLOv1 Re-Implementation in PyTorch," May 2023, Accessed: Jan. 30, 2025. [Online]. Available: https://arxiv.org/abs/2305.17786v1

[4] J. Betz *et al.*, "Autonomous Vehicles on the Edge: A Survey on Autonomous Vehicle Racing," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458–488, 2022, doi: 10.1109/OJITS.2022.3181510.

[5] "Kalman filter - Wikipedia." Accessed: Jan. 31, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Kalman_filter

[6] "אוטונומיה_הכשרות_מצגת_-_planning[1]".

[7] S. D. R. Santos, J. R. Azinheira, M. A. Botto, and D. Valério, "Path Planning and Guidance Laws of a Formula Student Driverless Car," *World Electric Vehicle Journal*, vol. 13, no. 6, Jun. 2022, doi: 10.3390/wevj13060100.

[8] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, "Kinematic and dynamic vehicle models for autonomous driving control design," *IEEE Intelligent Vehicles Symposium, Proceedings*, vol. 2015-August, pp. 1094–1099, Aug. 2015, doi: 10.1109/IVS.2015.7225830.

[9] J. ; Kabzan, L. ; Hewing, A. ; Liniger, and M. N. Zeilinger, "Learning-based Model Predictive Control for Autonomous Racing," 2019, doi: 10.3929/ethz-b-000351561.

[10] J. A. Matute, R. Lattarulo, A. Zubizarreta, and J. Perez, "A comparison between coupled and decoupled vehicle motion controllers based on prediction models," *IEEE Intelligent Vehicles Symposium, Proceedings*, vol. 2019-June, pp. 1843–1848, Jun. 2019, doi: 10.1109/IVS.2019.8814256.

[11] S. Li, Z. Sun, Y. Liu, A. Ji, S. Zhang, and L. Chen, "Coupled lateral and longitudinal trajectory tracking control with a modified vehicle kinematics model for autonomous vehicles," *Intelligent Transportation Infrastructure*, vol. 2, pp. 1–11, May 2023, doi: 10.1093/ITI/LIAD009.

[12] Y. Kebbati, N. A. Oufroukh, D. Ichalal, V. Vigneron, and N. Ait-Oufroukh, "Lateral control for autonomous wheeled vehicles: a technical review Yassine Kebbati, Naïma Aït Oufroukh, Dalil Ichalal, Vincent Vigneron. Lateral control for au-tonomous wheeled vehicles: a technical review Lateral control for autonomous wheeled vehicles: a

technical review," *Asian J Control*, vol. 2023, no. 4, pp. 2539–2563, doi: 10.1002/asjc.2980ï.

[13] C. Cheung, A. Mohammadi, S. Rawashdeh, and S. Baek, "Delivery of Healthcare Resources Using Autonomous Ground Vehicle Convoy Systems: An Overview," *Front Robot AI*, vol. 8, p. 611978, Aug. 2021, doi: 10.3389/FROBT.2021.611978/BIBTEX.

[14] "Self-Driving Cars | Coursera." Accessed: Jun. 08, 2025. [Online]. Available: https://www.coursera.org/specializations/self-driving-cars

[15] "qiaoxu123/Self-Driving-Cars: Coursera Open Courses from University of Toronto." Accessed: Jun. 08, 2025. [Online]. Available: https://github.com/qiaoxu123/Self-Driving-Cars/tree/master

[16] "Three Methods of Vehicle Lateral Control: Pure Pursuit, Stanley and MPC | by Yan Ding | Medium." Accessed: Jan. 28, 2025. [Online]. Available: https://dingyan89.medium.com/three-methods-of-vehicle-lateral-control-pure-pursuit-stanley-and-mpc-db8cc1d32081

[17] P. Gautam, S. Sahai, S. S. Kelkar, P. S. Agrawal, and M. R. D, "Designing Variable Ackerman Steering Geometry for Formula Student Race Car," *International Journal of Analytical, Experimental and Finite Element Analysis (IJAEFEA)*, vol. 8, no. 1, Feb. 2021, doi: 10.26706/ijaefea.1.8.20210101.

[18] "FS-Driverless/Formula-Student-Driverless-Simulator: A virtual world where Autonomous Systems from different Formula Student teams can compete in time-trial challenges." Accessed: Jan. 28, 2025. [Online]. Available: https://github.com/FS-Driverless/Formula-Student-Driverless-Simulator

[19] "papalotis/ft-fsd-path-planning: Formula Student Driverless Path Planning Algorithm. Colorblind centerline calculation algorithm developed by FaSTTUBe. It introduces a novel approach that uses neither Delaunay Triangulation nor RRT." Accessed: Jan. 28, 2025. [Online]. Available: https://github.com/papalotis/ft-fsd-path-planning/tree/main

[20] J. Becker, N. Imholz, L. Schwarzenbach, E. Ghignone, N. Baumann, and M. Magno, "Model- and Acceleration-based Pursuit Controller for High-Performance Autonomous Racing," Sep. 2022, doi: 10.1109/ICRA48891.2023.10161472.

[21] "simple-pid · PyPI." Accessed: Jan. 28, 2025. [Online]. Available: https://pypi.org/project/simple-pid/

[22] V. V. Patel, "Ziegler-Nichols Tuning Method: Understanding the PID Controller," *Resonance*, vol. 25, no. 10, pp. 1385–1397, Oct. 2020, doi: 10.1007/S12045-020-1058-Z/METRICS.

[23] I. Georgiev, C. Chatzikomis, T. Völkl, J. Smith, and M. Mistry, "Iterative Semi-parametric Dynamics Model Learning For Autonomous Racing," Nov. 2020, Accessed: Jun. 11, 2025. [Online]. Available: http://arxiv.org/abs/2011.08750

[24] O. Arslan, K. Berntorp, and P. Tsiotras, "Sampling-based algorithms for optimal motion planning using closed-loop prediction," *Proc IEEE Int Conf Robot Autom*, pp. 4991–4996, Jul. 2017, doi: 10.1109/ICRA.2017.7989581.

[25] T. Yuan and R. Zhao, "LQR-MPC-Based Trajectory-Tracking Controller of Autonomous Vehicle Subject to Coupling Effects and Driving State Uncertainties," *Sensors 2022, Vol. 22, Page 5556*, vol. 22, no. 15, p. 5556, Jul. 2022, doi: 10.3390/S22155556.

[26] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Sci Robot*, vol. 8, no. 82, Sep. 2023, doi: 10.1126/SCIROBOTICS.ADG1462/SUPPL_FILE/SCIROBOTICS.ADG1462_MOVIES_S1_TO_S4.ZIP.

[27] S. Asri and L. Rodrigues, "Data-Driven LQR using Reinforcement Learning and Quadratic Neural Networks," Nov. 2023, Accessed: Jun. 11, 2025. [Online]. Available: https://arxiv.org/pdf/2311.10235

[28] "Nav2 — Nav2 1.0.0 documentation." Accessed: Jun. 11, 2025. [Online]. Available: https://docs.nav2.org/

-

10. Appendix

10.1. Appendix A -Path Planner full pipeline

Pseudocode for Path Planning Pipeline

1. **Initialization**

   Input Parameters:

   Mission type (e.g., skidpad, acceleration,)

   Vehicle position and direction

   List of cone positions

2. **Initialize Components**:

Relocalizer (optional, based on mission type)

Cone sorting module

Cone matching module

Path calculation module

Set Global Path (Optional):

If a global path is predefined, set it for relocalization.

3. **Main Path Planning Pipeline**

Input Validation:

Convert vehicle direction to an array format (e.g., 2D unit vector).

Relocalization Step (if applicable):

Attempt to relocalize using cone positions, vehicle position, and direction.

If successful:

Transform vehicle position and yaw to a known map frame.

Update the global path using relocalizer's data.

Return placeholders for sorted cones and matches (skip further steps).

4. **Cone Sorting**:

Input: Cone positions, vehicle position, and direction.

Process: Use sorting module to separate cones into left and right groups.

Output: Sorted left and right cones.

5. **Cone Matching**:

Input: Sorted left and right cones.

Process: Match left cones to right cones using the matching module.

Output: Virtual cones and match indices for left-to-right and right-to-left.

6. **Path Calculation**:

Input: Virtual cones, match indices, vehicle position, and direction.

Process: Generate a smooth path using the path calculation module.

Output: Final path (Nx4 array with spline parameter, X, Y, curvature).

Relocalization Adjustment

If Relocalized:

Convert final path points from the known map frame back to the global frame.

Update path points with original frame coordinates.

7. Return Outputs

Final path as an Nx4 array (spline parameter, X, Y, curvature).

*__Intermediate Results -__ If enabled, the algorithm can provide intermediate results such as sorted cones, matched cones, and intermediate path calculations for debugging and visualization purposes.

- Final path
- Sorted left cones
- Sorted right cones
- Virtual cones (left and right)
- Match indices (left-to-right and right-to-left).

10.2. Appendix B - Pure Pursuit pseudo code

Pure Pursuit Algorithm

Input: Vehicle position (x, y), heading θ, look-ahead distance d, path points P

Output: Steering angle δ

1 .Find the nearest point on the path to the vehicle's current position.

2 .Determine the look-ahead point on the path, which is at distance d ahead of the vehicle.

3 .Calculate the angle α between the vehicle's heading θ and the line connecting the vehicle to the look-ahead point.

4 .Compute the curvature $\kappa = 2 \cdot \frac{sin(\alpha)}{d}$. *for speed-based look-ahead d=$K_{lad} \cdot V$

5 .Calculate the steering angle δ = arctan (L · κ), where L is the wheelbase.

6. Apply δ to the steering system.

10.3. Appendix C – Stanley controller pseudo code

Stanley controller

Input: Vehicle position (x, y), heading θ, velocity v, path points P.

Output: Steering angle δ

1 .Find the nearest point on the path to the vehicle's current position.

2 .Calculate the cross-track error e, the perpendicular distance from the vehicle to the path.

3 .Compute the heading error $\theta_e = \theta_{path} - \theta$, where θ_{path} is the tangent angle of the path at the nearest point.

4 .Combine errors into a control law:

$\delta = \theta_e + \arctan\left(k \cdot \frac{e}{v+k_{soft}}\right)$, where k is a tuning parameter.

5 .Apply δ to the steering system.

10.4. Apendix D – Main.py

```
#!/usr/bin/env python3
"""
Run either a simulation or calibration session with a pluggable provider/
nodes pipeline.  Use --help for options.
"""
import sys
import os

# Add project root to sys.path
project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
sys.path.insert(0, project_root)

import argparse
import logging
from core.manager import Manager
from providers.sim.car_state import SimCarStateProvider
from providers.sim.cones     import SimConeProvider
from providers.sim.map_data  import SimMapProvider
from providers.sim.referee_state import SimRefereeStateProvider
from providers.lidar.cones   import LidarConeProvider
# Swap these two for ROS2 later:
# from providers.ros.car_state import ROSCarStateProvider
# from providers.ros.cones     import ROSConeProvider
# from providers.ros.map_data  import ROSMapProvider
from nodes.planner     import Planner
from nodes.controller  import Controller
from core.logger import init_logger
from providers.gps_imu.python.gps_imu_provider  import GPSIMUProvider




def cli() -> argparse.Namespace:
    p = argparse.ArgumentParser(
        description="Modular path-planning & control framework")
    p.add_argument("--mode",    choices=["simulation", "calibration"],
                   default="simulation")
    p.add_argument("--plot",    action="store_true",
                   help="Enable live plots / post-run analysis")
    p.add_argument("--dt", type=float, default=0.05,
                   help="Main-loop period [s]")
    p.add_argument("--output_dir", type=str, default="output",
                   help="Output directory for plots and data")
    p.add_argument("--simulation", action="store_true",)
    p.add_argument('--steering-controller',choices=['pure_pursuit',
'stanley'], default='pure_pursuit',
```

```
                    help='Choose steering controller type')
    p.add_argument("--global_ref", action="store_true",
                    help="global refrence frame for path planning or car
reference frame")

    return p.parse_args()

def build_manager(args):
    # Providers (simulation flavour)
    if args.simulation:
        providers = [
            SimCarStateProvider(is_global_frame=args.global_ref),      #
vehicle pose & twist
            SimConeProvider(is_global_frame=args.global_ref),          #
simulated cones
            SimRefereeStateProvider(is_global_frame=args.global_ref),  #
referee state (Optional)
            SimMapProvider() # map data (Optional)
        ]
    else:
        providers = [
            # SimCarStateProvider(),      # vehicle pose & twist
            LidarConeProvider(),          # LiDAR-derived track cones
            GPSIMUProvider(),          # GPS/IMU-derived vehicle pose
        ]

    nodes = [
        Planner(),               # produces path based on cones & map
        Controller(simulation=args.simulation,is_global_frame=args.global_ref,
steering_controller=args.steering_controller)             # produces throttle
/ steer commands
    ]

    return Manager(providers, nodes, dt=args.dt, enable_plots=args.plot,
output_dir=args.output_dir)

def main():
    args = cli()
    # logging.basicConfig(level=logging.INFO,
    #                     format="%(asctime)s %(levelname)s %(name)s:
%(message)s")
    init_logger()
    mgr = build_manager(args)

    if args.mode == "calibration":
        mgr.run_calibration()
    else:
        mgr.run()
```

```
if __name__ == "__main__":
    main()
```

10.5. Apendix E - Manager.py

```
import time
import logging
from typing import Dict, List, Protocol, Any
from core.data.plot_data import PlotData
from core.visualization import PlotManager
from pathlib import Path
from core.logger import log_timing
import numpy as np
from icecream import ic

log = logging.getLogger("ControlManager")


class Manager:
    def __init__(self, providers, nodes, dt, enable_plots, output_dir,
simulation=False):
        log.info("Initializing Control Manager")
        self.providers = providers
        self.nodes = nodes
        self.dt = dt
        self.enable_plots = enable_plots
        self.plot_data = PlotData()
        self.plotter = PlotManager(live=enable_plots)
        self.output_dir = Path(output_dir)
        self.simulation = simulation

        log.debug(f"Configuration - dt: {dt}, plots enabled: {enable_plots}, "
                  f"output directory: {output_dir}")
        log.debug(f"Initialized with {len(providers)} providers and
{len(nodes)} nodes")

    def run(self):
        log.info(f"Starting main control loop - dt={self.dt:.3f}s,
plots={self.enable_plots}")

        try:
            t_prev = time.perf_counter()
            iteration = 0

            while True:
                iteration += 1
                now = time.perf_counter()
```

```
                # Timing control
                if now - t_prev < self.dt:
                    time.sleep(0.001)
                    continue

                dt = now - t_prev
                t_prev = now

                if dt > self.dt * 1.5:  # Warn if loop takes too long
                    log.warning(f"Loop iteration {iteration} took {dt:.3f}s "
                                f"(target: {self.dt:.3f}s)")
                # Read from providers
                t_read_providers = time.perf_counter()
                try:
                    data = {}
                    for p in self.providers:
                        start_lidar = time.time()
                        provider_data = p.read()
                        print(f"Provider {p.__class__.__name__} read time:
{time.time() - start_lidar:.4f}s")

                        log.debug(f"Read from {p.__class__.__name__}: "
                                  f"{len(provider_data) if provider_data else 0}
items")
                        data.update(provider_data)
                except Exception as e:
                    log.error(f"Error reading from providers: {str(e)}",
exc_info=True)
                    raise

                log_timing("read", time.perf_counter() - t_read_providers)
                # Update nodes
                t_update_nodes = time.perf_counter()
                try:
                    for n in self.nodes:
                        n.update(data)
                        log.debug(f"Updated node {n.__class__.__name__}")
                except Exception as e:
                    log.error(f"Error updating nodes: {str(e)}",
exc_info=True)
                    raise
                log_timing("update_nodes", time.perf_counter() -
t_update_nodes)
                data["current_time"] = now


                if self.enable_plots:
                    self._update_plot_data(data)
```

```
                    try:
                        self.plotter.update({"plot_data": self.plot_data})
                        log.debug("Plot data updated successfully")
                    except Exception as e:
                        log.error(f"Error updating plots: {str(e)}",
exc_info=True)
                    if self.simulation and
self.laps_limit(data.get("referee_state"), max_laps=4):
                            self._generate_final_plots()
                            self._cleanup()
                            break

        except KeyboardInterrupt:
            log.info("Received keyboard interrupt, stopping control loop")
            self._generate_final_plots()
        except Exception as e:
            log.error(f"Unexpected error in control loop: {str(e)}",
exc_info=True)
            raise
        finally:
            self._cleanup()

    def run_calibration(self):
        log.info("Starting calibration routine")
        # Implement calibration steps here
        pass

    def laps_limit(self, referee_state, max_laps: int = 1):
        """Limit the number of laps for simulation purposes."""
        limit_reached = len(referee_state.laps ) >= max_laps
        if limit_reached:
            log.info(f"Maximum laps reached: {len(referee_state.laps)} >=
{max_laps}")
            log.info("Stopping simulation")
            log.info(f"referee_state.laps: {referee_state.laps}")
            log.info(f"cones dropped: {referee_state.doo_counter}")
        return limit_reached

    def _cleanup(self):
        """Clean up resources and shut down components."""
        log.info("Cleaning up resources")
        try:
            for n in self.nodes:
                try:
                    if hasattr(n, 'stop'):
                        n.stop()
                    log.debug(f"Node {n.__class__.__name__} finished
successfully")
```

```
                except Exception as e:
                    log.error(f"Error finishing node {n.__class__.__name__}:
{str(e)}",
                              exc_info=True)

            for p in self.providers:
                try:
                    if hasattr(p, 'stop'):
                        p.stop()
                    log.debug(f"Provider {p.__class__.__name__} stopped
successfully")
                except Exception as e:
                    log.error(f"Error stopping provider
{p.__class__.__name__}: {str(e)}",
                              exc_info=True)
        except Exception as e:
            log.error(f"Error during cleanup: {str(e)}", exc_info=True)

    def _generate_final_plots(self):
        """Generate and save all plots at the end of the run."""
        log.info("Generating final plots...")
        uid = time.strftime("%Y-%m-%d_%H-%M-%S")
        self.plotter.export_to_video(f"output/videos/video_{uid}.mp4")
        try:
            # Create plots directory
            plots_dir = self.output_dir / "plots"
            plots_dir.mkdir(parents=True, exist_ok=True)

            # Generate all available plots
            if self.plot_data.states is not None and self.plot_data.cx is not
None and self.plot_data.cy is not None:
                # Path tracking plot
                self.plotter.show(self.plot_data.cx, self.plot_data.cy,
self.plot_data.states)
                self.plotter.plot_path_deviation1(self.plot_data.cx,
self.plot_data.cy, self.plot_data.states, self.plot_data.X,
self.plot_data.Y,self.plot_data.cones_map,self.plot_data.cones_lidar)

                # Speed profile
                self.plotter.plot_speed_profile(self.plot_data.states)

                # Control inputs
                self.plotter.plot_control_inputs(self.plot_data.states)

                # Path deviation
                if self.plot_data.full_path:
                    self.plotter.plot_path_deviation(
                        self.plot_data.cx,
```

```
                        self.plot_data.cy,
                        self.plot_data.states,
                        self.plot_data.full_path
                    )
                # TODO: add to carState all atributes needed for the plots
                # # Acceleration plots
                # self.plotter.plot_all_accelerations(self.plot_data.states)
                self.plotter.plot_gg(self.plot_data.states)

                #plot acceleration
                self.plotter.plot_acceleration(self.plot_data.states)

                #plot all accelerations
                self.plotter.plot_all_accelerations(self.plot_data.states)

                # Cross track error
                if hasattr(self.plot_data, 'cte_history') and
len(self.plot_data.cte_history) > 0:
                    self.plotter.plot_cte()

            # Save all generated plots
            self.plotter.save_all(plots_dir)
            log.info(f"All plots saved to {plots_dir}")

        except Exception as e:
            log.error(f"Error generating final plots: {str(e)}",
exc_info=True)

    def _update_plot_data(self, data: Dict[str, Any]):
        """Update plot data from node outputs."""
        self.plot_data.state = data.get("car_state")
        self.plot_data.states = data.get("states")
        self.plot_data.path = data.get("path")
        self.plot_data.cones_map = data.get("cones_map")
        self.plot_data.cones_lidar = data.get("cones_lidar")
        self.plot_data.cx = data.get("cx")
        self.plot_data.cy = data.get("cy")
        self.plot_data.acceleration = data.get("acceleration")
        self.plot_data.steering = data.get("steering")
        self.plot_data.target_ind = data.get("target_ind")
        self.plot_data.v_log = data.get("v_log")
        self.plot_data.full_path = data.get("full_path", [])
        path_track = np.column_stack((np.arange(len(data.get("cx"))),
data.get("cx"), data.get("cy"))) #List of XY cords of track
        self.plot_data.X.append(path_track[:5,1])
        self.plot_data.Y.append(-path_track[:5,2])
        self.plot_data.intermediate = data.get("intermediate_results", {})
```

10.6. Apendix F – controller.py

```
from typing import Dict
import numpy as np
import logging
from vehicle_config import Vehicle_config as conf
from core.controllers import AccelerationPIDController,
PurePursuitController,StanleyController
from providers.sim.sim_util import sim_car_controls    # wrapper for AirSim
command
from providers.sim.sim_util import FSDSClientSingleton
# from visualization import PlotManager
from core.data.car_state import State, States
from core.vcu import VCU, cleanup_vcus
import time
from core.serial_setup import get_devices

# Get logger for this module
log = logging.getLogger("Controller")

class Controller:

    def __init__(self, simulation=False, is_global_frame=True,
steering_controller="pure_pursuit"):
        log.info("Initializing Controller with PID and Pure Pursuit
controllers")
        self.accel_ctrl = AccelerationPIDController(
            kp=conf.kp_accel,
            ki=conf.ki_accel,
            kd=conf.kd_accel,
            setpoint=conf.TARGET_SPEED
        )

        # Initialize steering controller based on user choice
        if steering_controller.lower() == "stanley":
            from core.controllers import StanleyController
            self.steer_ctrl = StanleyController(k=conf.k_stanley,
k_soft=conf.k_soft_stanley)
            log.info("Using Stanley steering controller")
        elif steering_controller.lower() == "pure_pursuit":
            self.steer_ctrl =
PurePursuitController(look_ahead_distance=conf.LOOKAHEAD_DISTANCE)
            log.info("Using Pure Pursuit steering controller")
        else:
            log.warning(f"Unknown steering controller '{steering_controller}',
defaulting to Pure Pursuit")
            self.steer_ctrl =
PurePursuitController(look_ahead_distance=conf.LOOKAHEAD_DISTANCE)
```

```
        self.steering_controller_type = steering_controller
        self.states = States()
        self.simulation = simulation
        if not self.simulation:
            devices = get_devices()  # Initialize devices, this will set up
CAN if GPS_IMU is connected
            steering_port = devices.get("steering")
            torque_port = devices.get("torque")
            self.vcu_steering = VCU(port_name=steering_port) if steering_port
else None
            self.vcu_tourqe = VCU(port_name=torque_port) if torque_port else
None

        self.first_time = True
        self.is_global_frame = is_global_frame

        log.debug(f"Controller initialized with target speed:
{conf.TARGET_SPEED}, "
                  f"lookahead distance: {conf.LOOKAHEAD_DISTANCE}")


    def update(self, data: Dict):
        try:
            path = data.get("path")
            cx = data.get("cx")
            cy = data.get("cy")
            curve = data.get("curve")
            car_state = data.get("car_state")
            referee_state = data.get("referee_state")
            target_ind = data.get("target_ind")
            gps_speed = data.get("gps_data")
            if gps_speed is not None:
                gps_speed = gps_speed.gps_speed # km/h
                gps_speed = gps_speed / 3.6 # convert to m/s



            if self.simulation:
                if any(x is None for x in [path, cx, cy, car_state]):
                    log.warning("Missing required data for controller update:
"
                                f"path={path is not None}, cx={cx is not None}, "
                                f"cy={cy is not None}, car_state={car_state is not
None}")
                    return
            # if not self.simulation:
                # create car state for gps velocity and 0,0 x y and 0 yaw
because the world is coming to the car
            if not self.is_global_frame:
```

```
                car_state = State(
                    x=0,
                    y=0,
                    yaw=0,
                    v=car_state.v)

            state_modifier = State(
                x=car_state.x,
                y=-car_state.y,
                yaw=car_state.yaw,
                v=car_state.v,
                v_linear= car_state.v_linear,
                v_angular= car_state.v_angular,
                a_angular= car_state.a_angular,
                a_linear= car_state.a_linear,
                timestamp=car_state.timestamp
            )


            path_track = np.column_stack((np.arange(len(cx)), cx, cy))
            log.debug(f"Car state - Position: ({car_state.x:.2f},
{car_state.y:.2f}), "
                      f"Yaw: {car_state.yaw:.2f}, Velocity: {car_state.v:.2f}")

            # Calculate control inputs
            if self.steering_controller_type.lower() == "stanley":
                # Stanley controller needs dt parameter
                dt = conf.dt  # or get from data if available
                steering, target_ind =
self.steer_ctrl.compute_steering(car_state, path_track, target_ind, dt)
            else:
                # Pure Pursuit controller
                steering, target_ind =
self.steer_ctrl.compute_steering(car_state, path_track, target_ind)

            # steering, target_ind =
self.steer_ctrl.compute_steering(car_state, path_track, target_ind)
            curvature = curve[target_ind] if target_ind < len(curve) else
curve[-1]
            acceleration = self.accel_ctrl.compute_acceleration(car_state.v,
curvature)

            current_time = data.get("current_time", 0)
            self.states.append(
                current_time,
                state_modifier,
                steering if steering else 0.0,
                acceleration if acceleration else 0.0,
                self.accel_ctrl.maxspeed if self.accel_ctrl.maxspeed else 0.0,
```

```
                v_linear= car_state.v_linear,
                v_angular=car_state.v_angular,
                a_linear=car_state.a_angular,
                a_angular=car_state.a_linear,
                timestamp=car_state.timestamp
                )
            data["states"] = self.states # expose to other nodes
            # Log control decisions
            log.debug(f"Control outputs - Steering: {steering:.3f},
Acceleration: {acceleration:.3f}, "
                      f"Target Index: {target_ind}, Curvature:
{curvature:.3f}")

            # expose to other nodes
            data["v_log"] = self.accel_ctrl.maxspeed
            data["acceleration"] = acceleration
            data["steering"] = steering

            # Apply controls and log the action
            log.info(f"Applying controls - Steering: {-steering:.3f},
Acceleration: {acceleration:.3f}")

            self.apply_controls(steering, acceleration)

        except Exception as e:
            log.error(f"Error in controller update: {str(e)}", exc_info=True)
            raise  # Re-raise the exception after logging

    def stop(self):
        log.info("Cleaning up Controller resources")
        if not self.simulation:
            cleanup_vcus(self.vcu_steering, self.vcu_tourqe)
        else:
            log.info("Simulation mode, no VCU cleanup required")

    def apply_controls(self, steering: float, acceleration: float):
        """
        Apply control commands to the vehicle.
        """
        if self.simulation:
            sim_car_controls(FSDSClientSingleton.instance(), -steering,
acceleration)
        else:
            if self.vcu_tourqe is not None:
                if self.first_time:
                    self.vcu_tourqe.set_torque(18.0)
                    self.vcu_steering.set_angle(0.0)
                    time.sleep(0.8)  # Allow time for VCU to initialize
```

```
                    self.first_time = False
                else:
                    self.vcu_tourqe.set_torque(acceleration*5.5)
            if self.vcu_steering is not None:
                vcu_send_steering_angle = float(np.rad2deg(steering)*4.5)
                if vcu_send_steering_angle>= 75:
                    vcu_send_steering_angle=float(75)
                elif vcu_send_steering_angle<=-75:
                    vcu_send_steering_angle=float(-75)
                self.vcu_steering.set_angle(vcu_send_steering_angle)
```

10.7. Apendix G – planner.py

```
from typing import Dict, List
import logging
import numpy as np
from vehicle_config import Vehicle_config as conf
from fsd_path_planning import PathPlanner, MissionTypes, ConeTypes
from types import SimpleNamespace

log = logging.getLogger("Planner")

class Planner:
    def __init__(self):
        log.info("Initializing Path Planner for trackdrive mission")
        self.planner = PathPlanner(MissionTypes.trackdrive)
        # self.planner = PathPlanner(MissionTypes.skidpad)
        self.current_path = None
        self.target_ind = 0
        log.debug("Path Planner initialized successfully")

    def update(self, data: Dict):
        try:
            cones, car_position, car_direction = data.get("cones")

            if cones is None:
                log.warning("No cone data available for path planning")
                return

            # Check if new path calculation is needed
            should_recalculate = (self.current_path is None or self.target_ind
> 3)
            if should_recalculate:
                log.info(f"Recalculating path (target_ind={self.target_ind})")

                # Log the planning inputs
                log.debug(f"Planning inputs - Car position: {car_position}, "
                          f"Car direction: {car_direction}, "
```

```
                        f"Number of cones: {sum(len(cone_group) for
cone_group in cones)}")

                # Calculate new path
                try:

                    out = self.planner.calculate_path_in_global_frame(
                        cones, car_position, car_direction,
return_intermediate_results=True
                    )

                    (
                        path,
                        sorted_left,
                        sorted_right,
                        left_cones_with_virtual,
                        right_cones_with_virtual,
                        left_to_right_match,
                        right_to_left_match,
                    ) = out
                    self.current_path = path
                    all_cones = cones[ConeTypes.UNKNOWN]

                    data["intermediate_results"] = {
                        "left_cones_with_virtual": left_cones_with_virtual,
                        "right_cones_with_virtual": right_cones_with_virtual,
                        "left_to_right_match": left_to_right_match,
                        "right_to_left_match": right_to_left_match,
                        "all_cones": all_cones,
                    }
                    log.info("Path calculation successful")
                    self.target_ind = 0
                except Exception as e:
                    log.error(f"Path calculation failed: {str(e)}",
exc_info=True)
                    return

            # Extract path coordinates
            cx, cy = self.current_path[:, 1], -self.current_path[:, 2]

            # Update target index
            prev_target_ind = self.target_ind
            while self.target_ind < len(cx) - 1:
                distance = np.hypot(cx[self.target_ind] -
data.get("car_state", SimpleNamespace(**{'x':0})).x,
                                    cy[self.target_ind] - data.get("car_state",
SimpleNamespace(**{'y':0})).y)
                if distance >= conf.LOOKAHEAD_DISTANCE:
```

```
                    break
                self.target_ind += 1

            if self.target_ind != prev_target_ind:
                log.debug(f"Target index updated: {prev_target_ind} ->
{self.target_ind}")

            # Process path data based on intermediate results flag
            if data.get("return_intermediate_results"):
                log.debug("Processing intermediate results")
                cx, cy = self.current_path[0][:, 1], -self.current_path[0][:,
2]
                curve = self.current_path[0][:, 3]
            else:
                cx, cy = self.current_path[:, 1], -self.current_path[:, 2]
                curve = self.current_path[:, 3]

            # Log path statistics
            log.debug(f"Path statistics - Points: {len(cx)}, "
                      f"Target index: {self.target_ind}, "
                      f"Average curvature: {np.mean(curve):.3f}")

            # Expose data to other nodes
            data["path"] = self.current_path
            data["cx"] = cx
            data["cy"] = cy
            data["curve"] = curve
            data["target_ind"] = self.target_ind
            data["return_intermediate_results"] = False

        except Exception as e:
            log.error(f"Error in planner update: {str(e)}", exc_info=True)
            raise  # Re-raise the exception after logging
```

10.8. Apendix H – car_state.py

```
from typing import Dict
import numpy as np
from providers.sim.sim_util import FSDSClientSingleton
from sub_modules.fsds.client import FSDSClient   # your existing AirSim
wrapper
from sub_modules.fsds.utils import to_eularian_angles
from core.data.car_state import State
import logging

log = logging.getLogger("SimLogger")
class SimCarStateProvider:
    def __init__(self, is_global_frame: bool = True):
```

```
        self.client = FSDSClientSingleton.instance()
        self.is_global_frame = is_global_frame
        log.debug(f"Provider {self.__class__.__name__} initialized
successfully")


    def start(self):  pass
    def stop(self):   pass

    def read(self) -> Dict[str, np.ndarray]:
        car_state = self.client.getCarState()
        referee_state = self.client.getRefereeState()

        orientation = car_state.kinematics_estimated.orientation
        _, _, yaw = to_eularian_angles(orientation)
        yaw = (yaw + np.pi) % (2 * np.pi) - np.pi #normalize_yaw
        v = car_state.speed if car_state.speed > 0 else 0

        state = State(
            x = car_state.kinematics_estimated.position.x_val if
self.is_global_frame else 0,
            y = car_state.kinematics_estimated.position.y_val if
self.is_global_frame else 0,
            yaw = yaw if self.is_global_frame else 0,
            v = v,
            v_linear = car_state.kinematics_estimated.linear_velocity if
self.is_global_frame else np.array([0, 0, 0]),
            v_angular = car_state.kinematics_estimated.angular_velocity if
self.is_global_frame else np.array([0, 0, 0]),
            a_angular = car_state.kinematics_estimated.angular_acceleration if
self.is_global_frame else np.array([0, 0, 0]),
            a_linear = car_state.kinematics_estimated.linear_acceleration if
self.is_global_frame else np.array([0, 0, 0]),
            timestamp = car_state.timestamp if self.is_global_frame else 0
            )
        return {"car_state": state, "referee_state": referee_state}   # keep
as raw object for now
```

10.9. Apendix I – cones.py

```
from typing import Dict
import math
import logging
import numpy as np
from providers.sim.sim_util import FSDSClientSingleton, get_car_orientation,
pointgroup_to_cone
from sub_modules.fsds.utils import to_eularian_angles
from fsd_path_planning import ConeTypes
```

```
from .sim_util import pointgroup_to_cone


log = logging.getLogger("SimLogger")


class SimConeProvider:
    def __init__(self, is_global_frame: bool = True):
        self.client = FSDSClientSingleton.instance()
        self.is_global_frame = is_global_frame
        log.debug(f"Provider {self.__class__.__name__} initialized
successfully")

    def start(self):     pass
    def stop(self):      pass

    def read(self) -> Dict[str, np.ndarray]:
        min_cone_distance = 0 # Minimum distance threshold [m]
        max_cone_distance = 40 # Maximum distance threshold [m]

        # Retrieve LiDAR data from the simulator
        lidardata = self.client.getLidarData(lidar_name='Lidar')

        # Check if sufficient points are available
        if len(lidardata.point_cloud) < 3:
            return None, None, None

        # Convert LiDAR data into a numpy array of xyz points
        points = np.array(lidardata.point_cloud, dtype=np.float32).reshape(-1,
3)

        # Compute distances from the LiDAR origin to each point
        distances = np.linalg.norm(points[:, :2], axis=1)

        # Filter points based on the specified range limits
        valid_indices = np.logical_and(distances >= min_cone_distance,
distances <= max_cone_distance)
        points = points[valid_indices]

        # Obtain car's orientation and position
        car_position, car_direction, yaw = get_car_orientation(self.client)

        # Group points to identify cones
        current_group, cones = [], []
        for i in range(1, len(points)):
            distance_to_last_point = math.dist([points[i][0], points[i][1]],
[points[i-1][0], points[i-1][1]])

            if distance_to_last_point < 0.1:
```

```
                current_group.append({'x': points[i][0], 'y': points[i][1]})
            else:
                if current_group:
                    cone = pointgroup_to_cone(current_group)
                    if self.is_global_frame:
                        old_x, old_y = cone['x'] + 1.3, -cone['y']  # Adjust
for LiDAR position relative to the car center
                        # Rotate points to global reference frame
                        cone['x'] = np.cos(-yaw) * old_x - np.sin(-yaw) *
old_y + car_position[0]
                        cone['y'] = np.sin(-yaw) * old_x + np.cos(-yaw) *
old_y + car_position[1]
                        cones.append(np.array([cone['x'], cone['y']]))
                    else:
                        cones.append(np.array([cone['x'] + 1.3, -
cone['y']]))
                current_group = []

        cones_by_type = [np.zeros((0, 2)) for _ in range(5)]
        log.info(f"lidar detect: {np.array(cones).shape}")
        # remove all cones that left to the car (y axis)
        # cones = [cone for cone in cones if cone[1] >=0]
        cones_by_type[ConeTypes.UNKNOWN] = np.array(cones)
        cones_lidar = np.array(cones)
        if not self.is_global_frame:
            car_position = np.array([0,0])
            car_direction = np.array([np.cos(0),np.sin(0)])
        return {
            'cones':(cones_by_type, car_position, car_direction),
            'cones_lidar':cones_lidar
            }
```

10.10. Apendix J – map_data.py

```
from typing import Dict
import numpy as np
from providers.sim.sim_util import FSDSClientSingleton, get_car_orientation
from fsd_path_planning import ConeTypes
import logging

log = logging.getLogger("SimLogger")

class SimMapProvider:
    def __init__(self):
        self.client = FSDSClientSingleton.instance()
        log.debug(f"Provider {self.__class__.__name__} initialized
successfully")
```

```
    def start(self):  pass
    def stop(self):   pass

    def read(self) -> Dict[str, np.ndarray]:
        # Referee Colors:
        # 0 = Yellow (Left), 1 = Blue (Right), 2 = OrangeLarge, 3 =
OrangeSmall, 4 = Unknown
        referee_state = self.client.getRefereeState()
        cones_by_colors = {color: [] for color in range(5)}
        cones = referee_state.cones
        initial_position = referee_state.initial_position

        car_position, car_direction, _ = get_car_orientation(self.client)

        for cone in cones:
            color = cone['color']
            if color in cones_by_colors:
                x = cone['x'] / 100 - (initial_position.x / 100)
                y = cone['y'] / 100 - (initial_position.y / 100)

                # Rotate cone position to car frame
                cone_position = np.array([x, y])
                cones_by_colors[color].append(cone_position)

        cones_by_type = [np.zeros((0, 2)) for _ in range(5)]
        cones_by_type[ConeTypes.LEFT] = np.array(cones_by_colors[0])  # Yellow
cones (Left)
        cones_by_type[ConeTypes.RIGHT] = np.array(cones_by_colors[1])  # Blue
cones (Right)
        cones_by_type[ConeTypes.ORANGE_BIG] = np.array(cones_by_colors[2])  #
Large Orange cones
        cones_by_type[ConeTypes.ORANGE_SMALL] =
np.array(cones_by_colors[3])  # Small Orange cones

        return {'map':(cones_by_type, car_position, car_direction),
                'cones_map': cones_by_type
                }
```

10.11. Apendix K – referee_state.py

```
from typing import Dict
import numpy as np
from providers.sim.sim_util import FSDSClientSingleton
from sub_modules.fsds.client import FSDSClient   # your existing AirSim
wrapper
from sub_modules.fsds.utils import to_eularian_angles
from core.data.car_state import State
import logging
```

```
log = logging.getLogger("SimLogger")
class SimRefereeStateProvider:
    def __init__(self, is_global_frame: bool = True):
        self.client = FSDSClientSingleton.instance()
        self.is_global_frame = is_global_frame
        log.debug(f"Provider {self.__class__.__name__} initialized
successfully")


    def start(self):  pass
    def stop(self):   pass

    def read(self) -> Dict[str, np.ndarray]:
        referee_state = self.client.getRefereeState()
        return {"referee_state": referee_state}
```

10.12. Apendix L – sim_util

```
import time
import numpy as np
from fsd_path_planning import ConeTypes
from fsd_path_planning.utils.math_utils import unit_2d_vector_from_angle
from vehicle_config import Vehicle_config as conf
import logging
import sub_modules.fsds as fsds # local directory
from sub_modules.fsds.utils import to_eularian_angles
import math
# from visualization import Visualizer
import logging, time, threading
# Initialize logger
logger = logging.getLogger('SimLogger')


class FSDSClientSingleton:
    _instance = None
    _lock     = threading.Lock()

    @classmethod
    def instance(cls):
        """Returns the single FSDSClient, creating it once in a thread-safe
way."""
        if cls._instance is None:
            with cls._lock:           # double-checked locking
                if cls._instance is None:
                    logger.info("Initializing FSDS client…")
                    time.sleep(1)
                    client = fsds.FSDSClient()
```

```
                    client.confirmConnection()
                    try:
                        client.reset()
                    except Exception:
                        pass
                    client.enableApiControl(True)
                    cls._instance = client
        return cls._instance


# Initialize FSDS Client
def init_client():
    logger.info('Initializing FSDS client...')
    time.sleep(1)
    client = fsds.FSDSClient()
    client.confirmConnection()
    try:
        client.reset()
    except:
        pass
    client.enableApiControl(True)
    return client

def pointgroup_to_cone(group):
    average_x = 0
    average_y = 0
    for point in group:
        average_x += point['x']
        average_y += point['y']
    average_x = average_x / len(group)
    average_y = average_y / len(group)
    return {'x': average_x, 'y': average_y}

def get_car_orientation(client: fsds.FSDSClient):
        # Get car state and orientation
    car_state = client.getCarState()
    car_position = np.array([car_state.kinematics_estimated.position.x_val, -
car_state.kinematics_estimated.position.y_val])
    orientation = car_state.kinematics_estimated.orientation
    _, _, yaw = to_eularian_angles(orientation)
    car_direction = unit_2d_vector_from_angle(-yaw)
    return car_position, car_direction, yaw

# Load cones from referee
def load_cones_from_referee():
    logger.info("Loading cones from referee...")
    # Referee Colors:
    # 0 = Yellow (Left), 1 = Blue (Right), 2 = OrangeLarge, 3 = OrangeSmall, 4
= Unknown
```

```
    referee_state = client.getRefereeState()
    cones_by_colors = {color: [] for color in range(5)}
    cones = referee_state.cones
    initial_position = referee_state.initial_position

    car_position, car_direction, _ = get_car_orientation(client)

    for cone in cones:
        color = cone['color']
        if color in cones_by_colors:
            x = cone['x'] / 100 - (initial_position.x / 100)
            y = cone['y'] / 100 - (initial_position.y / 100)

            # Rotate cone position to car frame
            cone_position = np.array([x, y])
            cones_by_colors[color].append(cone_position)

    cones_by_type = [np.zeros((0, 2)) for _ in range(5)]
    cones_by_type[ConeTypes.LEFT] = np.array(cones_by_colors[0])  # Yellow
cones (Left)
    cones_by_type[ConeTypes.RIGHT] = np.array(cones_by_colors[1])  # Blue
cones (Right)
    cones_by_type[ConeTypes.ORANGE_BIG] = np.array(cones_by_colors[2])  #
Large Orange cones
    cones_by_type[ConeTypes.ORANGE_SMALL] = np.array(cones_by_colors[3])  #
Small Orange cones

    logger.info("Cones by type:")
    for i, cones in enumerate(cones_by_type):
        logger.info(f"Type {i}: {len(cones)} cones")

    logger.info(f"Car position: {car_position}")
    logger.info(f"Car direction: {car_direction}")

    return cones_by_type, car_position, car_direction


# Get car state from simulator
def sim_car_state(client):
    car_state = client.getCarState()
    orientation = car_state.kinematics_estimated.orientation
    _, _, yaw = fsds.utils.to_eularian_angles(orientation)
    x = car_state.kinematics_estimated.position.x_val
    y = car_state.kinematics_estimated.position.y_val
    yaw = (yaw + np.pi) % (2 * np.pi) - np.pi #normalize_yaw
    v_linear = car_state.kinematics_estimated.linear_velocity
    v_angular = car_state.kinematics_estimated.angular_velocity
    a_linear = car_state.kinematics_estimated.linear_acceleration
    a_angular = car_state.kinematics_estimated.angular_acceleration
```

```
    logger.info(f"car_state: {car_state}")
    v = car_state.speed if car_state.speed > 0 else 0
    return x, y, yaw, v, v_linear,v_angular,a_linear,a_angular

# Set car controls in simulator
def sim_car_controls(client, di, ai, brake_available=True):
    """
    Sends control commands to the vehicle.

    Args:
        client: AirSim client instance.
        di (float): Steering command [rad].
        ai (float): Acceleration command [m/s^2].
        #kaki
    """
    car_controls = fsds.CarControls()
    car_controls.steering = di / conf.MAX_STEER  # Normalize steering if
necessary

    throttle_cmd = ai / conf.MAX_ACCEL
    if ai >= 0:
        # Positive acceleration: map to throttle
        car_controls.throttle = np.clip(throttle_cmd, 0.0, 1.0)
        car_controls.brake = 0.0
    elif brake_available and ai < 0:
        # Negative acceleration: map to brake
        brake_cmd = -ai / conf.MAX_DECEL  # conf.MAX_DECEL should be negative
        # brake_cmd = ai   # conf.MAX_DECEL should be negative
        car_controls.throttle = 0.0
        car_controls.brake = np.clip(brake_cmd, 0.0, 1.0)
    else:
        car_controls.throttle = np.clip(throttle_cmd, 0.0, 1.0)
        car_controls.brake = 0.0


    # Log the control commands
    logger.debug(f"Steering command: {car_controls.steering}")
    logger.debug(f"Throttle command: {car_controls.throttle}")
    logger.debug(f"Brake command: {car_controls.brake}")

    # Send controls to AirSim
    client.setCarControls(car_controls)
```

10.13. Apendix M – controllers.py

```
# control.py

import numpy as np
```

```python
import math
import logging
import cvxpy as cp  # For MPC
from simple_pid import PID
from vehicle_config import Vehicle_config as conf
import control as ctrl
from fsd_path_planning import ConeTypes
# from visualization import Visualizer

logger = logging.getLogger('SimLogger')
# PID Controller for Acceleration
class AccelerationPIDController:
    def __init__(self, kp, ki, kd, setpoint):
        """
        PID Controller for acceleration (throttle control).

        Args:
            kp (float): Proportional gain.
            ki (float): Integral gain.
            kd (float): Derivative gain.
            setpoint (float): Desired speed [m/s].
        """
        self.pid = PID(kp, ki, kd, setpoint)
        self.maxspeed = setpoint

    def compute_acceleration(self, current_speed, curvature=0.0):
        """
        Compute acceleration (throttle input) based on current speed.

        Args:
            current_speed (float): Current speed of the vehicle [m/s].

        Returns:
            float: Throttle input [0,1].
        """
        desired_v = self.compute_breaking(curvature)
        self.maxspeed = desired_v
        self.pid.setpoint = self.maxspeed
        acceleration = self.pid(current_speed)
        acceleration = acceleration/ conf.TARGET_SPEED  # Normalize to [0, 1]
        acceleration = np.clip(acceleration, conf.MAX_DECEL, conf.MAX_ACCEL)
        return acceleration

    def compute_breaking_old(self, curve, target_ind):
        # Logistic decay
        b = 8 # Sensitivity parameter
        c = 1 # Power parameter
        v_logistic = self.maxspeed / (1 + b * np.abs(curve)**c)
```

```
        velocity = v_logistic[target_ind]
        self._update_desired_speed(velocity)
        return velocity

    def compute_breaking(self, curvature):
        """
        Compute the desired speed based on curvature.

        Args:
            curvature (float): Curvature at the target point.

        Returns:
            float: Desired speed [m/s].
        """
        # Define the relationship between curvature and desired speed
        a = conf.k_expo       # Sensitivity parameter for exponential decay

        max_speed = conf.TARGET_SPEED  # Maximum desired speed
        v_desired = max_speed * np.exp(-a * np.abs(curvature))
        return v_desired

    def _update_desired_speed(self, velocity):
        self.pid.setpoint = velocity


# Pure Pursuit Controller
class PurePursuitController:
    def __init__(self, look_ahead_distance=conf.LOOKAHEAD_DISTANCE):
        """
        Pure Pursuit Steering Controller.

        Args:
            look_ahead_distance (float): Look-ahead distance for target point
[m].
        """
        self.look_ahead_distance = look_ahead_distance

    def compute_steering(self, state, path, target_ind):
        """
        Compute steering angle using Pure Pursuit algorithm.

        Args:
            state (State): Current state of the vehicle.
            path (numpy.ndarray): Path coordinates [[index, x1, y1], [index,
x2, y2], ...].
            target_ind (int): Current target index on the path.

        Returns:
            float: Steering angle [rad].
```

```
            int: Updated target index.
        """
        cx = path[:, 1]
        cy = path[:, 2]
        ind = target_ind

        # Search for the target point
        Lf = self.look_ahead_distance
        while ind < len(cx):
            dx = cx[ind] - state.x
            dy = cy[ind] - state.y
            distance = np.hypot(dx, dy)
            if distance >= Lf:
                break
            ind += 1
        if ind >= len(cx):
            ind = len(cx) - 1

        # Calculate the steering angle
        tx = cx[ind]
        ty = cy[ind]
        alpha = math.atan2(ty - state.y, tx - state.x) - state.yaw
        alpha = normalize_angle(alpha)  # Normalize angle

        steering = math.atan2(2.0 * conf.WB * math.sin(alpha) / Lf, 1.0)
        steering = np.clip(steering, -conf.MAX_STEER, conf.MAX_STEER)
        return steering, ind

# Stanley Controller
# class StanleyController: #old
#     def __init__(self, k=conf.k_stanley, k_soft=conf.k_soft_stanley,
max_steer_rate=np.deg2rad(30), alpha=0.5, lookahead_distance=0.0):
#         """
#         Improved Stanley Steering Controller.

#         Args:
#             k (float): Proportional gain for the cross-track error.
#             k_soft (float): Softening constant to avoid division by zero.
#             max_steer_rate (float): Maximum rate of change of the steering
angle [rad/s].
#             alpha (float): Smoothing factor for steering command (0 < alpha
<= 1).
#             lookahead_distance (float): Optional look-ahead distance [m].
#         """
#         self.k = k
#         self.k_soft = k_soft
#         self.max_steer_rate = max_steer_rate
#         self.alpha = alpha  # For low-pass filtering
```

```
#         self.lookahead_distance = lookahead_distance
#         self.previous_steering = 0.0  # Initialize previous steering angle

#     def compute_steering(self, state, path, target_ind, dt):
#         """
#         Compute steering angle using the improved Stanley control algorithm.

#         Args:
#             state (State): Current state of the vehicle.
#             path (numpy.ndarray): Path coordinates [[index, x1, y1], [index,
x2, y2], ...].
#             target_ind (int): Current target index on the path.
#             dt (float): Time step [s].

#         Returns:
#             float: Steering angle [rad].
#             int: Updated target index.
#         """
#         cx = path[:, 1]
#         cy = path[:, 2]

#         # Find the nearest path point (with optional look-ahead)
#         dx = cx - state.x
#         dy = cy - state.y
#         d = np.hypot(dx, dy)
#         target_ind = np.argmin(d)

#         # Apply look-ahead if specified
#         if self.lookahead_distance > 0:
#             while target_ind < len(cx) - 1:
#                 distance = np.hypot(cx[target_ind] - state.x, cy[target_ind]
- state.y)
#                 if distance >= self.lookahead_distance:
#                     break
#                 target_ind += 1

#         nearest_x = cx[target_ind]
#         nearest_y = cy[target_ind]

#         # Path heading at the nearest point
#         if target_ind < len(cx) - 1:
#             path_dx = cx[target_ind + 1] - cx[target_ind]
#             path_dy = cy[target_ind + 1] - cy[target_ind]
#         else:
#             path_dx = cx[target_ind] - cx[target_ind - 1]
#             path_dy = cy[target_ind] - cy[target_ind - 1]

#         path_yaw = math.atan2(path_dy, path_dx)
```

```
#         # Heading error
#         theta_e = normalize_angle(path_yaw - state.yaw)

#         # Cross-track error using lateral error projection
#         dx = nearest_x - state.x
#         dy = nearest_y - state.y
#         e_ct = -dx * np.sin(path_yaw) + dy * np.cos(path_yaw)

#         # Adaptive gain scheduling based on speed
#         adaptive_k = self.k / (state.v + self.k_soft)

#         # Steering control law
#         steering = theta_e + math.atan2(adaptive_k * e_ct, 1.0)
#         steering = normalize_angle(steering)

#         # Limit steering rate
#         max_delta = self.max_steer_rate * dt
#         steering = np.clip(steering, self.previous_steering - max_delta,
self.previous_steering + max_delta)

#         # Low-pass filter steering angle
#         steering = self.alpha * steering + (1 - self.alpha) *
self.previous_steering

#         # Update previous steering angle
#         self.previous_steering = steering

#         # Clip to maximum steering angle
#         steering = np.clip(steering, -conf.MAX_STEER, conf.MAX_STEER)

#         return steering, target_ind
#     def __init__(self, k=1.0, k_soft=0.1):
#         """
#         Stanley Steering Controller.

#         Args:
#             k (float): Gain for the cross-track error.
#             k_soft (float): Softening constant to avoid division by zero.
#         """
#         self.k = k
#         self.k_soft = k_soft

#     def compute_steering1(self, state, path, target_ind):
#         """
#         Compute steering angle using Stanley control algorithm.

#         Args:
```

```
#             state (State): Current state of the vehicle.
#             path (numpy.ndarray): Path coordinates [[index, x1, y1], [index,
x2, y2], ...].
#             target_ind (int): Current target index on the path.

#         Returns:
#             float: Steering angle [rad].
#             int: Updated target index.
#         """
#         cx = path[:, 1]
#         cy = path[:, 2]

#         # Find the nearest path point
#         dx = cx - state.x
#         dy = cy - state.y
#         d = np.hypot(dx, dy)
#         target_ind = np.argmin(d)
#         nearest_x = cx[target_ind]
#         nearest_y = cy[target_ind]

#         # Path heading at the nearest point
#         if target_ind < len(cx) - 1:
#             path_dx = cx[target_ind + 1] - cx[target_ind]
#             path_dy = cy[target_ind + 1] - cy[target_ind]
#         else:
#             path_dx = cx[target_ind] - cx[target_ind - 1]
#             path_dy = cy[target_ind] - cy[target_ind - 1]

#         path_yaw = math.atan2(path_dy, path_dx)

#         # Heading error
#         theta_e = normalize_angle(path_yaw - state.yaw)

#         # Cross-track error
#         dx = nearest_x - state.x
#         dy = nearest_y - state.y
#         e_ct = -dx * np.sin(path_yaw) + dy * np.cos(path_yaw)

#         # Steering control law
#         steering = theta_e + math.atan2(self.k * e_ct, state.v +
self.k_soft)
#         steering = normalize_angle(steering)
#         steering = np.clip(steering, -conf.MAX_STEER, conf.MAX_STEER)
#         # Visualizer.cross_track_error(e_ct,path,cx,cy,theta_e)


#         return steering, target_ind
```

```
# class StanleyController:
#     def __init__(self, k=conf.k_stanley, k_soft=conf.k_soft_stanley,
#                  max_steer_rate=np.deg2rad(30),
#                  alpha=0.9):
#         self.k = k
#         self.k_soft = k_soft
#         self.max_steer_rate = max_steer_rate
#         self.alpha = alpha
#         self.previous_steering = 0.0
#         self.last_target_ind = 0
#         self.l_f = conf.l_f

#     def compute_steering(self, state, path, current_target_ind, dt):
#         cx = path[:, 1]
#         cy = path[:, 2]

#         # Project to front axle (ENU: x=east, y=north)
#         front_axle_x = state.x + self.l_f * math.cos(state.yaw)
#         front_axle_y = state.y + self.l_f * math.sin(state.yaw)

#         # 1. Find closest path point to the front axle
#         dx = cx - front_axle_x
#         dy = cy - front_axle_y
#         distances_sq = dx**2 + dy**2
#         closest_ind = np.argmin(distances_sq)

#         # 2. Ensure forward progress - use closest point + small lookahead
#         target_ind = max(current_target_ind, self.last_target_ind,
closest_ind)
#         lookahead_ind = min(target_ind + 2, len(cx) - 1)  # Small lookahead
for heading

#         # 3. Calculate path heading at lookahead point
#         if lookahead_ind < len(cx) - 1:
#             path_dx = cx[lookahead_ind + 1] - cx[lookahead_ind]
#             path_dy = cy[lookahead_ind + 1] - cy[lookahead_ind]
#         else:
#             path_dx = cx[lookahead_ind] - cx[lookahead_ind - 1]
#             path_dy = cy[lookahead_ind] - cy[lookahead_ind - 1]

#         path_yaw = math.atan2(path_dy, path_dx)

#         # 4. Calculate heading error
#         heading_error = normalize_angle(path_yaw - state.yaw)

#         # 5. Calculate cross-track error using CLOSEST point
#         # Use the SAME path heading for consistency
#         dx_cte = cx[closest_ind] - front_axle_x
```

```
#         dy_cte = cy[closest_ind] - front_axle_y

#         # CORRECTED CTE: Positive when vehicle is to the RIGHT of path
#         # Standard formula: rotate the error vector by -90 degrees relative
to path
#         cte = dx_cte * math.sin(path_yaw) - dy_cte * math.cos(path_yaw)

#         # 6. CORRECTED Stanley control law
#         speed = abs(state.v) + self.k_soft  # Add k_soft to speed (not max)
#         delta_cte = math.atan2(self.k * cte, speed)

#         steering_cmd = heading_error + delta_cte
#         steering_cmd = normalize_angle(steering_cmd)

#         # 7. Apply rate limiting
#         if dt > 0:
#             max_delta_steer = self.max_steer_rate * dt
#             steering_cmd = np.clip(steering_cmd,
#                                    self.previous_steering - max_delta_steer,
#                                    self.previous_steering + max_delta_steer)

#         # 8. Apply smoothing
#         steering_cmd = self.alpha * steering_cmd + (1.0 - self.alpha) *
self.previous_steering

#         # 9. Apply steering limits
#         steering_cmd = np.clip(steering_cmd, -conf.MAX_STEER,
conf.MAX_STEER)

#         self.previous_steering = steering_cmd
#         self.last_target_ind = target_ind

#         return steering_cmd, target_ind


# # class StanleyController:
#     """
#     Stanley lateral controller (front-axle formulation).

#     Args
#     ----
#     k : float
#         Gain multiplying the cross-track error term.
#     k_soft : float
#         Small positive number added to |v| to avoid division by zero at low
speed.
#     wheelbase : float
```

```
#         Distance between front and rear axle (used only to project the
control
#         point to the front axle).
#     max_steer : float
#         Absolute steering limit [rad].
#     max_steer_rate : float
#         Maximum change rate of steering per second [rad s⁻¹] (optional).
#     alpha : float
#         First-order low-pass filter coefficient for extra smoothness
#         (0 ⇒ no smoothing, 1 ⇒ full smoothing).
#     """

#     def __init__(self,
#                  k            = conf.k_stanley,
#                  k_soft       = conf.k_soft_stanley,
#                  wheelbase    = conf.WB,
#                  max_steer    = conf.MAX_STEER,
#                  max_steer_rate = np.deg2rad(300),   # 300 ° s⁻¹
#                  alpha        = 0.85):

#         self.k               = k
#         self.k_soft          = k_soft
#         self.WB              = wheelbase
#         self.max_steer       = max_steer
#         self.max_steer_rate  = max_steer_rate
#         self.alpha           = alpha

#         self.prev_delta      = 0.0         # last issued steering command
#         self.prev_target_ind = 0           # for forward progress


#     # ------------------------------------------------------------------ #
#     #                               API                                  #
#     # ------------------------------------------------------------------ #
#     def compute_steering(self, state, path, prev_target_ind: int, dt:
float):
#         """
#         Parameters
#         ----------
#         state : object with attributes x, y, yaw [rad], v [m/s]
#         path  : np.ndarray, columns = [index, x, y]  (same as in Pure
Pursuit)
#         prev_target_ind : int
#             Greedy index you used last cycle (helps skip already-passed
points)
#         dt : float
#             Controller cycle time [s] – only needed for rate limiting

#         Returns
```

```
#         -------
#         delta : float
#             Steering angle command [rad]
#         target_ind : int
#             Index on the reference path associated with the command
#         """

#         cx, cy = path[:, 1], path[:, 2]

#         # ---------------------------------------------------------------
#
#         # 1. Front-axle position (gives crisper path tracking than CG point)
#
#         # ---------------------------------------------------------------
#
#         fx = state.x + self.WB * math.cos(state.yaw)*0.5
#         fy = state.y + self.WB * math.sin(state.yaw)*0.5
#         # Note: we use the front axle position to compute the steering angle

#         # ---------------------------------------------------------------
#
#         # 2. Find closest path point to the front
axle                      #
#         # ---------------------------------------------------------------
#
#         dx = cx - fx
#         dy = cy - fy
#         d2 = dx**2 + dy**2
#         closest_ind = int(np.argmin(d2))

#         # Keep moving forward along the path (never go backwards in index)
#         target_ind = max(closest_ind, prev_target_ind)
#         target_ind = min(target_ind, len(cx) - 2)   # leave one point for
heading

#         # ---------------------------------------------------------------
#
#         # 3. Path heading (ψ_ref) at the target
index                       #
#         # ---------------------------------------------------------------
#
#         path_dx = cx[target_ind + 1] - cx[target_ind]
#         path_dy = cy[target_ind + 1] - cy[target_ind]
#         psi_ref = math.atan2(path_dy, path_dx)

#         # ---------------------------------------------------------------
#
```

```
#         # 4. Heading error (ψ_ref –
ψ_car)                                  #
#         # ------------------------------------------------------------
#
#         heading_error = normalize_angle_stanley(psi_ref - state.yaw)

#         # ------------------------------------------------------------
#
#         # 5. Cross-track error sign: positive when vehicle is to the
RIGHT   #
#         #    of the path heading direction (right-hand
rule).              #
#         # ------------------------------------------------------------
#
#         # Rotate error vector into the path frame and take its lateral part:
#         cte =  dx[closest_ind] * math.sin(psi_ref) - dy[closest_ind] *
math.cos(psi_ref)
#         cte = -cte  # Correct sign: positive when vehicle is to the RIGHT of
path
#         # ------------------------------------------------------------
#
#         # 6. Stanley control
law                                             #
#         # ------------------------------------------------------------
#
#         v_soft = abs(state.v) + self.k_soft
#         delta = heading_error + math.atan2(self.k * cte, v_soft)
#         delta = normalize_angle_stanley(delta)

#         # ------------------------------------------------------------
#
#         # 7. Optional rate
limiting                                          #
#         # ------------------------------------------------------------
#
#         if dt > 0.0:
#             max_change = self.max_steer_rate * dt
#             delta = np.clip(delta,
#                             self.prev_delta - max_change,
#                             self.prev_delta + max_change)

#         # ------------------------------------------------------------
#
#         # 8. Extra smoothing (first-order
IIR)                               #
#         # ------------------------------------------------------------
#
#         delta = self.alpha * delta + (1.0 - self.alpha) * self.prev_delta
```

```
#         # ---------------------------------------------------------------
#
#         # 9. Clamp to steering
limits                                      #
#         # ---------------------------------------------------------------
#
#         delta = np.clip(delta, -self.max_steer, self.max_steer)

#         # House-keeping for the next timestep
#         self.prev_delta      = float(delta)
#         self.prev_target_ind = int(target_ind)

#         return float(delta), int(target_ind)


# class StanleyController:  #working prototype
#     def __init__(self, k=conf.k_stanley, k_soft=conf.k_soft_stanley,
#                  max_steer_rate=np.deg2rad(30), alpha=0.7,
lookahead_distance=0.0):
#         """
#         Stanley Steering Controller.

#         Args:
#             k (float): Control gain for cross-track error.
#             k_soft (float): Softening constant to avoid division by zero.
#             max_steer_rate (float): Maximum steering rate [rad/s].
#             alpha (float): Smoothing factor for steering command.
#             lookahead_distance (float): Additional lookahead distance [m].
#         """
#         self.k = k
#         self.k_soft = k_soft
#         self.max_steer_rate = max_steer_rate
#         self.alpha = alpha
#         self.lookahead_distance = lookahead_distance
#         self.prev_steering = 0.0

#     def compute_steering(self, state, path, target_ind,dt):
#         """
#         Compute steering angle using Stanley algorithm.

#         Args:
#             state (State): Current state of the vehicle.
#             path (numpy.ndarray): Path coordinates [[index, x1, y1], [index,
x2, y2], ...].
#             target_ind (int): Current target index on the path.

#         Returns:
#             float: Steering angle [rad].
```

```
#             int: Updated target index.
#         """
#         cx = path[:, 1]
#         cy = path[:, 2]

#         # Find closest point on path
#         min_dist = float('inf')
#         closest_ind = target_ind

#         # Search around current target index for closest point
#         search_range = min(50, len(cx) - 1)  # Limit search range for
efficiency
#         start_ind = max(0, target_ind - 10)
#         end_ind = min(len(cx), target_ind + search_range)

#         for i in range(start_ind, end_ind):
#             dx = cx[i] - state.x
#             dy = cy[i] - state.y
#             dist = np.hypot(dx, dy)
#             if dist < min_dist:
#                 min_dist = dist
#                 closest_ind = i

#         # Update target index to closest point or ahead
#         target_ind = max(closest_ind, target_ind)
#         if target_ind > 3:
#             target_ind = 2
#         print(f"Closest index: {closest_ind}, Target index: {target_ind}")
#         # Ensure we don't go beyond path end
#         # if target_ind >= len(cx) - 1:
#         #     target_ind = len(cx) - 1

#         # Calculate path heading at closest point
#         if target_ind < len(cx) - 1:
#             path_heading = math.atan2(cy[target_ind + 1] - cy[target_ind],
#                                       cx[target_ind + 1] - cx[target_ind])
#         else:
#             # Use previous segment if at end
#             path_heading = math.atan2(cy[target_ind] - cy[target_ind - 1],
#                                       cx[target_ind] - cx[target_ind - 1])

#         # Calculate heading error
#         heading_error = normalize_angle_stanley(path_heading - state.yaw)

#         # Calculate cross-track error
#         # Vector from vehicle to closest path point
#         dx = cx[target_ind] - state.x+0.9
#         dy = cy[target_ind] - state.y+0.9
```

```
#         # Cross-track error is the perpendicular distance to path
#         # Project vehicle position error onto normal to path
#         front_axle_vec = [math.cos(state.yaw + math.pi/2),
#                           math.sin(state.yaw + math.pi/2)]
#         cross_track_error = dx * front_axle_vec[0] + dy * front_axle_vec[1]

#         # Stanley control law
#         # δ = ψ + arctan(k * e / (k_soft + v))
#         cross_track_term = math.atan2(self.k * cross_track_error,
#                                       self.k_soft + state.v)

#         steering = heading_error + cross_track_term

#         # Apply steering limits
#         steering = np.clip(steering, -conf.MAX_STEER, conf.MAX_STEER)

#         # Apply steering rate limit (simple first-order filter)
#         if hasattr(self, 'prev_steering'):
#             max_change = self.max_steer_rate * 0.1  # Assuming 10Hz control
#             steering_change = steering - self.prev_steering
#             steering_change = np.clip(steering_change, -max_change,
max_change)
#             steering = self.prev_steering + steering_change

#         # Smooth steering command
#         steering = self.alpha * steering + (1 - self.alpha) *
self.prev_steering
#         self.prev_steering = steering

#         return steering, target_ind


class StanleyController:
    def __init__(self, k=conf.k_stanley, k_soft=conf.k_soft_stanley):
        """
        Stanley Steering Controller - Simple Implementation.

        Args:
            k (float): Cross-track error gain.
            k_soft (float): Softening constant to avoid division by zero.
        """
        self.k = k
        self.k_soft = k_soft

    def compute_steering(self, state, path, target_ind, dt):
        """
        Compute steering angle using Stanley algorithm.
```

```
        Args:
            state (State): Current state of the vehicle.
            path (numpy.ndarray): Path coordinates [[index, x1, y1], [index,
x2, y2], ...].
            target_ind (int): Current target index on the path.
            dt (float): Time step [s] (not used in simple version).

        Returns:
            float: Steering angle [rad].
            int: Updated target index.
        """
        cx = path[:, 1]
        cy = path[:, 2]

        # 1. Find closest point (same logic as Pure Pursuit)
        min_dist = float('inf')
        closest_ind = target_ind

        # Search forward from current target
        for i in range(target_ind, min(target_ind + 50, len(cx))):
            dx = cx[i] - state.x
            dy = cy[i] - state.y
            dist = np.hypot(dx, dy)
            if dist < min_dist:
                min_dist = dist
                closest_ind = i

        # Ensure we don't go backwards
        closest_ind = max(closest_ind, target_ind)

        # 2. Calculate path heading at closest point
        if closest_ind < len(cx) - 1:
            path_dx = cx[closest_ind + 1] - cx[closest_ind]
            path_dy = cy[closest_ind + 1] - cy[closest_ind]
        else:
            path_dx = cx[closest_ind] - cx[closest_ind - 1]
            path_dy = cy[closest_ind] - cy[closest_ind - 1]

        path_yaw = math.atan2(path_dy, path_dx)

        # 3. Calculate heading error (same as Pure Pursuit's alpha concept)
        heading_error = normalize_angle(path_yaw - state.yaw)

        # 4. Calculate cross-track error (simple perpendicular distance)
        dx = cx[closest_ind] - state.x
        dy = cy[closest_ind] - state.y

        # Cross-track error: positive when vehicle is RIGHT of path
```

```
        cross_track_error = -dx * math.sin(path_yaw) + dy * math.cos(path_yaw)

        # 5. Stanley control law (similar structure to Pure Pursuit)
        # Pure Pursuit: steering = atan2(2*WB*sin(alpha)/Lf, 1)
        # Stanley: steering = heading_error + atan2(k*cte, v+k_soft)
        speed = max(abs(state.v), self.k_soft)  # Prevent division by zero
        cross_track_term = math.atan2(self.k * cross_track_error, speed)

        steering = heading_error + cross_track_term

        # 6. Apply limits (same as Pure Pursuit)
        steering = normalize_angle(steering)
        steering = np.clip(steering, -conf.MAX_STEER, conf.MAX_STEER)

        return steering, closest_ind


class MPCController:
    def __init__(self, N=10, dt=0.1):
        """
        Model Predictive Controller using a linearized vehicle model with
fixed speed.

        Args:
            N (int): Prediction horizon.
            dt (float): Time step size [s].
        """
        self.N = N  # Prediction horizon
        self.dt = dt  # Time step
        # Define weights for the cost function
        self.Q = np.diag([1.0, 1.0, 0.5, 0.1])  # State weighting matrix
        self.R = np.diag([0.1, 0.1])  # Control weighting matrix

    def compute_control(self, state, path):
        """
        Compute the optimal control inputs using MPC with a fixed speed.

        Args:
            state (State): Current state of the vehicle.
            path (numpy.ndarray): Path coordinates [[index, x1, y1], [index,
x2, y2], ...].

        Returns:
            float: Acceleration [m/s^2].
            float: Steering angle [rad].
        """
        # Extract reference trajectory
        ref_x = path[:, 1]
```

```
        ref_y = path[:, 2]

        # Define optimization variables
        x = cp.Variable((self.N + 1, 4))  # States: [x, y, yaw, v]
        u = cp.Variable((self.N, 2))      # Controls: [acceleration, steering]

        # Define constraints and cost
        constraints = []
        cost = 0

        # Initial condition
        constraints += [x[0, :] == [state.x, state.y, state.yaw, state.v]]

        # Precompute cos and sin of the current yaw angle
        cos_theta = np.cos(state.yaw)
        sin_theta = np.sin(state.yaw)

        # Fix the speed to current value
        v_current = state.v

        for k in range(self.N):
            # System dynamics with fixed speed
            constraints += [
                x[k + 1, 0] == x[k, 0] + v_current * cos_theta * self.dt,
                x[k + 1, 1] == x[k, 1] + v_current * sin_theta * self.dt,
                x[k + 1, 2] == x[k, 2] + v_current / conf.WB * u[k, 1] *
self.dt,
                x[k + 1, 3] == x[k, 3] + u[k, 0] * self.dt
            ]

            # Control constraints
            constraints += [
                cp.abs(u[k, 0]) <= conf.MAX_ACCEL,
                cp.abs(u[k, 1]) <= conf.MAX_STEER
            ]

            # State constraints (optional)
            constraints += [x[k + 1, 3] >= 0]  # Speed cannot be negative

            # Reference tracking
            if k < len(ref_x):
                ref_state = np.array([ref_x[k], ref_y[k], 0.0,
conf.TARGET_SPEED])
            else:
                ref_state = np.array([ref_x[-1], ref_y[-1], 0.0,
conf.TARGET_SPEED])

            # Cost function
```

```
            cost += cp.quad_form(x[k, :] - ref_state, self.Q) +
cp.quad_form(u[k, :], self.R)

        # Terminal cost
        cost += cp.quad_form(x[self.N, :] - ref_state, self.Q)

        # Define and solve the optimization problem
        prob = cp.Problem(cp.Minimize(cost), constraints)
        prob.solve(solver=cp.OSQP, warm_start=True)

        if prob.status in [cp.OPTIMAL, cp.OPTIMAL_INACCURATE]:
            # Extract control inputs
            acceleration = u.value[0, 0]
            steering = u.value[0, 1]
        else:
            # If optimization fails, use default values
            acceleration = 0.0
            steering = 0.0
            print("MPC optimization failed. Using zero acceleration and
steering.")

        # Clip control inputs to limits
        acceleration = np.clip(acceleration, conf.MAX_DECEL, conf.MAX_ACCEL)
        steering = np.clip(steering, -conf.MAX_STEER, conf.MAX_STEER)

        return acceleration, -steering


# Factory function to get steering controller by name
def get_steering_controller(name):
    """
    Factory function to get the steering controller instance based on name.

    Args:
        name (str): Name of the steering controller ('pure_pursuit',
'stanley', 'mpc').

    Returns:
        An instance of the requested steering controller.
    """
    if name == 'pure_pursuit':
        return
PurePursuitController(look_ahead_distance=conf.LOOKAHEAD_DISTANCE)
    elif name == 'stanley':
        return StanleyController(k=conf.k_stanley,
k_soft=conf.k_soft_stanley,)
    elif name == 'mpc':
        return MPCController(N=10, dt=0.05)
```

```
    else:
        raise ValueError(f"Unknown steering controller name: {name}")

# Utility functions
def normalize_angle(angle):
    """
    Normalize angle to [-pi, pi].

    Args:
        angle (float): Angle [rad].

    Returns:
        float: Normalized angle [rad].
    """
    angle = (angle + np.pi) % (2 * np.pi) - np.pi
    return angle

def normalize_angle_stanley(angle):
    """Wrap any angle to [-π, π)."""
    while angle >  math.pi: angle -= 2.0*math.pi
    while angle <= -math.pi: angle += 2.0*math.pi
    return angle
```

10.14. Apendix N – Visualization.py

```
from __future__ import annotations

import itertools, math, logging, os
from pathlib import Path
from typing import List, Dict, Any

import matplotlib
import matplotlib.pyplot as plt
from matplotlib.figure import Figure
import numpy as np

from core.data.plot_data import PlotData
from fsd_path_planning import ConeTypes
from vehicle_config import Vehicle_config as conf
import cv2
from io import BytesIO

log = logging.getLogger("PlotManager")

_FIG_COUNTER = itertools.count(1)        # fig_001.png, fig_002.png, …
_FIGS: List[Figure] = []
_LIVE = False #doesnt do shit as of right now.
```

```
fig = plt.figure(figsize=(8, 6), dpi=100)
def _auto_fig() -> Figure:
    """Create a figure, keep it for later saving, and return it."""
    fig = plt.figure()
    _FIGS.append(fig)
    return fig


def _maybe_show():
    if _LIVE:
        plt.pause(0.001)

class PlotManager:
    def __init__(self, live: bool):
        global _LIVE
        _LIVE = live
        # if live:
        #     plt.ion()
        # else:
        #     matplotlib.use("Agg")  # headless / non-interactive
        self.frames = []  # list of frames
        self.data = PlotData()  # single data object


        # # histories
        # self.cte_history: List[float] = []
        # self.theta_e_history: List[float] = []


        # ---------- life-cycle hooks ----------------------------------- #
    def update(self, _data: Dict[str, Any]):
        self.data = _data.get("plot_data", self.data)
        # if _LIVE:
        self.draw_frame(
            self.data.cx,
            self.data.cy,
            self.data.states,
            self.data.cones_map,
            self.data.target_ind,
            self.data.state,
            self.data.steering,
            self.data.v_log,
            self.data.cones_lidar,
            self.data.intermediate,
        )
    # ---------------------------------------------------------------- #
    @staticmethod
    def save_all(output_dir: str | Path):
        Path(output_dir).mkdir(parents=True, exist_ok=True)
```

```
        for idx, fig in enumerate(_FIGS, 1):
            fname = Path(output_dir) / f"fig_{idx:03d}.png"
            fig.savefig(fname, dpi=150)
            log.info("saved %s", fname)
        log.info("exported %d figures to %s", len(_FIGS), output_dir)


    @staticmethod
    def plot_car(x, y, yaw, steer=0.0, truckcolor="-k"):
        """
        Plot a car at a specified position, orientation, and steering angle.
        """
        LENGTH = 4.5  # [m]
        WIDTH = 2.0  # [m]
        BACKTOWHEEL = 1.0  # [m]
        WHEEL_LEN = 0.3  # [m]
        WHEEL_WIDTH = 0.2  # [m]
        TREAD = 0.7  # [m]
        WB = 2.5  # [m]

        outline = np.array([[-BACKTOWHEEL, (LENGTH - BACKTOWHEEL), (LENGTH -
BACKTOWHEEL), -BACKTOWHEEL, -BACKTOWHEEL],
                            [WIDTH / 2, WIDTH / 2, - WIDTH / 2, -WIDTH / 2,
WIDTH / 2]])

        fr_wheel = np.array([[WHEEL_LEN, -WHEEL_LEN, -WHEEL_LEN, WHEEL_LEN,
WHEEL_LEN],
                            [-WHEEL_WIDTH - TREAD, -WHEEL_WIDTH - TREAD,
WHEEL_WIDTH - TREAD, WHEEL_WIDTH - TREAD,
                            -WHEEL_WIDTH - TREAD]])

        rr_wheel = fr_wheel.copy()
        fl_wheel = fr_wheel.copy()
        rl_wheel = fr_wheel.copy()
        fl_wheel[1, :] *= -1
        rl_wheel[1, :] *= -1

        Rot1 = np.array([[math.cos(yaw), math.sin(yaw)], [-math.sin(yaw),
math.cos(yaw)]])
        Rot2 = np.array([[math.cos(steer), math.sin(steer)], [-
math.sin(steer), math.cos(steer)]])

        fr_wheel = (fr_wheel.T @ Rot2).T
        fl_wheel = (fl_wheel.T @ Rot2).T
        fr_wheel[0, :] += WB
        fl_wheel[0, :] += WB

        fr_wheel = (fr_wheel.T @ Rot1).T
        fl_wheel = (fl_wheel.T @ Rot1).T
```

```
        outline = (outline.T @ Rot1).T
        rr_wheel = (rr_wheel.T @ Rot1).T
        rl_wheel = (rl_wheel.T @ Rot1).T

        outline[0, :] += x
        outline[1, :] += y
        fr_wheel[0, :] += x
        fr_wheel[1, :] += y
        rr_wheel[0, :] += x
        rr_wheel[1, :] += y
        fl_wheel[0, :] += x
        fl_wheel[1, :] += y
        rl_wheel[0, :] += x
        rl_wheel[1, :] += y

        plt.plot(outline[0, :], outline[1, :], truckcolor)
        plt.plot(fr_wheel[0, :], fr_wheel[1, :], truckcolor)
        plt.plot(rr_wheel[0, :], rr_wheel[1, :], truckcolor)
        plt.plot(fl_wheel[0, :], fl_wheel[1, :], truckcolor)
        plt.plot(rl_wheel[0, :], rl_wheel[1, :], truckcolor)

    @staticmethod
    def plot_map(cones_by_type):
        """
        Plot cones based on their type (left or right).
        """

        cones_left = cones_by_type[ConeTypes.LEFT]
        cones_right = cones_by_type[ConeTypes.RIGHT]
        orange_big = cones_by_type[ConeTypes.ORANGE_BIG]
        orange_small = cones_by_type[ConeTypes.ORANGE_SMALL]
        cones_unknown = cones_by_type[ConeTypes.UNKNOWN]

        if len(cones_left) > 0:
            plt.plot(cones_left[:, 0], cones_left[:, 1], "ob", label="Left
Cones",markersize=2)
        if len(cones_right) > 0:
            plt.plot(cones_right[:, 0], cones_right[:, 1], "oy", label="Right
Cones",markersize=2)
        if len(cones_unknown) > 0:
            plt.plot(cones_unknown[:, 0], cones_unknown[:, 1], "og",
label="Unkonwn Cones",markersize=2)
        # if len(orange_big) > 0:
        #     plt.plot(orange_big[:, 0], orange_big[:, 1], color="orange",
marker="o", label="orange big",markersize=2)
        # if len(orange_small) > 0:
        #     plt.plot(orange_small[:, 0], orange_small[:, 1], color="orange",
marker="o", label="orange",markersize=2)
```

```
        if len(orange_big) > 0:
            plt.plot(orange_big[:, 0], orange_big[:, 1], "o", color="orange",
label="Orange Big", markersize=2)
        if len(orange_small) > 0:
            plt.plot(orange_small[:, 0], orange_small[:, 1], "o",
color="orange", label="Orange Small", markersize=2)

    # @staticmethod
    # def plot_map(cones_by_type):
    #     """
    #     Plot cones based on their type (left or right).
    #     """

    #     cones_left = cones_by_type[ConeTypes.LEFT]
    #     cones_right = cones_by_type[ConeTypes.RIGHT]
    #     orange_big = cones_by_type[ConeTypes.ORANGE_BIG]
    #     orange_small = cones_by_type[ConeTypes.ORANGE_SMALL]
    #     cones_unknown = cones_by_type[ConeTypes.UNKNOWN]

    #     if len(cones_left) > 0:
    #         plt.scatter(cones_left[:, 0], cones_left[:, 1], c="blue",
label="Left Cones", s=4)
    #     if len(cones_right) > 0:
    #         plt.scatter(cones_right[:, 0], cones_right[:, 1], c="yellow",
label="Right Cones", s=4)
    #     if len(cones_unknown) > 0:
    #         plt.scatter(cones_unknown[:, 0], cones_unknown[:, 1], c="green",
label="Unknown Cones", s=4)
    #     if len(orange_big) > 0:
    #         plt.scatter(orange_big[:, 0], orange_big[:, 1], c="orange",
label="Orange Big", s=4)
    #     if len(orange_small) > 0:
    #         plt.scatter(orange_small[:, 0], orange_small[:, 1], c="orange",
label="Orange Small", s=4)

    @staticmethod
    def plot_cones(cones_by_type, cones_lidar=[],key=None):
        """
        Plot cones based on their type (left or right).
        """
        PlotManager.plot_map(cones_by_type)
        if len(cones_lidar) > 0 and isinstance(cones_lidar, np.ndarray):
            plt.plot(cones_lidar[:, 0], cones_lidar[:, 1], "og", label="Lidar
Cones",markersize=2,)


    def draw_frame(
        self, cx, cy, states, cones_by_type, target_ind,
        state, di, v_log, cones_lidar, intermediate_results={}, fig=None
```

```
    ):
        """Draw one animation frame, quietly skipping bad inputs."""
        plt.cla()

        # ———————— Planned path ————————
        if cx is not None and cy is not None and len(cx) and len(cy):
            plt.plot(cx, -cy, "r--", label="Planned Path", linewidth=2.5)

        # ———————— Vehicle path ————————
        if states is not None and getattr(states, "x", None) is not None and
len(states.x):
            plt.plot(states.x, states.y, "-c", label="Vehicle Path")

        # ———————— Cones from lidar / labels ————————
        if cones_by_type is not None and cones_lidar is not None:
            PlotManager.plot_cones(cones_by_type, cones_lidar)

        # ———————— Target point ————————
        if (cx is not None and cy is not None and len(cx)
                and target_ind is not None and 0 <= target_ind < len(cx)):
            plt.plot(cx[target_ind], -cy[target_ind], "xg",
                     label="Target", markersize=10, linewidth=1)

        # ———————— Vehicle body ————————
        if state is not None:
            PlotManager.plot_car(state.x, -state.y, -state.yaw, steer=di)
        else:
            PlotManager.plot_car(0, 0, 0, 0)

        # ———————— Intermediate results ————————
        ir = intermediate_results or {}
        left  = ir.get("left_cones_with_virtual")
        right = ir.get("right_cones_with_virtual")
        all_c = ir.get("all_cones")

        if left is not None and len(left) and np.asarray(left).shape[1] == 2:
            plt.plot(*np.asarray(left).T, "o-", c="blue")

        if right is not None and len(right) and np.asarray(right).shape[1] ==
2:
            plt.plot(*np.asarray(right).T, "o-", c="yellow")

        if all_c is not None and len(all_c) and np.asarray(all_c).shape[1] ==
2:
            plt.plot(*np.asarray(all_c).T, "o", c="k")

        # Draw matching lines only if both lists exist and indices are valid
        l2r = ir.get("left_to_right_match")
```

```
        r2l = ir.get("right_to_left_match")

        if (left is not None and right is not None
                and l2r is not None and len(l2r)):
            for lpt, r_idx in zip(left, l2r):
                if 0 <= r_idx < len(right):
                    plt.plot([lpt[0], right[r_idx][0]],
                             [lpt[1], right[r_idx][1]],
                             "-", c="#7CB9E8")

        if (right is not None and left is not None
                and r2l is not None and len(r2l)):
            for rpt, l_idx in zip(right, r2l):
                if 0 <= l_idx < len(left):
                    plt.plot([rpt[0], left[l_idx][0]],
                             [rpt[1], left[l_idx][1]],
                             "-", c="gold", alpha=0.5)

        # ———————— Cosmetics ————————
        plt.axis("equal")
        plt.grid(True)

        if state is not None and hasattr(state, "v"):
            plt.title(f"Speed [km/h]: {state.v * 3.6:.2f}")
        # if v_log is not None:
            # plt.suptitle(f"Target Speed [km/h]: {v_log * 3.6:.2f}")

        buf = BytesIO()
        plt.savefig(buf, format='png', bbox_inches='tight')
        buf.seek(0)
        img_array = np.asarray(bytearray(buf.read()), dtype=np.uint8)
        buf.close()

        # Decode PNG buffer into OpenCV BGR image
        img = cv2.imdecode(img_array, cv2.IMREAD_COLOR)
        self.frames.append(img)
        _maybe_show()

    @staticmethod
    def show(cx, cy, states):
        """
        Show the final animation plot.
        """
        fig = _auto_fig()
        plt.figure(fig.number)
        plt.title("Path Tracking")
        plt.plot(cx, -cy, "r--", label="Planned Path")
        plt.plot(states.x, states.y, "-b", label="Vehicle Path")
```

```
        plt.legend()
        plt.xlabel("X [m]")
        plt.ylabel("Y [m]")
        plt.axis("equal")
        plt.grid(True)
        _maybe_show()

    # Class variables to store the history of CTE and heading error
    cte_history = []
    theta_e_history = []

    @staticmethod
    def cross_track_error(e_ct, path, cx, cy, theta_e):
        """
        Store the cross track error (and optional heading error).

        Args:
            e_ct (float): The cross track error at the current timestep.
            path (numpy.ndarray): Path coordinates [[index, x, y], ...].
            cx (numpy.ndarray): X-coordinates of the path points.
            cy (numpy.ndarray): Y-coordinates of the path points.
            theta_e (float): Heading error at the current timestep.
        """
        PlotManager.cte_history.append(e_ct)
        PlotManager.theta_e_history.append(theta_e)

    @staticmethod
    def plot_cte(dt=0.05):
        """
        Plot the stored cross track errors over time.
        """
        if len(PlotManager.cte_history) == 0:
            print("No CTE data to plot.")
            return

        time = np.arange(0, len(PlotManager.cte_history) * dt, dt)

        fig = _auto_fig()
        plt.figure(fig.number)
        plt.plot(time, PlotManager.cte_history, label='Cross Track Error')
        plt.xlabel('Time [s]')
        plt.ylabel('CTE [m]')
        plt.title('Cross Track Error Over Time')
        plt.grid(True)
        plt.legend()
        _maybe_show()

    @staticmethod
```

```
    def plot_path_deviation(cx, cy, states, full_path):
        x, y = zip(*full_path)
        x = np.array(x)
        y = np.array(y)

        fig = _auto_fig()
        plt.figure(fig.number)
        plt.plot(cx, -cy, label="Planned Path", linestyle="--", color="r")
        plt.plot(states.x, states.y, label="Actual Path", linestyle="-",
color="b")
        plt.plot(x, -y, label='planned path', linestyle="-", color="g",
markersize=1)
        plt.title("Path Deviation")
        plt.xlabel("X [m]")
        plt.ylabel("Y [m]")
        plt.legend()
        plt.grid()
        _maybe_show()

    @staticmethod
    def plot_path_deviation1(cx, cy, states, X,Y,cones_by_type,cones_lidar):
        fig = _auto_fig()
        # x , y = zip(*full_path)
        # x , y = full_path[0],full_path[1]
        # x=full_path[:,1]
        # y=full_path[:,2]
        # Y=float(Y)
        # Y=Y*-1
        # sorted(x)
        # print("this is cx ",cx)
        # print("this is x ",x)
        # list(set(X))
        # list(set(Y))
        # X = np.unique(X, axis=0)
        # Y = np.unique(Y, axis=0)
        # cones_by_type = self.plot_data.cones_map
# If X (and similarly Y) is a list of 1D arrays, flatten it:
        X_flat = np.concatenate(X)
        Y_flat = np.concatenate(Y)
        actual_x = np.array(states.x)
        actual_y = np.array(states.y)
        # Determine the number of points to compare
        n_points = min(len(actual_x), len(X_flat))

        # Slice both planned and actual arrays to the same length
        planned_x_trunc = X_flat[:n_points]
        planned_y_trunc = Y_flat[:n_points]
        actual_x_trunc  = actual_x[:n_points]
```

```
        actual_y_trunc  = actual_y[:n_points]

        # Calculate the MSE
        mse = np.mean((actual_x_trunc - planned_x_trunc)**2 + (actual_y_trunc
- planned_y_trunc)**2)
        mse = mse/100
        rmse = np.sqrt(mse)

        print("MSE:", mse)


        # plt.figure()
        # plt.plot(cx, -cy, label="Planned Path", linestyle="--", color="r")
        plt.plot(states.x, states.y, label="Actual Path", linestyle="-",
color="b")
        PlotManager.plot_cones(cones_by_type, cones_lidar)
        # plt.plot (X,Y,label ='planned path combined',linestyle="-",
color="g", markersize=1)
        plt.plot(X_flat, Y_flat, label='planned path combined', linestyle="-",
color="g", markersize=1)
        plt.title(f"Path Deviation (MSE: {mse:.2f}, RMSE: {rmse:.2f})")
        # plt.text(0.05, 0.95, f"MSE: {mse:.2f}",
transform=plt.gca().transAxes,
        #  fontsize=12, verticalalignment='top')
        # plt.text(0.65, 0.65, f"RMSE: {rmse:.2f}",
transform=plt.gca().transAxes,
        #  fontsize=12, verticalalignment='top')


        # plt.title("Path Deviation")
        plt.xlabel("X [m]")
        plt.ylabel("Y [m]")
        plt.legend()
        plt.grid()
        plt.legend(loc='lower left')
        # plt.show()
        _maybe_show()

    @staticmethod
    def plot_speed_profile(states, dt=conf.dt):
        min_length = min(len(states.v), len(states.v_log))
        time = np.linspace(0, (min_length - 1) * dt, min_length)
        Target_speed_time = np.full_like(time, conf.TARGET_SPEED)

        fig = _auto_fig()
        plt.figure(fig.number)
        plt.plot(time, states.v[:min_length], label="Actual Speed [m/s]",
color='blue')
```

```
        plt.plot(time, states.v_log[:min_length], label="Target Speed (v_log)
[m/s]", linestyle="--", color='red')
        plt.plot(time, Target_speed_time, label="Target Speed (Target Speed)
[m/s]", linestyle="dashdot", color='Black',markersize = 5)
        plt.title("Speed Profile")
        plt.xlabel("Time [s]")
        plt.ylabel("Speed [m/s]")
        plt.legend()
        plt.grid()
        _maybe_show()

    @staticmethod
    def plot_control_inputs(states, dt=conf.dt):
        min_length = min(len(states.t), len(states.steering),
len(states.acceleration))
        time = np.linspace(0, (min_length - 1) * dt, min_length)

        fig = _auto_fig()
        plt.figure(fig.number)
        plt.plot(time, states.steering[:min_length], label="Steering Angle
[rad]", color='green')
        plt.plot(time, states.acceleration[:min_length], label="Acceleration
command", color='orange')
        plt.title("Control Inputs Over Time")
        plt.xlabel("Time [s]")
        plt.ylabel("Control Input")
        plt.legend()
        plt.grid()
        _maybe_show()


    @staticmethod
    def plot_point_cloud(points, title="3D Lidar Point Cloud"):
        """
        Visualize a 3D point cloud.
        """
        fig = _auto_fig()
        ax = fig.add_subplot(211, projection='3d')

        xs = points[:, 0]
        ys = points[:, 1]
        zs = points[:, 2]

        ax.scatter(xs, ys, zs, c='b', marker='.', s=1)

        ax.set_xlabel('X')
        ax.set_ylabel('Y')
        ax.set_zlabel('Z')
        ax.set_title(title)
```

```
        _maybe_show()

    @staticmethod
    def plot_acceleration(states, dt=conf.dt):
        print("Plotting accelerations...")
        # print("Available attributes:", dir(states))  # Debug line

    # If states has direct acceleration arrays (like in plot_gg)
        if hasattr(states, 'a_longitudinal') and hasattr(states, 'a_lateral'):
            # Use existing acceleration data
            longitudinal_accel = np.array(states.a_longitudinal)
            lateral_accel = np.array(states.a_lateral)

            # Calculate total acceleration magnitude
            total_accel = np.sqrt(longitudinal_accel**2 + lateral_accel**2)

            min_length = len(total_accel)
            time = np.linspace(0, (min_length - 1) * dt, min_length)

            fig = _auto_fig()
            plt.figure(fig.number)
            plt.plot(time, longitudinal_accel, label="Longitudinal
Acceleration [m/s²]", color='green')
            plt.plot(time, lateral_accel, label="Lateral Acceleration [m/s²]",
color='orange')
            plt.plot(time, total_accel, label="Acceleration magnitude [m/s²]",
color='red')
            plt.title("Accelerations Over Time")
            plt.xlabel("Time [s]")
            plt.ylabel("Acceleration")
            plt.legend()
            plt.grid()
            _maybe_show()
        else:
            print("No acceleration data available in states object")
    @staticmethod
    def plot_gg(states, dt=conf.dt):
        print("Plotting GG Diagram clearly with stored numeric data...")

        longitudinal_accel = np.array(states.a_longitudinal)/9.81
        lateral_accel = np.array(states.a_lateral)/9.81

        fig = _auto_fig()
        plt.figure(fig.number, figsize=(8, 8))
        plt.scatter(lateral_accel, longitudinal_accel, s=5, c='blue',
alpha=0.5)
```

```
        max_accel = max(np.max(np.abs(longitudinal_accel)),
np.max(np.abs(lateral_accel))) + 1
        plt.xlim(-max_accel, max_accel)
        plt.ylim(-max_accel, max_accel)

        plt.xlabel("Lateral Acceleration [m/s²]")
        plt.ylabel("Longitudinal Acceleration [m/s²]")
        plt.title("GG Diagram (Friction Circle)")
        plt.axhline(0, color='black', linewidth=0.5)
        plt.axvline(0, color='black', linewidth=0.5)
        plt.grid(True)
        plt.gca().set_aspect('equal', adjustable='box')
        _maybe_show()

    @staticmethod
    def plot_all_accelerations(states, dt=conf.dt):
        print("Plotting longitudinal, lateral, and angular accelerations
explicitly...")

        min_length = min(len(states.a_longitudinal), len(states.a_lateral),
len(states.v_angular))
        time = np.linspace(0, (min_length - 1) * dt, min_length)


        # Try different possible attribute names for Vector3r
        angular_velocity = np.array([v.z_val for v in states.v_angular])
        angular_acceleration = np.gradient(angular_velocity, dt)

        fig = _auto_fig()
        plt.figure(fig.number, figsize=(12, 6))
        plt.plot(time, states.a_longitudinal[:min_length], label="Longitudinal
Acceleration [m/s²]", color='green')
        plt.plot(time, states.a_lateral[:min_length], label="Lateral
Acceleration [m/s²]", color='blue')
        plt.plot(time, angular_acceleration[:min_length], label="Angular
Acceleration [rad/s²]", color='orange')

        plt.title("Longitudinal, Lateral, and Angular Accelerations Over
Time")
        plt.xlabel("Time [s]")
        plt.ylabel("Acceleration")
        plt.legend()
        plt.grid(True)
        _maybe_show()

    @staticmethod
    def plot_point_cloud(points, title="3D Lidar Point Cloud"):
        """
```

```
        Visualize a 3D point cloud.
        """
        fig = _auto_fig()
        ax = fig.add_subplot(211, projection='3d')

        xs = points[:, 0]
        ys = points[:, 1]
        zs = points[:, 2]

        ax.scatter(xs, ys, zs, c='b', marker='.', s=1)

        ax.set_xlabel('X')
        ax.set_ylabel('Y')
        ax.set_zlabel('Z')
        ax.set_title(title)
        _maybe_show()


    def export_to_video(self, output_path, dpi=150, fps=12):
        """
        Exports frames using OpenCV VideoWriter.
        """
        fourcc = cv2.VideoWriter_fourcc(*'mp4v')
        writer = cv2.VideoWriter(output_path, fourcc, fps,
(self.frames[0].shape[1],self.frames[0].shape[0]))
        for frame in self.frames:
            # cv2.imshow("frame2", frame)
            # cv2.waitKey(1)
            writer.write(frame)
        print(f"Video saved to {output_path}")
        writer.release()
```

10.15. Apendix O – Midterm report feedback

הערות בודק לדו"ח אמצע

מספר פרויקט: 25-87

שם הפרויקט: Designing and Implementing a high-level path planning and control system for an autonomous racer

שם הסטודנטים: יובל גרימברג

שמות המנחים: ד"ר יגאל ביליק

התייחסות כללית

מהדוח עולה שנעשה עבודה טובה בבחינת חלופות לשני חלקי הפרויקט (אופן מציאת המסלול אוטומטית ואופן ביצוע הבקרה על העקיבה אחריו), אף על פי שאף החלטה עדיין לא גובתה בבחינה אנליטית/ ניסיונית

כדי לאמת ברמה מסוימת את עדיפותה. מבחינת הכתוב בדו״ח, הדו״ח סוטה מאוד מהנדרש ממנו הן מבחינת פורמט שלא עומד כמעט ובכלל למוגדר ולמצופה מהסטודנט, הן מבחינת המידע שמוצג בו שבחלקו עודף בתוכן מיותר ובחלקו האחר מדלג על הסברים ברורים. בנוסף, חלקים שלמים של הדו״ח כתובים כרשימות, דבר שלא רק לא תקין אלא גם הופך את הקריאה שלו למעיקה ומורידה מהבנת הנעשה בפרויקט. לדו״ח נדרש שיפוץ מהיסוד, על מנת שיהיה תקין בהגשה הסופית בסוף סמסטר הבא.

הערות לפי פרקים

דף שער

- חסר שער בעברית.

תקציר

- חסר תקציר בעברית.
- תקציר טוב מאוד.

תוכן עניינים, רשימת טבלאות, רשימת איורים ורשימת סימנים

- ברשימת האיורים חלק מהכיתובים מודגשים ואין אחידות.
- המוצג בתוכן העניינים נדרש להתחיל מהמבוא ולא לכלול את התקציר ואת הרשימות השונות. כנ״ל גם למספור של הדו״ח.

הערות עריכה כלליות

- חסרה הערכה כלכלית לפרויקט. גם העבודה של הסטודנט צריכה להיכלל בה וגם עלות הרישיונות לשימוש בתוכנות השונות ששימושו את הפרויקט.
- המספור של הדו״ח נדרש להתחיל במבוא ולא בתקציר. לפני המבוא צריך להיות מספור רומי.
- אורך הדו״ח חורג בארבעה עמודים משהוגדר.
- הכותרות בדו״ח לא לפי פורמט.
- הטקסט לא מיושר לשני הצדדים לפי פורמט.
- חלק מהטקסט מוסט באופן לא אחדי ולא תקין.
- עודף של איורים שלא מוסיף מידע כלל לקורא ולכן מיותרים.
- מספור המשוואות לא ממורכז למשוואות.

פרק 1 - Introduction

- המבוא ארוך משמעותי מהרצוי, כי הוא כולל עודף של מידע מיותר עבור הפרויקט הזה. בעוד המוטיבציה הכללית היא בניית מכלול בקרה לרכב המירוץ, אין צורך להפריז בהרחבה על התחרויות המירוץ מעבר לרלוונטי לפרויקט (המקצים להם הבקרה צריכה לענות והמגבלות הרלוונטיות למערך הבקרה). בכללי חלוקה של הבוא לתתי כותרות גם היא מיותרת ויש להציג את כלל המידע הנדרש כאן בצורה סיפורית ולא רשימות (המידע הרלוונטי הוא מוטיבציה כללי, מטרות ישירות, גבולות הפרויקט ו"ספוילר" לנעשה בפרויקט).

- לא מכניסים איור ללא התייחסות לדו"ח כי הוא כנראה מיותר.

- גם טבלת הציון בתחרות מיותרת במיוחד אחרי שהוגדר כי לא תספיקו לעמוד ביעד של שילוב הבקרה ברכב המעשי וההתמקדות הסימולציות.

- פירוט הקבוצה BGRacing מיותר. ההתייחסות הזו לקבוצה נכונה רק בחלק של התודות.

- איור 3 גם חורג מגבולות העמוד, מטושטש ולא רלוונטי.

- "…details of each dynamic event**s**…"

- ניתן להסתפק בתיאור שני המקצים הראשונים (Autodrive ו- Trackdrive) שלהבנתי הם היחידים שרלוונטיים לפרויקט הזה. השאר מיותרים.

- יש בולט מיותר בהגדרת הגבולות לפרויקט.

פרק 2 – Literature Review and Theoretical Background

- אם LiDAR הם ראשי תיבות יש לתת את הפירוש שלהם בטקסט ולא רק בכותרת, וגם לשמור על אחידות של רישומם עם אותיות לאורך כל המלל ולא רק פעם אחת.

- "We use The Innoviz one lidar that a solid state lidar that…" מה זה אומר ש- LiDAR הוא מצב מוצק?

- עודף של מונחים שאינם שמות שמסומנים עם אות ראשונה גדולה כנגד לחוקי הדקודוק של השפה האנגלית. אם הסטודנט לא יכול לשמור על רמת גבוהה והמנחה לא תומך בו עם עריכה לשוני ראויה, מיותר לחלוטין להגיש את הדו"ח באנגלית.

- מה אמורים להבין מאיור 8? אין אליו התייחסות בטקסט והמוצג לו לא מתקשר למה שנאמר לפניו כדי להבין אותו.

- כשמפנים למקור, גם באיורים, רק שרושמים את מספר ההפניה מתוך רשימת המקורות ולא את שמה המלא.

- מיותר ומעיק שניתן לכל שתי משפטים תת-כותרת.

- איור 10 גם באיכות ירודה וגם לא מוסבר כלל. אם נעשה שימוש בעיקרון השערוך שמוצג באיור בפרויקט, העקרון צריך להיות ברור.

- אני מאמין שיש טעות במשוואה (1). הנגזרת ל- x זהה לנגזרת ל- y?

פרק 3 – System requirements

- מובן ותיקן.

פרק 4 – System Architecture

- אחרי הכותרת חייב לבוא מלל, לא כותרת אחרת, איור או טבלה.

- תת הכותרת "Overview of the Architecture" מיותרת. בפרק היה צריך להתחיל עם תיאור הרכב המערכת ולהסתיים עם איור הבלוקים.

- "This chapter provides a shallow high-level overview" מה 'high-level' אומר בהקשר הזה.

בכללי לא הוגדר מה בדיוק 'high-level' בהקשר של הפרויקט. מציאת המסלול איכותית? היא מבוססת על מבט על? אם הכוונה לאיכותית, זה לא עולה מה השיטות והמודלים שנסקרו לפני למציאת המסלול או לבקרה.

פרק 5 – Implementation Details

- "To sun things…".

- "The controllers output looks like this where δ is…" נראה כמו מה? למה אתה מתכוון פה? בנוסף משוואות (7) ו- (8) לא ברורות/נכונות (בראשונה לא ברורה הכוונה ובשנייה אין יחידות אז זה יכול להיות 1 דקה/שעה/שנה.

פרק 6 – Analysis and Validation Plan

- למה לשמור על מהירות הרצויה כמהירות המקסימלית בנסיעה ישרה ולא כמהירות הממוצעת?

פרק 7 – Summary

- סיכום לא כתוב כנדרש.

פרק 8 – Plans and Risk Analysis

- תקין.

תקציר

פרויקט זה עוסק בתכנון מערכת בקרה ותכנון מסלול עילית ברכב מירוץ אוטונומי במסגרת נבחרת הפורמולה סטודנטים של בן גוריון מטרת העל של הפרויקט זה לייצר מערכת עמידה של בקרה ותכנון מסלול שתאפשר ניווט מדויק וביצועים מספקים תחת אילוצי התחרות. כדי לוודא את תקינות המערכת, הושמש סימולטור מבוסס AirSim אשר תוכנן במיוחד לתחרות, סימולטור זה עוזר לנו לבדוק את המערכת שלנו בשלל תנאים ומשם לעבור למימוש על הרכב האמיתי. הפרויקט הזה הוא לא רק הזדמנות להציג חדשנות הנדסית אלא גם פלטפורמה מצוינת לסטונדטים להפוך את מה שנלמד בתאוריה למציאות ובעצם להיות פורצי דרך בתחום הנהיגה האוטונומית.

פרויקט זה מתמקד בעיקר במימוש האלגוריתם רק בסביבת הסימולטור מכיוון שלא ניתן להבטיח מימוש מלא ברכב עקב אילוצים הנדסיים חיצוניים, אך נכון לרגע כתיבת דוח זה רוב המערכת כבר שולבה ברכב הניסוי האוטונומי והיא בימים אלה עוברת בדיקות נוספות על מנת שהתוצאות שמוצגות כאן בפרויקט יתורגמו לביצועי אמת בעולם האמיתי.

מטרת העל של הפרויקט הזה ושל הקבוצה עצמה זה לייצר את רכב הסטודנטים החשמלי האוטונומי הראשון בישראל שיוכל להתחרות בחול וכרגע מסתמן שדבר זה הולך להצליח.





אוניברסיטת בן גוריון בנגב
הפקולטה למדעי ההנדסה

המחלקה להנדסת מכונות

דוח סופי – תכנון ומימוש של מערכת בקרה ותכנון מסלול עילית ברכב מירוץ אוטונומי.

שם הסטודנט – יובל גרימברג 315823328

25-87

12/06/25

מנחה : דר' יגאל בילק